

OpenVLA: An Open-Source Vision-Language-Action Model

OpenVLA: 一个开源的视觉-语言-动作模型

> Author

<https://openvla.github.io>

<https://openvla.github.io>

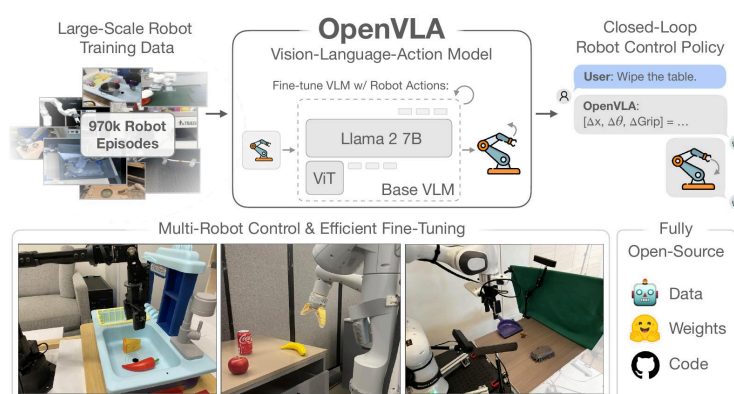


Figure 1: We present OpenVLA, a 7B-parameter open-source vision-language-action model (VLA), trained on 970k robot episodes from the Open X-Embodiment dataset [1]. OpenVLA sets a new state of the art for generalist robot manipulation policies. It supports controlling multiple robots out of the box and can be quickly adapted to new robot domains via parameter-efficient fine-tuning. The OpenVLA checkpoints and PyTorch training pipeline are fully open-source and models can be downloaded and fine-tuned from HuggingFace.

图1：我们展示了OpenVLA，一个70亿参数的开源视觉-语言-动作模型（VLA），在来自Open X-Embodiment数据集的97万个机器人episode上进行训练[1]。OpenVLA为通用机器人操作策略设定了一个新的技术水平。它支持开箱即用控制多个机器人，并且可以通过参数高效的微调快速适应新的机器人领域。OpenVLA检查点和PyTorch训练管道是完全开源的，模型可以从HuggingFace下载和微调。

Abstract: Large policies pretrained on a combination of Internet-scale visionlanguage data and diverse robot demonstrations have the potential to change how we teach robots new skills: rather than training new behaviors from scratch, we can fine-tune such vision-language-action (VLA) models to obtain robust, generalizable policies for visuomotor control. Yet, widespread adoption of VLAs for robotics has been challenging as 1) existing VLAs are largely closed and inaccessible to the public, and 2) prior work fails to explore methods for efficiently fine-tuning VLAs for new tasks, a key component for adoption. Addressing these challenges, we introduce OpenVLA, a 7B-parameter open-source VLA trained on a diverse collection of 970k real-world robot demonstrations. OpenVLA builds on

a Llama 2 language model combined with a visual encoder that fuses pretrained features from DINOv2 and SigLIP. As a product of the added data diversity and new model components, OpenVLA demonstrates strong results for generalist manipulation, outperforming closed models such as RT-2-X (55B) by 16.5% in absolute task success rate across 29 tasks and multiple robot embodiments, with 7x fewer parameters. We further show that we can effectively fine-tune OpenVLA for new settings, with especially strong generalization results in multi-task environments involving multiple objects and strong language grounding abilities, and outperform expressive from-scratch imitation learning methods such as Diffusion Policy by 20.4%. We also explore compute efficiency; as a separate contribution, we show that OpenVLA can be fine-tuned on consumer GPUs via modern low-rank adaptation methods and served efficiently via quantization without a hit to downstream success rate. Finally, we release model checkpoints, fine-tuning notebooks, and our PyTorch codebase with built-in support for training VLAs at scale on Open X-Embodiment datasets.

摘要：在互联网规模的视觉语言数据和各种机器人演示上预训练的大型策略有可能改变我们教导机器人新技能的方式：我们可以微调这种视觉-语言-动作

（VLA）模型，而不是从头开始训练新的行为，从而获得用于视觉运动控制的稳健、可泛化的策略。然而，VLA在机器人技术中的广泛应用一直具有挑战性，因为1）现有的VLA在很大程度上是封闭的，公众无法访问；2）先前的工作未能探索有效微调VLA以适应新方法，这是采用的关键组成部分。为了应对这些挑战，我们推出了OpenVLA，这是一个70亿参数的开源VLA，它是在97万个真实世界机器人演示的多样化集合上训练的。OpenVLA建立在Llama 2语言模型的基础上，并结合了一个视觉编码器，该编码器融合了来自DINOv2和SigLIP的预训练特征。作为增加数据多样性和新模型组件的产物，OpenVLA在通用操作方面表现出强大的结果，在29个任务和多个机器人形态中，其绝对任务成功率超过了RT-2-X（55B）等封闭模型16.5%，而参数减少了7倍。我们进一步表明，我们可以有效地微调OpenVLA以适应新的设置，尤其是在涉及多个对象和强大的语言基础能力的多任务环境中，具有强大的泛化结果，并且优于从头开始的表达性模仿学习方法，例如Diffusion Policy，达到20.4%。我们还探讨了计算效率；作为一项单独的贡献，我们表明OpenVLA可以通过现代低秩适应方法在消费级GPU上进行微调，并通过量化有效地提供服务，而不会影响下游成功率。最后，我们发布了模型检查点、微调笔记本和我们的PyTorch代码库，其中内置了对在Open X-Embodiment数据集上大规模训练VLA的支持。

1 Introduction

1 引言

A key weakness of learned policies for robotic manipulation is their inability to generalize beyond their training data: while existing policies trained for individual skills or language instructions have the capacity to extrapolate behaviors to new

initial conditions such as object positions or lighting [2, 3], they lack robustness to scene distractors or novel objects [4, 5] and struggle to execute unseen task instructions [6, 7]. Yet beyond robotics, existing foundation models for vision and language such as CLIP [8], SigLIP [9], and Llama 2 [10] are capable of these types of generalization and more, stemming from the priors captured by their Internet-scale pretraining datasets. While reproducing this scale of pretraining for robotics is still an open challenge — even the largest robot manipulation datasets [1, 11] only have 100K to 1M examples – this imbalance suggests an opportunity: using existing foundation models for vision and language *as a core building block* for training robotic policies that can generalize to objects, scenes, and tasks beyond their training data.

机器人操作的学习策略的一个主要弱点是它们无法推广到训练数据之外：虽然现有的针对个体技能或语言指令训练的策略有能力将行为外推到新的初始条件，如物体位置或光照[2, 3]，但它们对场景干扰物或新物体的鲁棒性不足[4, 5]，并且难以执行未见过的任务指令[6, 7]。然而，除了机器人技术之外，现有的视觉和语言基础模型，如CLIP [8]、SigLIP [9]和Llama 2 [10]，都能够实现这些类型的泛化以及更多，这源于它们互联网规模的预训练数据集所捕获的先验知识。虽然为机器人技术重现这种规模的预训练仍然是一个开放的挑战——即使是最大的机器人操作数据集[1, 11]也只有10万到100万个例子——这种不平衡表明了一个机会：使用现有的视觉和语言基础模型作为训练机器人策略的核心构建块，这些策略可以推广到训练数据之外的物体、场景和任务。

Towards this goal, existing work has explored integrating pretrained language and vision-language models for robotic representation learning [12–14] and as a component in modular systems for task planning and execution [15, 16]. More recently, they have been used for directly learning visionlanguage-action models [VLAs; 1, 7, 17, 18] for control. VLAs provide a direct instantiation of using pretrained vision-and-language foundation models for robotics, directly fine-tuning visuallyconditioned language models (VLMs) such as PaLI [19, 20] to generate robot control actions. By building off of strong foundation models trained on Internet-scale data, VLAs such as RT-2 [7] demonstrate impressive robustness results, as well as an ability to generalize to novel objects and tasks, setting a new standard for generalist robot policies. Yet, there are two key reasons preventing the widespread use of existing VLAs: 1) current models [1, 7, 17, 18] are *closed*, with limited visibility into model architecture, training procedures, and data mixture, and 2) existing works do not provide best practices for *deploying and adapting* VLAs to new robots, environments, and tasks — especially on commodity hardware (e.g., consumer-grade GPUs). We argue that to develop a rich foundation for future research and development, robotics needs open-source, generalist VLAs that support effective fine-tuning and adaptation, akin to the existing ecosystem around open-source language models [21–24].

为了实现这一目标，现有的工作已经探索了整合预训练的语言和视觉-语言模型，用于机器人表征学习[12 - 14]，并将其作为模块化系统中任务规划和执行的组件[15, 16]。最近，它们已被用于直接学习视觉语言-动作模型[VLA; 1, 7, 17, 18]以进行控制。VLA直接实例化了使用预训练的视觉和语言基础模型进行机器人技术，直接微调视觉条件语言模型（VLM），例如PaLI [19, 20]，以生成机器人控制动作。通过建立在经过互联网规模数据训练的强大基础模型之上，RT-2等VLA [7] 展示了令人印象深刻的鲁棒性结果，以及泛化到新颖物体和任务的能力，为通用机器人策略设定了新的标准。然而，有两个关键原因阻碍了现有VLA的广泛使用：1）当前的模型[1, 7, 17, 18]是封闭的，对模型架构、训练程序和数据混合的可见性有限；2）现有的工作没有提供将VLA部署和适应到新机器人、环境和任务的最佳实践——尤其是在商品硬件（例如，消费级GPU）上。我们认为，为了为未来的研究和开发奠定丰富的基础，机器人技术需要开源的、通用的VLA，以支持有效的微调和适应，类似于围绕开源语言模型[21 - 24]的现有生态系统。

To this end, we introduce OpenVLA, a 7B-parameter open-source VLA that establishes a new state of the art for generalist robot manipulation policies.¹ OpenVLA consists of a pretrained visually-conditioned language model backbone that captures visual features at multiple granularities, fine-tuned on a large, diverse dataset of 970k robot manipulation trajectories from the Open-X Embodiment [1] dataset — a dataset that spans a wide range of robot embodiments, tasks, and scenes. As a product of increased data diversity and new model components, OpenVLA outperforms the 55B-parameter RT-2-X model [1, 7], the prior state-of-the-art VLA, by 16.5% absolute success rate across 29 evaluation tasks on the WidowX and Google Robot embodiments. We additionally investigate *efficient fine-tuning strategies for VLAs*, a new contribution not explored in prior work, across 7 diverse manipulation tasks spanning behaviors from object pick-and-place to cleaning a table. We find that fine-tuned OpenVLA policies clearly outperform fine-tuned pretrained policies such as Octo [5]. Compared to from-scratch imitation learning with diffusion policies [3], fine-tuned OpenVLA shows substantial improvement on tasks involving grounding language to behavior in multi-task settings with multiple objects. Following these results, we are the first to demonstrate the effectiveness of compute-efficient fine-tuning methods leveraging low-rank adaptation [LoRA; 26] and model quantization [27] to facilitate adapting OpenVLA models on consumer-grade GPUs instead of large server nodes without compromising performance. As a final contribution, we open-source all models, deployment and fine-tuning notebooks, and the OpenVLA codebase for training VLAs at scale, with the hope that these resources enable future work exploring and adapting VLAs for robotics.

为此，我们引入了 OpenVLA，一个 70 亿参数的开源 VLA，它为通用机器人操作策略建立了新的技术水平。¹ OpenVLA 由一个预训练的视觉条件语言模型主干组

成，该主干捕获多个粒度的视觉特征，并在来自 Open-X Embodiment [1] 数据集的 97 万条机器人操作轨迹的大型多样化数据集上进行微调——该数据集涵盖了广泛的机器人形态、任务和场景。由于数据多样性的增加和新的模型组件，OpenVLA 在 WidowX 和 Google Robot 形态的 29 个评估任务中，比之前的最先进的 VLA，即 550 亿参数的 RT-2-X 模型 [1, 7]，的绝对成功率高出 16.5%。此外，我们还研究了 VLA 的高效微调策略，这是先前工作中未探索的一项新贡献，涵盖了从物体拾取和放置到清洁桌面的 7 个不同的操作任务。我们发现，经过微调的 OpenVLA 策略明显优于经过微调的预训练策略，例如 Octo [5]。与从头开始使用扩散策略进行模仿学习 [3] 相比，经过微调的 OpenVLA 在涉及将语言与多任务环境中多个物体的行为联系起来的任务中显示出显著改进。根据这些结果，我们率先证明了利用低秩自适应 [LoRA; 26] 和模型量化 [27] 的计算高效微调方法的有效性，以促进在消费级 GPU 上而不是大型服务器节点上调整 OpenVLA 模型，而不会影响性能。作为最后的贡献，我们开源了所有模型、部署和微调笔记本，以及用于大规模训练 VLA 的 OpenVLA 代码库，希望这些资源能够促进未来的工作，探索和调整用于机器人的 VLA。

2 Related Work

2 相关工作

Visually-Conditioned Language Models Visually-conditioned language models (VLMs), which are trained on Internet-scale data to generate natural language from input image(s) and language prompts, have been adopted for myriad applications from visual question answering [28–31] to object localization [32, 33]. One of the key advances fueling recent VLMs are model architectures that bridge features from pretrained vision encoders [8, 9, 25] with pretrained language models [10, 23, 34–36], directly building on advances in both computer vision and natural language modelling to create powerful multimodal models. While early work explored various architectures for cross-attending between vision and language features [37–41], new open-source VLMs [20, 42–44] have converged on a simpler “patch-as-token” approach, in which patch features from pretrained visual transformers are treated as tokens, and are then projected into the input space of a language model. This simplicity makes it easy to repurpose existing tools for training language models at scale for VLM training. We employ these tools in our work to scale VLA training, and specifically use VLMs from Karamcheti et al. [44] as our pretrained backbone, as they are trained from *multi-resolution visual features*, fusing low-level spatial information from DINOv2 [25] with higher-level semantics from SigLIP [9] to aid in visual generalization.

视觉条件语言模型 视觉条件语言模型（VLMs）在互联网规模的数据上进行训练，以从输入图像和语言提示生成自然语言，已被广泛应用于从视觉问答 [28–31] 到对象定位 [32, 33] 的各种应用。推动近期 VLM 发展的关键进步之一是模型架构，该架构将来自预训练视觉编码器的特征 [8, 9, 25] 与预训练语言模

型 [10, 23, 34 - 36] 连接起来, 直接建立在计算机视觉和自然语言建模的进步之上, 以创建强大的多模态模型。虽然早期的工作探索了用于视觉和语言特征之间交叉注意的各种架构 [37 - 41], 但新的开源 VLM [20, 42 - 44] 已经趋向于一种更简单的“patch-as-token”方法, 其中来自预训练视觉转换器的 patch 特征被视为 tokens, 然后被投影到语言模型的输入空间中。这种简单性使得为 VLM 训练大规模地重新利用现有工具来训练语言模型变得容易。我们在我们的工作中采用这些工具来扩展 VLA 训练, 并且特别使用来自 Karamcheti et al. [44] 的 VLM 作为我们的预训练骨干, 因为它们是从多分辨率视觉特征训练的, 融合了来自 DINOv2 [25] 的低级空间信息和来自 SigLIP [9] 的高级语义信息, 以帮助视觉泛化。

Generalist Robot Policies A recent trend in robotics works towards training multi-task “generalist” robot policies [2, 6, 45–49] on large diverse robot datasets [1, 2, 6, 11, 45, 49–56], spanning many different robot embodiments [1, 5, 53, 57–66]. Notably, Octo [5] trains a generalist policy that can control multiple robots out-of-the-box and allows for flexible fine-tuning to new robot setups. A key difference between these approaches and OpenVLA is the model architecture. Prior works like Octo typically compose pretrained components such as language embeddings or visual encoders with additional model components initialized from scratch [2, 5, 6], learning to “stitch” them together during the course of policy training. Unlike these works, OpenVLA adopts a more end-to-end approach, directly fine-tuning VLMs to generate robot actions by treating them as tokens in the language model vocabulary. Our experimental evaluation shows that this simple yet scalable pipeline substantially boosts performance and generalization ability over prior generalist policies.

通用机器人策略：机器人领域的一个最新趋势是训练多任务“通用”机器人策略 [2, 6, 45 - 49], 使用大型多样化的机器人数据集 [1, 2, 6, 11, 45, 49 - 56], 涵盖许多不同的机器人形态 [1, 5, 53, 57 - 66]。值得注意的是, Octo [5] 训练了一种通用策略, 可以开箱即用地控制多个机器人, 并允许灵活地对新的机器人设置进行微调。这些方法与 OpenVLA 之间的主要区别在于模型架构。像 Octo 这样的先前工作通常将预训练的组件 (如语言嵌入或视觉编码器) 与从头开始初始化的其他模型组件组合在一起 [2, 5, 6], 从而在策略训练过程中学习将它们“缝合”在一起。与这些工作不同, OpenVLA 采用了一种更端到端的方法, 通过将视觉语言模型 (VLM) 视为语言模型词汇表中的标记, 直接对视觉语言模型进行微调, 以生成机器人动作。我们的实验评估表明, 这种简单而可扩展的流程大大提高了性能和泛化能力, 优于先前的通用策略。

Vision-Language-Action Models A number of works have explored the use of VLMs for robotics, e.g., for visual state representations [12, 13], object detection [67], high-level planning [16], and for providing a feedback signal [68–71]. Others integrate VLMs directly into end-to-end visuomotor manipulation policies [14,

[15], but incorporate significant structure into the policy architecture or require calibrated cameras, which limits their applicability. A number of recent works have explored similar recipes to ours and directly fine-tuned large pretrained VLMs for predicting robot actions [1, 7, 17, 18, 72–74]. Such models are often referred to as vision-language-action models (VLAs), since they fuse robot control actions directly into VLM backbones. This has three key benefits: (1) it performs alignment of pretrained vision and language components on a large, Internetscale vision-language dataset, (2) the use of a generic architecture, not custom-made for robot control, allows us to leverage the scalable infrastructure underlying modern VLM training [75–77] and scale to training billion-parameter policies with minimal code modifications, and (3) it provides a direct pathway for robotics to benefit from the rapid improvements in VLMs. Existing works on VLAs either focus on training and evaluating in single robot or simulated setups [72–74, 78] and thus lack generality, or are closed and do not support efficient fine-tuning to new robot setups [1, 7, 17, 18].

视觉-语言-动作模型 许多工作探索了使用视觉-语言模型（VLM）进行机器人学研究，例如用于视觉状态表示 [12, 13]、目标检测 [67]、高级规划 [16] 以及提供反馈信号 [68 – 71]。其他研究将视觉-语言模型直接集成到端到端的视觉-运动操作策略中 [14, 15]，但策略架构中包含大量结构或需要校准的相机，这限制了其适用性。许多近期研究探索了与我们类似的方法，并直接微调了大型预训练视觉-语言模型以预测机器人动作 [1, 7, 17, 18, 72 – 74]。这类模型通常被称为视觉-语言-动作模型（VLA），因为它们将机器人控制动作直接融合到视觉-语言模型的骨干网络中。这有三个关键优势：（1）在大型、互联网规模的视觉-语言数据集上对预训练的视觉和语言组件进行对齐；（2）使用通用架构，而非为机器人控制定制的架构，使我们能够利用现代视觉-语言模型训练的可扩展基础设施 [75 – 77]，并以最小的代码修改扩展到训练数十亿参数的策略；（3）为机器人学直接受益于视觉-语言模型的快速改进提供了途径。现有的视觉-语言-动作模型研究要么专注于在单一机器人或模拟环境中进行训练和评估 [72 – 74, 78]，因此缺乏通用性，要么是闭源的，不支持对新机器人环境进行有效微调 [1, 7, 17, 18]。

Most closely related, RT-2-X [1] trains a 55B-parameter VLA policy on the Open X-Embodiment dataset and demonstrates state-of-the-art generalist manipulation policy performance. However, our work differs from RT-2-X in multiple important aspects: (1) by combining a strong open VLM backbone with a richer robot pretraining dataset, OpenVLA outperforms RT-2-X in our experiments while being an order of magnitude smaller; (2) we thoroughly investigate fine-tuning of OpenVLA models to new target setups, while RT-2-X does not investigate the fine-tuning setting; (3) we are the first to demonstrate the effectiveness of modern parameter-efficient fine-tuning and quantization approaches for VLAs; and (4) OpenVLA is the first generalist VLA that is open-source and thus supports future research on VLA training, data mixtures, objectives, and inference.

最密切相关的 RT-2-X [1] 在 Open X-Embodiment 数据集上训练了一个 55B 参数的 VLA 策略，并展示了最先进的通用操作策略性能。然而，我们的工作多个重要方面与 RT-2-X 不同：(1) 通过将强大的开放 VLM 主干与更丰富的机器人预训练数据集相结合，OpenVLA 在我们的实验中优于 RT-2-X，同时规模小了一个数量级；(2) 我们彻底研究了将 OpenVLA 模型微调到新的目标设置，而 RT-2-X 没有研究微调设置；(3) 我们是第一个证明现代参数高效微调 and 量化方法对 VLA 有效性的人；(4) OpenVLA 是第一个开源的通用 VLA，因此支持未来对 VLA 训练、数据混合、目标和推理的研究。

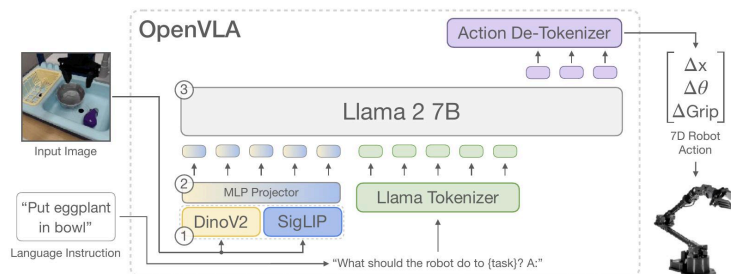


Figure 2: **OpenVLA model architecture.** Given an image observation and a language instruction, the model predicts 7-dimensional robot control actions. The architecture consists of three key components: (1) a **vision encoder** that concatenates Dino V2 [25] and SigLIP [79] features, (2) a **projector** that maps visual features to the language embedding space, and (3) the **LLM backbone**, a Llama 2 7B-parameter large language model [10].

图 2: OpenVLA 模型架构。给定图像观测和语言指令，该模型预测 7 维机器人控制动作。该架构由三个关键组件组成：(1) 一个视觉编码器，它连接 Dino V2 [25] 和 SigLIP [79] 特征，(2) 一个投影器，它将视觉特征映射到语言嵌入空间，以及 (3) LLM 主干，一个 Llama 2 7B 参数的大型语言模型 [10]。

3 The OpenVLA Model

3 OpenVLA 模型

We introduce the OpenVLA model, a 7B-parameter vision-language-action model (VLA) trained on 970k robot demonstrations from the Open X-Embodiment dataset [1]. There are many, largely unexplored, questions around best practices for developing VLA models, e.g., what are the best model backbones, datasets, and hyperparameters to use for training. Below, we detail our approach for developing OpenVLA and summarize our key learnings. Concretely, we first provide a brief overview of modern VLMs, which form the backbone of OpenVLA (Section 3.1); then describe our basic training recipe and dataset (Section 3.2 and Section 3.3); discuss key design decisions (Section 3.4); and provide details of the used infrastructure for training and inference (Section 3.5).

我们介绍了 OpenVLA 模型，这是一个拥有 70 亿参数的视觉-语言-动作模型 (VLA)，它在来自 Open X-Embodiment 数据集的 97 万个机器人演示数据上进行了训练 [1]。关于开发 VLA 模型的最佳实践，存在许多尚未被充分探索的问题，例如，用于训练的最佳模型骨干、数据集和超参数是什么。下面，我们将详

介绍我们开发 OpenVLA 的方法，并总结我们的主要经验。具体来说，我们首先简要概述了现代 VLM，它们构成了 OpenVLA 的骨干（第 3.1 节）；然后描述了我们的基本训练方法和数据集（第 3.2 节和第 3.3 节）；讨论了关键的设计决策（第 3.4 节）；并提供了用于训练和推理的基础设施的详细信息（第 3.5 节）。

3.1 Preliminaries: Vision-Language Models

3.1 预备知识：视觉-语言模型

The architecture of most recent VLMs [20, 42–44] consists of three main parts (see Fig. 2): (1) a **visual encoder** that maps image inputs to a number of “image patch embeddings”, (2) a **projector** that takes the output embeddings of the visual encoder and maps them into the input space of a language model, and (3) a **large language model (LLM) backbone**. During VLM training, the model is trained end-to-end with a next text token prediction objective on paired or interleaved vision and language data curated from various Internet sources.

大多数最新视觉语言模型（VLMs）的架构[20, 42 - 44]由三个主要部分组成（见图2）：（1）一个视觉编码器，将图像输入映射到若干个“图像块嵌入”；（2）一个投影器，获取视觉编码器的输出嵌入，并将它们映射到语言模型的输入空间；以及（3）一个大型语言模型（LLM）骨干。在视觉语言模型（VLM）训练期间，该模型通过在从各种互联网来源收集的配对或交错的视觉和语言数据上，使用下一个文本标记预测目标进行端到端训练。

In this work, we build on the Prismatic-7B VLM [44]. Prismatic follows the same standard architecture described above, with a 600M-parameter **visual encoder**, a small 2-layer MLP **projector**, and a 7B-parameter Llama 2 **language model backbone** [10]. Notably, Prismatic uses a *two-part* visual encoder, consisting of pretrained SigLIP [79] and DinoV2 [25] models. Input image patches are passed separately through both encoders and the resulting feature vectors are concatenated channel-wise. In contrast to the more commonly used vision encoders such as CLIP- [80] or SigLIP-only encoders, the addition of DinoV2 features has been shown to be helpful for improved spatial reasoning [44], which can be particularly helpful for robot control.

在这项工作中，我们以Prismatic-7B VLM [44]为基础构建。Prismatic遵循上述相同的标准架构，具有一个600M参数的视觉编码器、一个小的2层MLP投影器和一个7B参数的Llama 2语言模型主干[10]。值得注意的是，Prismatic使用一个由预训练的SigLIP [79]和DinoV2 [25]模型组成的两部分视觉编码器。输入图像块分别通过两个编码器，并将得到的特征向量按通道连接。与更常用的视觉编码器（如CLIP- [80]或仅SigLIP编码器）相比，添加DinoV2特征已被证明有助于改进空间推理[44]，这对于机器人控制尤其有帮助。

SigLIP, DinoV2, and Llama 2 do not release details about their training data, which likely consists of trillions of tokens of Internet-sourced image-text, image-only, and text-only data respectively. The Prismatic VLM is fine-tuned on top of these components using the LLaVA 1.5 data mixture [43], which contains a total of approximately 1M image-text and text-only data samples from open-source datasets [29, 42, 81–83].

SigLIP、DinoV2 和 Llama 2 没有公布关于其训练数据的细节，这些数据可能分别包含数万亿个来自互联网的图像-文本、仅图像和仅文本的数据 token。Prismatic VLM 在这些组件之上进行了微调，使用了 LLaVA 1.5 数据混合 [43]，其中包含总共约 100 万个来自开源数据集的图像-文本和仅文本数据样本 [29, 42, 81 – 83]。

3.2 OpenVLA Training Procedure

3.2 OpenVLA训练流程

To train OpenVLA, we fine-tune a pretrained Prismatic-7B VLM backbone for robot action prediction (see Fig. 2). We formulate the action prediction problem as a “vision-language” task, where an input observation image and a natural language task instruction are mapped to a string of predicted robot actions [7]. To enable the VLM’s language model backbone to predict robot actions, we represent the actions in the output space of the LLM by mapping continuous robot actions to discrete tokens used by the language model’s tokenizer. Following Brohan et al. [7], we discretize each dimension of the robot actions separately into one of 256 bins. For each action dimension, we set the bin width to uniformly divide the interval between the 1st and 99th quantile of the actions in the training data. Using quantiles instead of the min-max bounds Brohan et al. [7] used allows us to ignore outlier actions in the data that could otherwise drastically expand the discretization interval and reduce the effective granularity of our action discretization.

为了训练OpenVLA，我们对一个预训练的Prismatic-7B VLM主干网络进行微调，用于机器人动作预测（见图2）。我们将动作预测问题表述为一个“视觉-语言”任务，其中输入观察图像和自然语言任务指令被映射到预测的机器人动作字符串[7]。为了使VLM的语言模型主干网络能够预测机器人动作，我们通过将连续的机器人动作映射到语言模型分词器使用的离散token来表示LLM输出空间中的动作。遵循Brohan等人[7]的方法，我们将机器人动作的每个维度分别离散化为256个bin中的一个。对于每个动作维度，我们将bin宽度设置为均匀划分训练数据中动作的第1和第99个百分位数之间的间隔。使用分位数而不是Brohan等人[7]使用的最小值-最大值边界，使我们能够忽略数据中的异常值动作，否则这些异常值可能会极大地扩展离散化间隔并降低我们动作离散化的有效粒度。

Using this discretization, we obtain N discrete integers $\in [0 \dots 255]$ for an N - dimensional robot action. Unfortunately, the tokenizer used by OpenVLA’s

language backbone, the Llama tokenizer [10], only reserves 100 “special tokens” for tokens newly introduced during fine-tuning, which is too few for the 256 tokens of our action discretization. Instead, we again opt for simplicity and follow Brohan et al. [7]’s approach by simply overwriting the 256 *least used* tokens in the Llama tokenizer’s vocabulary (which corresponds to the last 256 tokens) with our action tokens. Once the actions are processed into a sequence of tokens, OpenVLA is trained with a standard next-token prediction objective, evaluating the cross-entropy loss on the predicted action tokens only. We discuss key design decisions for implementing this training procedure in Section 3.4. Next, we describe the robot dataset we use for OpenVLA training.

使用此离散化方法，我们为 N 维度的机器人动作获得了 N 个属于 $[0 \dots 255]$ 区间内的离散整数。不幸的是，OpenVLA 的语言主干所使用的 Llama 分词器 [10] 仅为微调过程中新引入的标记保留了 100 个“特殊标记”，这对于我们动作离散化中的 256 个标记来说远远不够。相反，我们再次选择简单性，并遵循 Brohan 等人 [7] 的方法，简单地用我们的动作标记覆盖 Llama 分词器词汇表中 256 个使用频率最低的标记（对应于最后的 256 个标记）。一旦动作被处理成一个标记序列，OpenVLA 就以标准的下一个标记预测目标进行训练，仅在预测的动作标记上评估交叉熵损失。我们在 3.4 节讨论了实现此训练过程的关键设计决策。接下来，我们描述用于 OpenVLA 训练的机器人数据集。

3.3 Training Data

3.3 训练数据

The goal in constructing the OpenVLA training dataset is to capture a large diversity of robot embodiments, scenes, and tasks. This enables the final model to control various robots out of the box *and* admits efficient fine-tuning to new robot setups. We leverage the Open X-Embodiment dataset [1] (OpenX) as a base to curate our training dataset. The full OpenX dataset, at the time of writing, consists of more than 70 individual robot datasets, with more than 2M robot trajectories, that were pooled into a coherent and easy-to-use data format in a large community effort. To make training on this data practical, we apply multiple steps of data curation to the raw dataset.

构建 OpenVLA 训练数据集的目标是捕捉机器人形态、场景和任务的巨大多样性。这使得最终模型能够开箱即用控制各种机器人，并允许对新的机器人设置进行高效的微调。我们利用 Open X-Embodiment 数据集 [1] (OpenX) 作为基础来管理我们的训练数据集。在撰写本文时，完整的 OpenX 数据集包含超过 70 个独立的机器人数据集，其中包含超过 200 万条机器人轨迹，这些轨迹在一个大型社区的努力下被汇集为连贯且易于使用的数据格式。为了使在此数据上的训练切实可行，我们对原始数据集应用了多个数据管理步骤。

The goals of this curation are to ensure (1) a coherent input and output space across all training datasets, and (2) a balanced mix of embodiments, tasks, and scenes in the final training mixture.² To address (1), we follow [1, 5] and restrict our training dataset to contain only manipulation datasets with at least one 3rd person camera and use single-arm end-effector control. For (2), we leverage the data mixture weights of Octo [5] for all datasets that pass the first round of filtering. Octo heuristically down-weights or removes less diverse datasets and up-weights datasets with larger task and scene diversity; see Octo Model Team et al. [5] for details.

本次整理的目标是确保：（1）所有训练数据集具有连贯的输入和输出空间；
（2）最终训练混合中，具身、任务和场景的均衡组合。² 为了解决（1），我们遵循[1, 5]，并将我们的训练数据集限制为仅包含至少一个第三人称相机且使用单臂末端执行器控制的操控数据集。对于（2），我们利用 Octo [5] 的数据混合权重来处理通过第一轮筛选的所有数据集。Octo 启发式地降低或移除多样性较差的数据集，并提高具有较大任务和场景多样性的数据集的权重；详细信息请参见 Octo Model Team et al. [5]。

We also experimented with incorporating a few additional datasets into our training mixture that were added to the OpenX dataset since the release of Octo, including the DROID dataset [11], although at a conservative mixture weight of 10%. In practice, we found that the action token accuracy on DROID remained low throughout training, suggesting a larger mixture weight or model may be required to fit its diversity in the future. To not jeopardize the quality of the final model, we removed DROID from the data mixture for the final third of training. We provide a complete overview of the used datasets and mixture weights in Appendix A.

我们还尝试将一些额外的数据集纳入到我们的训练混合中，这些数据集是在 Octo 发布后添加到 OpenX 数据集的，包括 DROID 数据集 [11]，尽管混合权重保守地设置为 10%。在实践中，我们发现 DROID 上的动作 token 准确率在整个训练过程中仍然很低，这表明可能需要更大的混合权重或模型才能适应其多样性。为了不影响最终模型的质量，我们在最后三分之一的训练中从数据混合中移除了 DROID。我们在附录 A 中提供了所用数据集和混合权重的完整概述。

3.4 OpenVLA Design Decisions

3.4 OpenVLA 设计决策

When developing the OpenVLA model, we explored various design decisions in smaller-scale experiments before starting the final model training run. Concretely, we trained and evaluated OpenVLA models on BridgeData V2 [6] for our initial experiments, instead of training on the full OpenX mixture, to increase iteration

speed and reduce computational cost. We summarize key learnings from these explorations below.

在开发 OpenVLA 模型时，我们在开始最终模型训练运行之前，在较小规模的实验中探索了各种设计决策。具体而言，为了提高迭代速度并降低计算成本，我们训练并评估了 BridgeData V2 [6] 上的 OpenVLA 模型来进行初始实验，而不是在完整的 OpenX 混合数据集上进行训练。我们在下面总结了从这些探索中获得的关键经验。

VLM Backbone. Initially, we experimented with multiple VLM backbones. Apart from Prismatic [44], we tested fine-tuning IDEFICS-1 [84] and LLaVA [85] for robot action prediction. We found that LLaVA and IDEFICS-1 performed comparably on tasks with only one object in the scene, but LLaVA demonstrated stronger language grounding in tasks that involved multiple objects in the scene and required the policy to manipulate the *correct* object, i.e., the object specified in the language instruction. Concretely, LLaVA improved upon IDEFICS-1 by 35% in absolute success rate, averaged across five language grounding tasks in a BridgeData V2 sink environment. The fine-tuned Prismatic VLM policy achieved further improvements, outperforming the LLaVA policy by roughly 10% in absolute success rate across both simple single-object tasks and multi-object, language grounding tasks. We attribute this performance delta to improved spatial reasoning capabilities afforded by the fused SigLIP-DinoV2 backbones (see Section 3.1). In addition to the performance enhancements, Prismatic also provides a modular and easy-to-use codebase, so we ultimately chose it to be the backbone for the OpenVLA model.

VLM主干网络。最初，我们尝试了多种VLM主干网络。除了Prismatic [44]，我们还测试了对IDEFICS-1 [84]和LLaVA [85]进行微调，以用于机器人动作预测。我们发现，LLaVA和IDEFICS-1在场景中只有一个对象的任务上表现相当，但LLaVA在涉及多个对象的场景中表现出更强的语言基础能力，并且要求策略操纵正确的对象，即语言指令中指定的对象。具体而言，LLaVA在BridgeData V2 sink环境中的五个语言基础任务中，绝对成功率平均提高了35%，优于IDEFICS-1。经过微调的Prismatic VLM策略取得了进一步的改进，在简单的单对象任务和多对象、语言基础任务中，绝对成功率都比LLaVA策略高出约10%。我们将这种性能差异归因于融合的SigLIP-DinoV2主干网络所提供的改进的空间推理能力（参见第3.1节）。除了性能增强之外，Prismatic还提供了一个模块化且易于使用的代码库，因此我们最终选择它作为OpenVLA模型的主干网络。

Image Resolution. The resolution of input images has significant impact on the computational requirements of VLA training, since higher-resolution images result in more image patch tokens and thus longer context lengths that quadratically increase training compute. We compared VLAs with 224×224 px and 384×384 px inputs, but found no performance difference in our evaluations, while the

latter takes 3x longer to train. We thus opt for a resolution of 224×224 px for the final OpenVLA model. Note that on many VLM benchmarks, increased resolution does improve performance [44, 86, 87], but we did not see this trend (yet) for VLAs.

图像分辨率。输入图像的分辨率对 VLA 训练的计算需求有显著影响，因为分辨率更高的图像会导致更多的图像块标记，从而导致更长的上下文长度，进而使训练计算量呈二次方增长。我们比较了 224×224 px 和 384×384 px 输入的 VLA，但在我们的评估中没有发现性能差异，而后的训练时间却长了 3 倍。因此，我们为最终的 OpenVLA 模型选择了 224×224 px 的分辨率。请注意，在许多 VLM 基准测试中，分辨率的提高确实能提高性能 [44, 86, 87]，但我们尚未在 VLA 中看到这种趋势。

Fine-Tuning Vision Encoder. Prior work on VLMs found that freezing vision encoders during VLM training typically leads to higher performance [44]. Intuitively, a frozen vision encoder may better preserve the robust features learned from its Internet-scale pretraining. However, we found fine-tuning the vision encoder during VLA training to be crucial for good VLA performance. We hypothesize that the pretrained vision backbone may not capture sufficient fine-grained spatial details about important parts of the scene to enable precise robotic control.

微调视觉编码器。先前关于视觉语言模型（VLM）的研究发现，在VLM训练期间冻结视觉编码器通常会导致更高的性能 [44]。直观地说，冻结的视觉编码器可能更好地保留从其互联网规模预训练中学习到的鲁棒特征。然而，我们发现，在视觉语言动作（VLA）训练期间微调视觉编码器对于良好的VLA性能至关重要。我们假设，预训练的视觉骨干网络可能无法捕捉到关于场景重要部分的足够精细的空间细节，从而实现精确的机器人控制。

Training Epochs. Typical LLM or VLM training runs complete at most one or two epochs through their training dataset. In contrast, we found it important for VLA training to iterate through the training dataset significantly more times, with real robot performance continually increasing until training action token accuracy surpasses 95%. Our final training run completes 27 epochs through its training dataset.

训练轮数。典型的LLM或VLM训练运行最多完成一到两个epoch的训练数据集遍历。相比之下，我们发现对于VLA训练来说，更重要的是多次迭代训练数据集，直到训练动作token的准确率超过95%，实际机器人性能才会持续提高。我们最终的训练运行完成了27个epoch的训练数据集遍历。

Learning Rate. We swept the learning rate across multiple orders of magnitude for VLA training, and achieved the best results using a fixed learning rate of $2e-5$

(the same learning rate used during VLM pretraining [44]). We did not find learning rate warmup to provide benefits.

学习率。对于VLA训练，我们扫描了多个数量级的学习率，并使用固定的学习率 $2e-5$ 获得了最佳结果（与VLM预训练期间使用的学习率相同[44]）。我们没有发现学习率预热能带来好处。

3.5 Infrastructure for Training and Inference

3.5 训练和推理的基础设施

The final OpenVLA model is trained on a cluster of 64 A100 GPUs for 14 days, or a total of 21,500 A100-hours, using a batch size of 2048. During inference, OpenVLA requires 15GB of GPU memory when loaded in bfloat16 precision (i.e., without quantization) and runs at approximately 6Hz on one NVIDIA RTX 4090 GPU (without compilation, speculative decoding, or other inference speed-up tricks). We can further reduce the memory footprint of OpenVLA during inference via quantization, without compromising performance in real-world robotics tasks, as shown in Section 5.4. We report inference speed on various consumer- and server-grade GPUs in Fig. 6. For convenience, we implement a remote VLA inference server to allow real-time remote streaming of action predictions to the robot – removing the requirement of having access to a powerful local compute device to control the robot. We release this remote inference solution as part of our open-source code release (Section 4).

最终的OpenVLA模型在一个由64个A100 GPU组成的集群上训练了14天，总计21,500个A100小时，使用的批处理大小为2048。在推理过程中，OpenVLA在以bfloat16精度加载时（即，没有量化）需要15GB的GPU内存，并且在一块NVIDIA RTX 4090 GPU上以大约6Hz的频率运行（没有编译、推测解码或其他推理加速技巧）。我们可以通过量化进一步减少OpenVLA在推理过程中的内存占用，而不会影响实际机器人任务中的性能，如第5.4节所示。我们在图6中报告了在各种消费级和服务器级GPU上的推理速度。为了方便起见，我们实现了一个远程VLA推理服务器，以允许将动作预测实时远程流式传输到机器人——从而无需访问强大的本地计算设备来控制机器人。我们将此远程推理解决方案作为我们的开源代码发布的一部分发布（第4节）。

4 The OpenVLA Codebase

4 OpenVLA 代码库

Along with our model, we release the OpenVLA codebase, a modular PyTorch codebase for training VLA models (see <https://openvla.github.io>). It scales from fine-tuning VLAs on individual GPUs to training billion-parameter VLAs on multi-node GPU clusters, and supports modern techniques for large transformer

model training such as automatic mixed precision (AMP, PyTorch [75]), FlashAttention [76], and fully sharded data parallelism (FSDP, Zhao et al. [77]). Out of the box, the OpenVLA codebase has full support for training on the Open X dataset, integrates with HuggingFace’s [21] AutoModel class, and supports LoRA fine-tuning [26] and quantized model inference [27, 88].

与我们的模型一起，我们发布了 OpenVLA 代码库，这是一个模块化的 PyTorch 代码库，用于训练 VLA 模型（参见 <https://openvla.github.io>）。它可以从单个 GPU 上微调 VLA 扩展到在多节点 GPU 集群上训练数十亿参数的 VLA，并支持大型 Transformer 模型训练的现代技术，例如自动混合精度（AMP, PyTorch [75]）、FlashAttention [76] 和完全分片数据并行（FSDP, Zhao et al. [77]）。开箱即用，OpenVLA 代码库完全支持在 Open X 数据集上进行训练，与 HuggingFace 的 [21] AutoModel 类集成，并支持 LoRA 微调 [26] 和量化模型推理 [27, 88]。

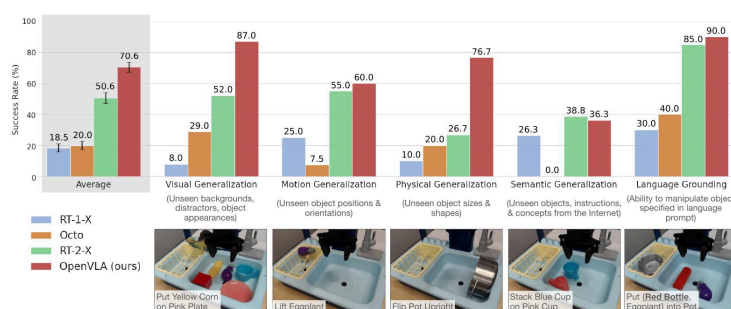


Figure 3: **BridgeData V2 WidowX robot evaluation tasks and results.** We evaluate OpenVLA and prior state-of-the-art generalist robot policies on a comprehensive suite of tasks covering several axes of generalization, as well as tasks that specifically assess language conditioning ability. OpenVLA achieves highest overall performance and even outperforms closed-source model RT-2-X in all categories except for semantic generalization. Average success rates $\pm$ StdErr are computed across 170 total rollouts per approach. See Table 4 for detailed results.

图 3: BridgeData V2 WidowX 机器人评估任务和结果。我们评估了 OpenVLA 和先前最先进的通用机器人策略在一套全面的任务上，这些任务涵盖了多个泛化轴，以及专门评估语言调节能力的任务。OpenVLA 实现了最高的总体性能，甚至在除语义泛化之外的所有类别中都优于闭源模型 RT-2-X。平均成功率 $\pm$ StdErr 是根据每种方法总共 170 次的试验计算得出的。详细结果见表 4。

5 Experiments

5 实验

The goal of our experimental evaluations is to test OpenVLA’s ability to serve as a powerful multirobot control policy out of the box, as well as be a good initialization for fine-tuning to new robot tasks. Concretely, we aim to answer the following questions:

我们实验评估的目标是测试 OpenVLA 作为强大的多机器人控制策略的开箱即用能力，以及作为微调到新机器人任务的良好初始化。具体而言，我们的目标是回答以下问题：

1. How does OpenVLA compare to prior generalist robot policies, when evaluating on multiple robots and various types of generalization?
1. 在评估多个机器人和各种泛化类型时，OpenVLA 与先前的通用机器人策略相比如何？
2. Can OpenVLA be effectively fine-tuned on a new robot setup and task, and how does it compare to state-of-the-art data-efficient imitation learning approaches?
2. OpenVLA能否在新机器人设置和任务上进行有效的微调？它与最先进的数据高效模仿学习方法相比如何？
3. Can we use parameter-efficient fine-tuning and quantization to reduce the computational requirements for training and inference of OpenVLA models and make them more accessible? What are the performance-compute trade-offs?
3. 我们能否使用参数高效微调和量化来降低 OpenVLA 模型训练和推理的计算需求，并使其更易于访问？性能与计算之间有哪些权衡？

5.1 Direct Evaluations on Multiple Robot Platforms

5.1 在多个机器人平台上直接评估

Robot Setups and Tasks. We evaluate OpenVLA’s performance “out-of-the-box” on two robot embodiments: the WidowX robot from the BridgeData V2 evaluations [6] (see Fig. 1, left) and the mobile manipulation robot from the RT-1 and RT-2 evaluations [2, 7] (“Google robot”; see Fig. 1, middle). Both platforms have been extensively used in prior works for evaluating generalist robot policies [1, 2, 5, 7]. We define a comprehensive set of evaluation tasks in each environment that covers various axes of generalization, such as **visual** (unseen backgrounds, distractor objects, colors/appearances of objects); **motion** (unseen object positions/orientations); **physical** (unseen object sizes/shapes); and **semantic** (unseen target objects, instructions, and concepts from the Internet) generalization. We also assess language conditioning ability in scenes with multiple objects, testing whether the policy can manipulate the correct target object, as specified in the user’s prompt. See bottom row of Fig. 3 and Fig. 4 for example task images in the BridgeData V2 and Google robot evaluations, respectively. Overall, we evaluated each method in 170 rollouts (17 tasks with 10 trials each) for BridgeData V2 experiments and 60 rollouts (12 tasks with 5 trials each) for Google robot experiments. A detailed breakdown of all tasks and how they differ from the training data is in Appendix B. All evaluations in this and the following sections are conducted as A/B evaluations, using the same tasks with the same sets of initial robot and object states, to ensure fair comparison.

机器人设置与任务。我们评估了OpenVLA在两种机器人实体上的“开箱即用”性能：来自BridgeData V2评估的WidowX机器人[6]（见图1，左）和来自RT-1和RT-2评估的移动操作机器人[2, 7]（“Google机器人”；见图1，中）。这两种平台都已在先前的工作中被广泛用于评估通用机器人策略[1, 2, 5, 7]。我们在每个环境中定义了一套全面的评估任务，涵盖了泛化的各个方面，例如视觉泛化（未见过的背景、干扰物、物体的颜色/外观）；运动泛化（未见过的物体位置/方向）；物理泛化（未见过的物体大小/形状）；以及语义泛化（未见过的目标物体、指令和来自互联网的概念）。我们还在具有多个物体的场景中评估了语言条件能力，测试策略是否可以操纵用户提示中指定的正确目标物体。有关BridgeData V2和Google机器人评估中的示例任务图像，请参见图3和图4的底行。总的来说，我们针对BridgeData V2实验中的每种方法评估了170次rollout（17个任务，每个任务10次试验），针对Google机器人实验评估了60次rollout（12个任务，每个任务5次试验）。所有任务的详细分类以及它们与训练数据的区别在附录B中。本节和后续章节中的所有评估均作为A/B评估进行，使用相同的任务和相同的初始机器人和物体状态集，以确保公平比较。

Comparisons. We compare OpenVLA’s performance to three prior generalist manipulation policies: **RT-1-X** [1], **RT-2-X** [1], and **Octo** [5]. **RT-1-X** (35M parameters) and **Octo** (93M parameters) are transformer policies trained from scratch on subsets of the OpenX dataset; Octo is the state-of-the-art model among open-source manipulation policies. **RT-2-X** (55B parameters) is a state-of-the-art, closed-source VLA that leverages Internet-pretrained vision and language backbones.

比较。我们将OpenVLA的性能与三种先前的通用操作策略进行比较：RT-1-X [1]、RT-2-X [1]和Octo [5]。RT-1-X（35M参数）和Octo（93M参数）是transformer策略，它们是在OpenX数据集的子集上从头开始训练的；Octo是开源操作策略中目前最先进的模型。RT-2-X（55B参数）是一种最先进的闭源VLA，它利用了互联网预训练的视觉和语言骨干网络。

The results are summarized in Fig. 3 for BridgeData V2 evaluations and Fig. 4 for Google robot evaluations (per-task breakdown in Appendix, Table 4 and Table 6). We find that both RT-1-X and Octo struggle on the tested tasks, often failing to manipulate the correct object, especially when distractors are present, and in some cases causing the robot to wave its arm around aimlessly. Note that our evaluations test even larger degrees of generalization than the evaluations performed in those prior works to challenge the Internet-pretrained VLA models. Thus, lower performance of models without Internet pretraining is expected. RT-2-X clearly outperforms both RT-1-X and Octo, demonstrating the benefits of large, pretrained VLMs for robotics.

结果总结在图 3 中，针对 BridgeData V2 的评估；图 4 针对 Google 机器人评估（每个任务的详细分解见附录，表 4 和表 6）。我们发现 RT-1-X 和 Octo

在测试任务上都表现不佳，经常无法操纵正确的物体，尤其是在存在干扰物时，在某些情况下还会导致机器人漫无目的地挥舞手臂。请注意，我们的评估测试了比先前工作中执行的评估更大的泛化程度，以挑战互联网预训练的 VLA 模型。因此，没有经过互联网预训练的模型的性能较低是预期的。RT-2-X 明显优于 RT-1-X 和 Octo，证明了大型预训练 VLM 对于机器人的益处。

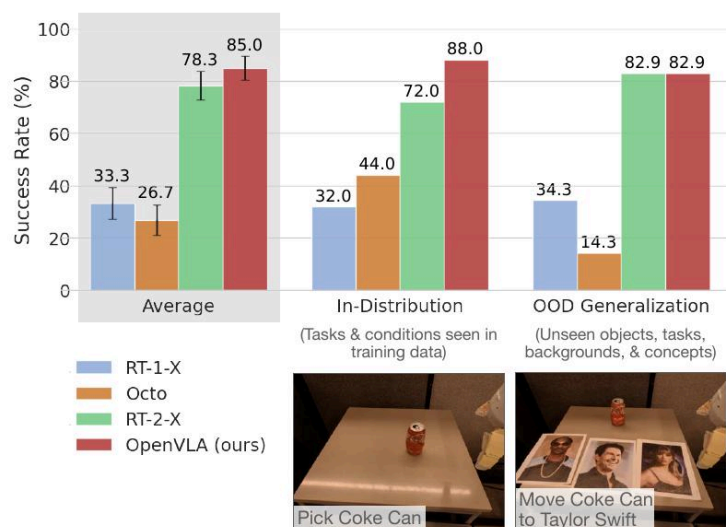


Figure 4: **Google robot evaluation results.** We evaluate generalist robot policies on in-distribution and out-of-distribution (OOD) tasks on the mobile manipulator used in RT-1 and RT-2 evaluations [2, 7]. We find that OpenVLA and RT-2-X attain comparable performance and significantly outperform RT-1-X and Octo overall. Average success rates $\pm$ StdErr are computed across 60 total rollouts per approach. See Table 6 for detailed results.

图 4：谷歌机器人评估结果。我们在 RT-1 和 RT-2 评估中使用的移动机械臂上，评估通用机器人策略在分布内和分布外（OOD）任务上的表现 [2, 7]。我们发现 OpenVLA 和 RT-2-X 获得了相当的性能，并且总体上明显优于 RT-1-X 和 Octo。平均成功率 $\pm$ StdErr 是根据每种方法总共 60 次的推出计算得出的。详细结果见表 6。

Notably, OpenVLA performs comparably to RT2-X on Google robot evaluations and significantly outperforms RT-2-X on BridgeData V2 evaluations despite being an order of magnitude smaller (7B vs. 55B parameters). Qualitatively, we find that both RT-2-X and OpenVLA exhibit markedly more robust behaviors than the other tested models, such as approaching the correct object when distractor objects are present, properly orienting the robot’s endeffector to align with the orientation of the target object, and even recovering from mistakes such as insecurely grasping objects (see <https://openvla.github.io> for qualitative rollout examples). RT-2-X achieves higher performance in semantic generalization tasks, as shown in Fig. 3, which is expected given that it uses larger-scale Internet pretraining data and is co-fine-tuned with both robot action data and Internet pretraining data to better preserve the pretraining knowledge, rather than being finetuned solely on robot data, like OpenVLA. However, OpenVLA performs comparably or better in all other task categories in both BridgeData V2 and Google robot evaluations. The performance difference can be attributed to a combination of factors: we curated a

much larger training dataset for OpenVLA with 970k trajectories (vs. 350k for RT-2-X); we performed more careful cleaning of the training dataset and, e.g., filtered out all-zero actions in the Bridge dataset (see Appendix C for a detailed discussion); and OpenVLA uses a fused vision encoder that combines pretrained semantic *and* spatial features. See Appendix D for ablation analyses of these components.

值得注意的是，在 Google 机器人评估中，OpenVLA 的表现与 RT2-X 相当，并且在 BridgeData V2 评估中明显优于 RT-2-X，尽管其规模小了一个数量级（7B 参数 vs. 55B 参数）。从定性角度来看，我们发现 RT-2-X 和 OpenVLA 都表现出明显比其他测试模型更稳健的行为，例如在存在干扰物时接近正确的物体，正确地调整机器人末端执行器的方向以与目标物体的方向对齐，甚至可以从诸如不安全地抓取物体等错误中恢复（有关定性展开示例，请参见 <https://openvla.a.github.io>）。如图 3 所示，RT-2-X 在语义泛化任务中实现了更高的性能，这是预期的，因为它使用了更大规模的互联网预训练数据，并且与机器人动作数据和互联网预训练数据共同进行微调，以更好地保留预训练知识，而不是像 OpenVLA 那样仅在机器人数据上进行微调。然而，在 BridgeData V2 和 Google 机器人评估的所有其他任务类别中，OpenVLA 的表现相当或更好。性能差异可归因于多种因素的结合：我们为 OpenVLA 策划了一个更大的训练数据集，包含 970k 条轨迹（RT-2-X 为 350k 条）；我们对训练数据集进行了更仔细的清理，例如，过滤掉了 Bridge 数据集集中的所有零动作（详细讨论请参见附录 C）；并且 OpenVLA 使用了融合的视觉编码器，该编码器结合了预训练的语义和空间特征。有关这些组件的消融分析，请参见附录 D。

5.2 Data-Efficient Adaptation to New Robot Setups

5.2 面向新机器人设置的数据高效适配

While prior works mainly focused on directly evaluating VLAs “out-of-the-box” [1, 7, 16], effective *fine-tuning* of VLA models to new tasks and robot setups is largely unexplored, yet is key for their widespread adoption. In this section, we investigate OpenVLA’s ability to be quickly adapted to a new *real-world* robot setup. (See Appendix E for fine-tuning experiments in simulation.)

虽然先前的工作主要集中于直接评估视觉语言动作模型（VLA）的“开箱即用”性能[1, 7, 16]，但将VLA模型有效微调到新任务和机器人设置在很大程度上尚未被探索，而这对于它们的广泛应用至关重要。在本节中，我们研究 OpenVLA 快速适应新的真实世界机器人设置的能力。（有关在模拟中进行微调实验，请参见附录E。）

Robot setups and tasks. We test a simple fine-tuning recipe for the OpenVLA model: full finetuning of all model parameters, using small datasets with 10–150 demonstrations of a target task (see Fig. 5; we explore parameter-efficient fine-

tuning approaches in Section 5.3). We test OpenVLA in two setups: **Franka-Tabletop**, a stationary, table-mounted Franka Emika Panda 7-DoF robot arm; and **Franka-DROID**, the Franka robot arm setup from the recently released DROID dataset [11], mounted on a movable standing desk. The setups use 5Hz and 15 Hz non-blocking controllers, respectively. We choose Franka robot arms as the target embodiment for our fine-tuning experiments since they are widely used in the robot learning community and thus a likely “target” of OpenVLA fine-tuning. We test on setups with different control frequencies to test OpenVLA’s applicability to a range of use cases.

机器人设置和任务。我们测试了 OpenVLA 模型的一个简单的微调方案：对所有模型参数进行完全微调，使用包含目标任务 10–150 个演示的小型数据集（参见图 5；我们在第 5.3 节中探讨了参数高效的微调方法）。我们在两种设置中测试 OpenVLA：Franka-Tabletop，一个固定的、安装在桌面的 Franka Emika Panda 7-DoF 机器人手臂；以及 Franka-DROID，最近发布的 DROID 数据集 [11] 中的 Franka 机器人手臂设置，安装在一个可移动的站立式办公桌上。这些设置分别使用 5Hz 和 15 Hz 的非阻塞控制器。我们选择 Franka 机器人手臂作为微调实验的目标实体，因为它们在机器人学习社区中被广泛使用，因此很可能是 OpenVLA 微调的“目标”。我们在具有不同控制频率的设置上进行测试，以检验 OpenVLA 对各种用例的适用性。

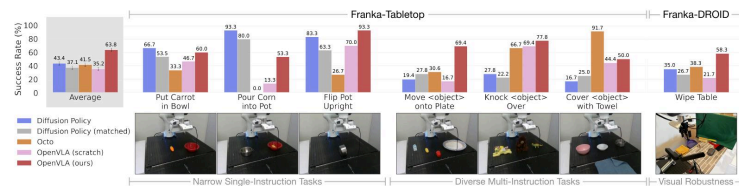


Figure 5: **Adapting to new robot setups.** We evaluate the state-of-the-art Diffusion Policy trained from scratch on seven Franka Emika Panda tasks (10–150 demonstrations each), as well as generalist robot policies Octo and OpenVLA fine-tuned on the same data. Diffusion Policy exhibits strong performance on narrow single-instruction tasks, while Octo and OpenVLA perform better on diverse fine-tuning tasks involving multiple instructions and distractor objects. Overall, OpenVLA achieves highest aggregate performance across both setups, suggesting that it is an effective default for learning a policy on a downstream task. Average success rates $\pm$ StdErr are computed across 129 rollouts per approach (99 for Franka-Tabletop tasks and 30 for Franka-DROID tasks). See Table 7 for detailed results.

图5：适应新的机器人设置。我们评估了在七个Franka Emika Panda任务上从头开始训练的最新扩散策略（每个任务10–150个演示），以及在相同数据上微调的通用机器人策略Octo和OpenVLA。扩散策略在狭窄的单指令任务上表现出强大的性能，而Octo和OpenVLA在涉及多个指令和干扰物的多样化微调任务上表现更好。总体而言，OpenVLA在两种设置中实现了最高的总体性能，表明它是学习下游任务策略的有效默认选择。平均成功率 $\pm$ 标准误差是根据每种方法的129次试验计算得出的（Franka-Tabletop任务为99次，Franka-DROID任务为30次）。详细结果见表7。

Comparisons. We compare to **Diffusion Policy** [3], a state-of-the-art data-efficient imitation learning approach, trained from scratch. We also compare to **Diffusion Policy (matched)**, a version of Diffusion Policy that matches the input

and output specifications of OpenVLA.³ Additionally, we evaluate **Octo** [5] fine-tuned on the target dataset, since it is currently the best generalist policy that supports fine-tuning (fine-tuning of RT-2-X is not supported through its inference API). We also fine-tune OpenVLA on the same target dataset, and the resulting policy is denoted by **OpenVLA**. Finally, as an ablation experiment, we compare to **OpenVLA (scratch)**, where we directly finetune the underlying base Prismatic VLM on the target robot setup – rather than fine-tuning the OpenX-pretrained OpenVLA model – to assess the benefit of large-scale robot pretraining.

比较。我们与 Diffusion Policy [3] 进行了比较，这是一种最先进的数据高效模仿学习方法，从头开始训练。我们还将与 Diffusion Policy（匹配）进行比较，该版本 Diffusion Policy 匹配 OpenVLA 的输入和输出规范。³ 此外，我们评估了在目标数据集上进行微调的 Octo [5]，因为它是目前支持微调的最佳通用策略（RT-2-X 的微调不支持通过其推理 API）。我们还在同一目标数据集上对 OpenVLA 进行了微调，得到的策略表示为 OpenVLA。最后，作为一个消融实验，我们与 OpenVLA (scratch) 进行了比较，其中我们直接在目标机器人设置上微调底层的基础 Prismatic VLM——而不是微调 OpenX 预训练的 OpenVLA 模型——以评估大规模机器人预训练的优势。

We present the results in Fig. 5 (per-task breakdown in Appendix, Table 7). We find that both versions of Diffusion Policy are competitive with or outperform the generalist policies Octo and OpenVLA on narrower single-instruction tasks like “Put Carrot in Bowl” and “Pour Corn into Pot”, but the pretrained generalist policies perform better in more diverse fine-tuning tasks that involve multiple objects in the scene and require language conditioning. OpenX pretraining for Octo and OpenVLA enables the models to better adapt to these more diverse tasks where language grounding is important; we see evidence for this in the lower performance of OpenVLA (scratch).

我们在图5中展示了结果（每个任务的详细分解见附录表7）。我们发现，Diffusion Policy的两个版本在较窄的单指令任务（如“将胡萝卜放入碗中”和“将玉米倒入锅中”）上与通用策略Octo和OpenVLA具有竞争力或优于它们，但预训练的通用策略在更多样化的微调任务中表现更好，这些任务涉及场景中的多个对象并需要语言条件反射。Octo和OpenVLA的OpenX预训练使模型能够更好地适应这些更多样化的任务，在这些任务中，语言基础非常重要；我们在OpenVLA (scratch) 的较低性能中看到了这一点。

Overall, we find that OpenVLA achieves the highest average performance. Notably, most prior works achieve strong performance only in *either* narrow single-instruction *or* diverse multi-instruction tasks, resulting in widely varying success rates. OpenVLA is the only approach that achieves at least 50% success rate across all tested tasks, suggesting that it can be a strong default option for imitation learning tasks, particularly if they involve a diverse set of language

instructions. For narrower but highly dexterous tasks, Diffusion Policy still shows smoother and more precise trajectories; incorporating action chunking and temporal smoothing, as implemented in Diffusion Policy, may help OpenVLA attain the same level of dexterity and may be a promising direction for future work (see Section 6 for a detailed discussion of current limitations).

总的来说，我们发现OpenVLA实现了最高的平均性能。值得注意的是，大多数先前的工作仅在狭窄的单指令或多样化的多指令任务中实现了强大的性能，导致成功率差异很大。OpenVLA是唯一在所有测试任务中至少达到50%成功率的方法，这表明它可以成为模仿学习任务的强大默认选项，特别是当它们涉及一组多样化的语言指令时。对于更窄但高度灵巧的任务，Diffusion Policy仍然表现出更平滑和更精确的轨迹；结合动作分块和时间平滑（如Diffusion Policy中所实现的），可能有助于OpenVLA达到相同的灵巧程度，并且可能是未来工作的一个有希望的方向（有关当前局限性的详细讨论，请参见第6节）。

5.3 Parameter-Efficient Fine-Tuning

5.3 参数高效微调

The full fine-tuning runs of OpenVLA in the previous section used 8 A100 GPUs for 5-15 hours per task (depending on the dataset size) to achieve high performance. While this is substantially less compute than what is required for VLA pretraining, in this section we explore even more computeand parameter-efficient fine-tuning approaches and investigate their effectiveness.

前一节中OpenVLA的完整微调运行使用了8个A100 GPU，每个任务耗时5-15小时（取决于数据集大小）以实现高性能。虽然这比VLA预训练所需的计算量少得多，但在本节中，我们将探索更高效的计算和参数微调方法，并研究它们的有效性。

Table 1: **Parameter-efficient fine-tuning evaluation.** LoRA finetuning achieves the best performance-compute trade-off, matching full fine-tuning performance while training only 1.4% of the model parameters. Mean success $\pm$ StdErr computed across 33 rollouts per approach on select Franka-Tabletop tasks (see Table 8 for details). *: Sharded across 2 GPUs with FSDP [77].

表1：参数高效微调评估。LoRA微调实现了最佳的性能-计算权衡，在仅训练1.4%的模型参数的同时，达到了完全微调的性能。平均成功率 $\pm$ 标准误差是在选定的Franka-Tabletop任务上，对每种方法进行33次rollout计算得出的（详见表8）。*：使用FSDP [77] 在2个GPU上分片。

Strategy	Success Rate	Train Params ($\times 10^6$)	VRAM (batch 16)
Full FT	69.7 $\pm$ 7.2 %	7,188.1	163.3 GB*
Last layer only	30.3 $\pm$ 6.1 %	465.1	51.4 GB
Frozen vision	47.0 $\pm$ 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 $\pm$ 7.9 %	914.2	64.0 GB
LoRA, rank=32	68.2 $\pm$ 7.5%	97.6	59.7 GB
rank=64	68.2 $\pm$ 7.8%	195.2	60.5 GB

Concretely, we compare the following fine-tuning approaches: **full finetuning** updates all weights during finetuning, as described in Section 5.2; **last layer only** fine-tunes only the last layer of OpenVLA’s transformer backbone and the token embedding matrix; **frozen vision** freezes the vision encoder but fine-tunes all other weights; **sandwich fine-tuning** unfreezes the vision encoder, token embedding matrix, and last layer; and **LoRA** uses the popular low-rank adaptation technique of Hu et al. [26] with multiple rank values r , applied to all linear layers of the model.

具体而言，我们比较了以下微调方法：完全微调更新微调期间的所有权重，如第 5.2 节所述；仅最后一层微调仅微调 OpenVLA 转换器主干的最后一层和令牌嵌入矩阵；冻结视觉冻结视觉编码器，但微调所有其他权重；夹层微调解冻视觉编码器、令牌嵌入矩阵和最后一层；而 LoRA 使用 Hu 等人 [26] 的流行低秩适应技术，具有多个秩值 r ，应用于模型的所有线性层。

We report fine-tuning success rates across multiple Franka-Tabletop tasks, as well as training parameter count and GPU memory requirements, in Table 1.⁴ We find that only fine-tuning the network’s last layer or freezing the vision encoder leads to poor performance, suggesting that further adaptation of the visual features to the target scene is crucial. In contrast, “sandwich fine-tuning” achieves better performance since it fine-tunes the vision encoder, and it consumes less GPU memory since it does not fine-tune the full LLM backbone. Lastly, LoRA achieves the best trade-off between performance and training memory consumption, outperforming “sandwich fine-tuning” and matching full fine-tuning performance while fine-tuning only 1.4% of the parameters. We find that the LoRA rank has negligible effect on policy performance and thus recommend using a default rank of $r = 32$. With LoRA, we can fine-tune OpenVLA on a new task within 10-15 hours on a *single* A100 GPU – an 8x reduction in compute compared to full fine-tuning.

我们在表1中报告了多个Franka-Tabletop任务的微调成功率，以及训练参数数量和GPU内存需求。⁴ 我们发现，仅微调网络的最后一层或冻结视觉编码器会导致较差的性能，这表明视觉特征进一步适应目标场景至关重要。相比之下，“夹层微调”实现了更好的性能，因为它微调了视觉编码器，并且由于它不微调完整的LLM主干，因此消耗更少的GPU内存。最后，LoRA在性能和训练内存消耗之间实现了最佳的权衡，优于“夹层微调”，并在仅微调1.4%的参数的情況下，与完全微调的性能相匹配。我们发现LoRA秩对策略性能的影响可以忽略不计，因此建议使用默认秩 $r = 32$ 。使用LoRA，我们可以在单个A100 GPU上，在10-15小时内完成OpenVLA在新任务上的微调——与完全微调相比，计算量减少了8倍。

5.4 Memory-Efficient Inference via Quantization

5.4 通过量化实现内存高效的推理

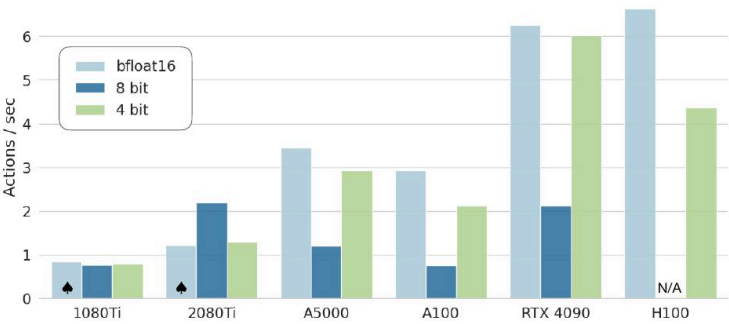


Figure 6: **OpenVLA inference speed for various GPUs.** Both bfloat16 and int4 quantization achieve high throughput, especially on GPUs with Ada Lovelace architecture (RTX 4090, H100). Further speed-ups are possible with modern LLM inference frameworks like TensorRT-LLM [89]. ♠: Model sharded across two GPUs to fit.

图 6: OpenVLA 在各种 GPU 上的推理速度。bfloat16 和 int4 量化均实现了高吞吐量，尤其是在采用 Ada Lovelace 架构的 GPU (RTX 4090、H100) 上。使用诸如 TensorRT-LLM [89] 之类的现代 LLM 推理框架，可以进一步提高速度。♠: 模型跨两个 GPU 分片以适应。

Precision	Bridge Success	VRAM
bfloat16	71.3 ± 4.8%	16.8 GB
int8	58.1 ± 5.1%	10.2 GB
int4	71.9 ± 4.7%	7.0 GB

Table 2: **Performance with quantized inference.** 4-bit quantization matches the performance of bfloat16 inference (our default approach) while reducing the GPU memory footprint by more than half. Mean success $\pm$ StdErr computed across 8 representative BridgeData V2 tasks [6] and 80 rollouts per approach (see Table 5 for details).

表2: 量化推理的性能。4位量化与bfloat16推理（我们的默认方法）的性能相匹配，同时将GPU内存占用减少了一半以上。平均成功率 $\pm$ 标准误差是在8个具有代表性的BridgeData V2任务[6]和每种方法的80次rollout中计算得出的（详见表5）。

OpenVLA, a 7B-parameter model, consumes more memory at inference time than prior open-source generalist policies such as Octo, which has <100M parameters. We follow best-practices from LLM serving by saving and loading OpenVLA in bfloat16 precision for inference (our default approach), which cuts the memory footprint in half, allowing us to serve OpenVLA on GPUs with only 16GB of GPU memory. In this section, we test whether we can further reduce the required memory for policy inference and broaden accessibility of VLA policies, by using modern quantization techniques developed for serving LLMs [27, 88]. These approaches load the weights of the network at lower precision, thereby trading off reduced memory requirements for potentially reduced inference speed and accuracy.

OpenVLA，一个具有70亿参数的模型，在推理时消耗的内存比之前的开源通用策略（如Octo，其参数小于1亿）更多。我们遵循LLM服务的最佳实践，通过以bfloat16精度保存和加载OpenVLA以进行推理（我们的默认方法），这会将内存

占用减少一半，使我们能够在仅具有16GB GPU内存的GPU上提供OpenVLA。在本节中，我们测试是否可以通过使用为服务LLM而开发的现代量化技术[27, 88]，进一步减少策略推理所需的内存并扩大VLA策略的可访问性。这些方法以较低的精度加载网络的权重，从而以潜在的推理速度和准确性降低为代价，来减少内存需求。

Concretely, we investigate serving the OpenVLA model with 8-bit and 4-bit precision on 8 representative BridgeData V2 tasks. We report memory footprint and rollout performance in Table 2. We also report achievable control frequencies on various consumer- and server-grade GPUs in Fig. 6. We observe that 8-bit quantization slows down inference across most GPUs, due to the overhead of the added quantization operations. 4-bit inference achieves higher throughput, since reduced GPU memory transfer compensates for the quantization overhead.

具体而言，我们研究了在8个具有代表性的BridgeData V2任务上，以8位和4位精度服务OpenVLA模型。我们在表2中报告了内存占用和展开性能。我们还在图6中报告了在各种消费级和服务器级GPU上可实现的控制频率。我们观察到，由于增加了量化操作的开销，8位量化降低了大多数GPU的推理速度。4位推理实现了更高的吞吐量，因为减少的GPU内存传输弥补了量化开销。

As a result of the reduced inference speed, we observe a substantial performance decrease with 8-bit quantization: on the A5000 GPU we use for our evaluations, we can only run the model at 1.2Hz, which significantly changes the system dynamics compared to the training dataset for the 5Hz non-blocking controller used in the BridgeData V2 tasks.⁵ Notably, 4-bit quantization results in similar performance as bfloat16 half-precision inference despite requiring less than half the amount of GPU memory. 4-bit quantized models can run at 3Hz on the A5000, thus more closely matching the system dynamics during data collection.

由于推理速度的降低，我们观察到 8 位量化导致了显著的性能下降：在我们用于评估的 A5000 GPU 上，我们只能以 1.2Hz 的频率运行模型，这与 BridgeData V2 任务中使用的 5Hz 非阻塞控制器的训练数据集相比，显著改变了系统动态。⁵ 值得注意的是，尽管 4 位量化所需的 GPU 内存不到 bfloat16 半精度推理的一半，但其性能与 bfloat16 半精度推理相似。4 位量化模型可以在 A5000 上以 3Hz 的频率运行，因此更接近数据收集期间的系统动态。

6 Discussion and Limitations

6 讨论与局限性

In this work, we presented OpenVLA, a state-of-the-art, open-source vision-language-action model that obtains strong performance for cross-embodiment

robot control out-of-the-box. We also demonstrated that OpenVLA can be easily adapted to new robot setups via parameter-efficient fine-tuning techniques.

在这项工作中，我们提出了OpenVLA，一个最先进的开源视觉-语言-动作模型，它开箱即用地获得了强大的跨具身机器人控制性能。我们还证明了OpenVLA可以通过参数高效的微调技术轻松地适应新的机器人设置。

The current OpenVLA model has several limitations. First, it currently only supports single-image observations. In reality, real-world robot setups are heterogeneous, with a wide range of possible sensory inputs [5]. Expanding OpenVLA to support multiple image and proprioceptive inputs as well as observation history is an important avenue for future work. Exploring the use of VLMs pretrained on *interleaved* image and text data may facilitate such flexible-input VLA fine-tuning.

目前的OpenVLA模型存在一些局限性。首先，它目前仅支持单图像观测。实际上，现实世界的机器人设置是异构的，具有各种可能的感官输入[5]。扩展OpenVLA以支持多图像和本体感受输入以及观测历史是未来工作的一个重要途径。探索使用在交错图像和文本数据上预训练的VLM可能有助于这种灵活输入VLA的微调。

Secondly, improving the inference throughput of OpenVLA is critical to enable VLA control for high-frequency control setups such as ALOHA [90], which runs at 50Hz. This will also enable testing VLAs on more dexterous, bi-manual manipulation tasks than what we investigated in this work. Exploring the use of action chunking or alternative inference-time optimization techniques such as speculative decoding [91] offer potential remedies.

其次，提高OpenVLA的推理吞吐量对于实现高频控制设置（如以50Hz运行的ALOHA [90]）的VLA控制至关重要。这也将使我们能够在比本文研究的更灵巧、双手操作的任务上测试VLA。探索使用动作分块或替代的推理时优化技术（如推测性解码[91]）提供了潜在的补救措施。

Additionally, there is room for further performance improvements. While OpenVLA outperforms prior generalist policies, it does not yet offer very high reliability on the tested tasks, typically achieving <90% success rate.

此外，仍有进一步提升性能的空间。虽然OpenVLA的性能优于先前的通用策略，但它在测试任务中的可靠性尚未达到非常高的水平，通常成功率<90%。

Finally, due to compute limitations, many VLA design questions remain underexplored: What effect does the size of the base VLM have on VLA performance? Does co-training on robot action prediction data and Internet-scale vision-language data substantially improve VLA performance? What visual

features are best-suited for VLA models? We hope that the release of the OpenVLA model and codebase will enable the community to jointly investigate these questions.

最后，由于计算限制，许多VLA设计问题仍未得到充分探索：基础VLM的大小对VLA性能有何影响？在机器人动作预测数据和互联网规模的视觉-语言数据上进行联合训练，能否显著提高VLA的性能？哪种视觉特征最适合VLA模型？我们希望OpenVLA模型和代码库的发布能够使社区共同研究这些问题。

Acknowledgments

致谢

We are grateful to the Toyota Research Institute for providing significant funding and compute resources required to carry out this research. We also thank the Stanford Center for Research on Foundation Models for providing additional compute resources and Google DeepMind for alpha access to the RT-2-X API for our evaluations. We acknowledge additional support from Volkswagen, Physical Intelligence, ONR grants N00014-22-1-2621 and N00014-22-1-2293, the National Science Foundation through IIS-2246811, and DARPA ANSR.

我们感谢丰田研究院为开展这项研究提供了重要的资金和计算资源。我们还要感谢斯坦福大学基础模型研究中心为我们的评估提供了额外的计算资源，以及谷歌DeepMind为我们提供了RT-2-X API的alpha访问权限。我们感谢大众汽车公司、Physical Intelligence、ONR拨款N00014-22-1-2621和N00014-22-1-2293、国家自然科学基金会通过IIS-2246811以及DARPA ANSR提供的额外支持。

References

参考文献

[1] Open X-Embodiment Collaboration, A. Padalkar, A. Pooley, A. Jain, A. Bewley, A. Herzog, A. Irpan, A. Khazatsky, A. Rai, A. Singh, A. Brohan, A. Raffin, A. Wahid, B. Burgess-Limerick, B. Kim, B. Schölkopf, B. Ichter, C. Lu, C. Xu, C. Finn, C. Xu, C. Chi, C. Huang, C. Chan, C. Pan, C. Fu, C. Devin, D. Driess, D. Pathak, D. Shah, D. Büchler, D. Kalashnikov, D. Sadigh, E. Johns, F. Ceola, F. Xia, F. Stulp, G. Zhou, G. S. Sukhatme, G. Salhotra, G. Yan, G. Schiavi, H. Su, H.-S. Fang, H. Shi, H. B. Amor, H. I. Christensen, H. Furuta, H. Walke, H. Fang, I. Mordatch, I. Radosavovic, I. Leal, J. Liang, J. Kim, J. Schneider, J. Hsu, J. Bohg, J. Bingham, J. Wu, J. Wu, J. Luo, J. Gu, J. Tan, J. Oh, J. Malik, J. Tompson, J. Yang, J. J. Lim, J. Silvério, J. Han, K. Rao, K. Pertsch, K. Hausman, K. Go, K. Gopalakrishnan, K. Goldberg, K. Byrne, K. Oslund, K. Kawaharazuka, K. Zhang, K. Majd, K. Rana, K. Srinivasan, L. Y. Chen, L. Pinto, L. Tan, L. Ott, L. Lee, M. Tomizuka, M. Du, M. Ahn, M. Zhang, M. Ding, M. K. Srirama, M. Sharma, M. J. Kim, N. Kanazawa, N. Hansen, N. Heess, N. J. Joshi, N. Suenderhauf, N. D. Palo,

N. M. M. Shafiullah, O. Mees, O. Kroemer, P. R. Sanketi, P. Wohlhart, P. Xu, P. Sermanet, P. Sundaresan, Q. Vuong, R. Rafailov, R. Tian, R. Doshi, R. Martín-Martín, R. Mendonca, R. Shah, R. Hoque, R. Julian, S. Bustamante, S. Kirmani, S. Levine, S. Moore, S. Bahl, S. Dass, S. Song, S. Xu, S. Haldar, S. Adebola, S. Guist, S. Nasiriany, S. Schaal, S. Welker, S. Tian, S. Dasari, S. Belkhale, T. Osa, T. Harada, T. Matsushima, T. Xiao, T. Yu, T. Ding, T. Davchev, T. Z. Zhao, T. Armstrong, T. Darrell, V. Jain, V. Vanhoucke, W. Zhan, W. Zhou, W. Burgard, X. Chen, X. Wang, X. Zhu, X. Li, Y. Lu, Y. Chebotar, Y. Zhou, Y. Zhu, Y. Xu, Y. Wang, Y. Bisk, Y. Cho, Y. Lee, Y. Cui, Y. hua Wu, Y. Tang, Y. Zhu, Y. Li, Y. Iwasawa, Y. Matsuo, Z. Xu, and Z. J. Cui. Open X-Embodiment: Robotic learning datasets and RT-X models. <https://arxiv.org/abs/2310.08864>, 2023.

[2] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-1: Robotics transformer for real-world control at scale. In *arXiv preprint arXiv:2212.06817*, 2022.

[3] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[4] A. Xie, L. Lee, T. Xiao, and C. Finn. Decomposing the generalization gap in imitation learning for visual robotic manipulation. *arXiv preprint arXiv:2307.03659*, 2023.

[5] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, C. Xu, J. Luo, T. Kreiman, Y. Tan, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. <https://octo-models.github.io>, 2023.

[6] H. Walke, K. Black, A. Lee, M. J. Kim, M. Du, C. Zheng, T. Zhao, P. Hansen-Estruch, Q. Vuong, A. He, V. Myers, K. Fang, C. Finn, and S. Levine. Bridgedata v2: A dataset for robot learning at scale, 2023.

[7] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S.

Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *arXiv preprint arXiv:2307.15818*, 2023.

[8] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, volume 139, pages 8748–8763, 2021.

[9] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pretraining. In *International Conference on Computer Vision (ICCV)*, 2023.

[10] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[11] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. D. Fagan, J. Hejna, M. Itkina, M. Lepert, Y. J. Ma, P. T. Miller, J. Wu, S. Belkhale, S. Dass, H. Ha, A. Jain, A. Lee, Y. Lee, M. Memmel, S. Park, I. Radosavovic, K. Wang, A. Zhan, K. Black, C. Chi, K. B. Hatch, S. Lin, J. Lu, J. Mercat, A. Rehman, P. R. Sanketi, A. Sharma, C. Simpson, Q. Vuong, H. R. Walke, B. Wulfe, T. Xiao, J. H. Yang, A. Yavary, T. Z. Zhao, C. Agia, R. Baijal, M. G. Castro, D. Chen, Q. Chen, T. Chung, J. Drake, E. P. Foster, J. Gao, D. A. Herrera, M. Heo, K. Hsu, J. Hu, D. Jackson, C. Le, Y. Li, K. Lin, R. Lin, Z. Ma, A. Maddukuri, S. Mirchandani, D. Morton, T. Nguyen, A. O’Neill, R. Scalise, D. Seale, V. Son, S. Tian, E. Tran, A. E. Wang, Y. Wu, A. Xie, J. Yang, P. Yin, Y. Zhang, O. Bastani, G. Berseth, J. Bohg, K. Goldberg, A. Gupta, A. Gupta, D. Jayaraman, J. J. Lim, J. Malik, R. Martín-Martín, S. Ramamoorthy, D. Sadigh, S. Song, J. Wu, M. C. Yip, Y. Zhu, T. Kollar, S. Levine, and C. Finn. Droid: A large-scale in-the-wild robot manipulation dataset. 2024.

[12] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3m: A universal visual representation for robot manipulation. In *CoRL*, 2022.

[13] S. Karamcheti, S. Nair, A. S. Chen, T. Kollar, C. Finn, D. Sadigh, and P. Liang. Languagedriven representation learning for robotics. *ArXiv*, abs/2302.12766, 2023. URL <https://api.semanticscholar.org/CorpusID:257205716>.

- [14] M. Shridhar, L. Manuelli, and D. Fox. Cliport: What and where pathways for robotic manipulation. In *Conference on robot learning*, pages 894–906. PMLR, 2022.
- [15] A. Stone, T. Xiao, Y. Lu, K. Gopalakrishnan, K.-H. Lee, Q. Vuong, P. Wohlhart, B. Zitkovich, F. Xia, C. Finn, et al. Open-world object manipulation using pre-trained vision-language models. *arXiv preprint arXiv:2303.00905*, 2023.
- [16] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.
- [17] A. S. et al. Introducing rfm-1: Giving robots human-like reasoning capabilities, 2024. URL <https://covariant.ai/insights/introducing-rfm-1-giving-robots-human-like-reasoning-capabilities/>.
- [18] Wayve. Lingo-2: Driving with natural language. 2024. URL <http://wayve.ai/thinking/lingo-2-driving-with-language/>.
- [19] X. Chen, X. Wang, S. Changpinyo, A. J. Piergiovanni, P. Padlewski, D. M. Salz, S. Goodman, A. Grycner, B. Mustafa, L. Beyer, A. Kolesnikov, J. Puigcerver, N. Ding, K. Rong, H. Akbari, G. Mishra, L. Xue, A. V. Thapliyal, J. Bradbury, W. Kuo, M. Seyedhosseini, C. Jia, B. K. Ayan, C. Riquelme, A. Steiner, A. Angelova, X. Zhai, N. Houlsby, and R. Soricut. Pali: A jointly-scaled multilingual language-image model. *ArXiv*, abs/2209.06794, 2022. URL <http://api.semanticscholar.org/CorpusID:252222320>.
- [20] X. Chen, X. Wang, L. Beyer, A. Kolesnikov, J. Wu, P. Voigtlaender, B. Mustafa, S. Goodman, I. M. Alabdulmohsin, P. Padlewski, D. M. Salz, X. Xiong, D. Vlasic, F. Pavetic, K. Rong, T. Yu, D. Keysers, X.-Q. Zhai, and R. Soricut. PaLI-3 vision language models: Smaller, faster, stronger. *arXiv preprint arXiv:2310.09199*, 2023.
- [21] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, and . . . Transformers: State-of-the-art natural language processing. In *Proceedings of the 6th International Conference on Learning Representations*, 2020. URL <https://arxiv.org/abs/1910.03771>.
- [22] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

- [23] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [24] G. Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.
- [25] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- [26] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [27] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 36, 2024.
- [28] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh. Making the V in VQA matter: Elevating the role of image understanding in visual question answering. In *Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [29] D. A. Hudson and C. D. Manning. GQA: A new dataset for real-world visual reasoning and compositional question answering. In *Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [30] A. Singh, V. Natarajan, M. Shah, Y. Jiang, X. Chen, D. Batra, D. Parikh, and M. Rohrbach. Towards VQA models that can read. In *Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [31] J. P. Bigham, C. Jayant, H. Ji, G. Little, A. Miller, R. C. Miller, R. Miller, A. Tatarowicz, B. White, S. White, and T. Yeh. VizWiz: nearly real-time answers to visual questions. In *User Interface Software and Technology (UIST)*, pages 333–342, 2010.
- [32] S. Kazemzadeh, V. Ordonez, M. Matten, and T. Berg. ReferItGame: Referring to objects in photographs of natural scenes. In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 787–798, 2014.
- [33] L. Yu, P. Poirson, S. Yang, A. C. Berg, and T. L. Berg. Modeling context in referring expressions. In *European Conference on Computer Vision (ECCV)*,

2016.

[34] T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanova, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikuła, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wahltinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean, K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin, N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.

[35] Y. Li, S. Bubeck, R. Eldan, A. D. Giorno, S. Gunasekar, and Y. T. Lee. Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*, 2023.

[36] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

[37] J. Li, D. Li, C. Xiong, and S. C. H. Hoi. BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning (ICML)*, 2022.

[38] J. Li, D. Li, S. Savarese, and S. C. H. Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning (ICML)*, 2023.

[39] W. Dai, J. Li, D. Li, A. M. H. Tiong, J. Zhao, W. Wang, B. A. Li, P. Fung, and S. C. H. Hoi. InstructBLIP: Towards general-purpose vision-language models with instruction tuning. *arXiv preprint arXiv:2305.06500*, 2023.

[40] H. H. Tan and M. Bansal. LXMERT: Learning cross-modality encoder representations from transformers. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

- [41] H. Laurençon, L. Saulnier, L. Tronchon, S. Bekman, A. Singh, A. Lozhkov, T. Wang, S. Karamcheti, A. M. Rush, D. Kiela, M. Cord, and V. Sanh. OBELICS: An open web-scale filtered dataset of interleaved image-text documents. In *Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS Datasets and Benchmarks)*, 2023.
- [42] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [43] H. Liu, C. Li, Y. Li, and Y. J. Lee. Improved baselines with visual instruction tuning. *arXiv preprint arXiv:2310.03744*, 2023.
- [44] S. Karamcheti, S. Nair, A. Balakrishna, P. Liang, T. Kollar, and D. Sadigh. Prismatic vlms: Investigating the design space of visually-conditioned language models. *arXiv preprint arXiv:2402.07865*, 2024.
- [45] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, et al. QT-Opt: Scalable deep reinforcement learning for vision-based robotic manipulation. *arXiv preprint arXiv:1806.10293*, 2018.
- [46] D. Kalashnikov, J. Varley, Y. Chebotar, B. Swanson, R. Jonschkowski, C. Finn, S. Levine, and K. Hausman. Mt-opt: Continuous multi-task robotic reinforcement learning at scale. *arXiv*, 2021.
- [47] F. Ebert, Y. Yang, K. Schmeckpeper, B. Bucher, G. Georgakis, K. Daniilidis, C. Finn, and S. Levine. Bridge data: Boosting generalization of robotic skills with cross-domain datasets. *arXiv preprint arXiv:2109.13396*, 2021.
- [48] K. Ehsani, T. Gupta, R. Hendrix, J. Salvador, L. Weihs, K.-H. Zeng, K. P. Singh, Y. Kim, W. Han, A. Herrasti, et al. Imitating shortest paths in simulation enables effective navigation and manipulation in the real world. *arXiv preprint arXiv:2312.02976*, 2023.
- [49] H. Bharadhwaj, J. Vakil, M. Sharma, A. Gupta, S. Tulsiani, and V. Kumar. Roboagent: Generalization and efficiency in robot manipulation via semantic augmentations and action chunking. *arXiv preprint arXiv:2309.01918*, 2023.
- [50] L. Pinto and A. Gupta. Supersizing self-supervision: Learning to grasp from 50k tries and 700 robot hours. In *2016 IEEE international conference on robotics and automation (ICRA)*, pages 3406–3413. IEEE, 2016.

- [51] A. Mandlekar, Y. Zhu, A. Garg, J. Booher, M. Spero, A. Tung, J. Gao, J. Emmons, A. Gupta, E. Orbay, et al. Roboturk: A crowdsourcing platform for robotic skill learning through imitation. In *Conference on Robot Learning*, pages 879–893. PMLR, 2018.
- [52] A. Gupta, A. Murali, D. P. Gandhi, and L. Pinto. Robot learning in homes: Improving generalization and reducing dataset bias. *Advances in neural information processing systems*, 31, 2018.
- [53] S. Dasari, F. Ebert, S. Tian, S. Nair, B. Bucher, K. Schmeckpeper, S. Singh, S. Levine, and C. Finn. Robonet: Large-scale multi-robot learning. *CoRL*, 2019.
- [54] S. Cabi, S. G. Colmenarejo, A. Novikov, K. Konyushkova, S. Reed, R. Jeong, K. Zolna, Y. Aytar, D. Budden, M. Vecerik, O. Sushkov, D. Barker, J. Scholz, M. Denil, N. de Freitas, and Z. Wang. Scaling data-driven robotics with reward sketching and batch reinforcement learning. *RSS*, 2019.
- [55] E. Jang, A. Irpan, M. Khansari, D. Kappler, F. Ebert, C. Lynch, S. Levine, and C. Finn. Bc-z: Zero-shot task generalization with robotic imitation learning. In *Conference on Robot Learning*, pages 991–1002. PMLR, 2022.
- [56] H.-S. Fang, H. Fang, Z. Tang, J. Liu, C. Wang, J. Wang, H. Zhu, and C. Lu. Rh20t: A comprehensive robotic dataset for learning diverse skills in one-shot. *Towards Generalist Robots: Learning Paradigms for Scalable Skill Acquisition@CoRL2023*, 3:5, 2023.
- [57] C. Devin, A. Gupta, T. Darrell, P. Abbeel, and S. Levine. Learning modular neural network policies for multi-task and multi-robot transfer. In *Proceedings of IEEE International Conference on Robotics and Automation*, 2017.
- [58] E. S. Hu, K. Huang, O. Rybkin, and D. Jayaraman. Know thyself: Transferable visual control policies through robot-awareness. In *International Conference on Learning Representations*, 2022.
- [59] J. H. Yang, D. Sadigh, and C. Finn. Polybot: Training one policy across robots while embracing variability. In *7th Annual Conference on Robot Learning*, 2023. URL <https://openreview.net/forum?id=HEIRj51lcS>.
- [60] S. Reed, K. Zolna, E. Parisotto, S. G. Colmenarejo, A. Novikov, G. Barthmaron, M. Giménez, Y. Sulsky, J. Kay, J. T. Springenberg, T. Eccles, J. Bruce, A. Razavi, A. Edwards, N. Heess, Y. Chen, R. Hadsell, O. Vinyals, M. Bordbar, and N. de Freitas. A generalist agent. *Transactions on Machine Learning Research*, 2022. ISSN 2835-8856.

- [61] G. Salhotra, I.-C. A. Liu, and G. Sukhatme. Bridging action space mismatch in learning from demonstrations. *arXiv preprint arXiv:2304.03833*, 2023.
- [62] I. Radosavovic, B. Shi, L. Fu, K. Goldberg, T. Darrell, and J. Malik. Robot learning with sensorimotor pre-training. In *Conference on Robot Learning*, 2023.
- [63] D. Shah, A. Sridhar, A. Bhorkar, N. Hirose, and S. Levine. Gnm: A general navigation model to drive any robot. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7226–7233. IEEE, 2023.
- [64] K. Bousmalis, G. Vezzani, D. Rao, C. Devin, A. X. Lee, M. Bauza, T. Davchev, Y. Zhou, A. Gupta, A. Raju, et al. Robocat: A self-improving foundation agent for robotic manipulation. *arXiv preprint arXiv:2306.11706*, 2023.
- [65] D. Shah, A. Sridhar, N. Dashora, K. Stachowicz, K. Black, N. Hirose, and S. Levine. ViNT: A foundation model for visual navigation. In *7th Annual Conference on Robot Learning*, 2023. URL <https://arxiv.org/abs/2306.14846>.
- [66] J. Yang, C. Glossop, A. Bhorkar, D. Shah, Q. Vuong, C. Finn, D. Sadigh, and S. Levine. Pushing the limits of cross-embodiment learning for manipulation and navigation. *arXiv preprint arXiv:2402.19432*, 2024.
- [67] S. Y. Gadre, M. Wortsman, G. Ilharco, L. Schmidt, and S. Song. Cows on pasture: Baselines and benchmarks for language-driven zero-shot object navigation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 23171–23181, 2023.
- [68] Y. Du, K. Konyushkova, M. Denil, A. Raju, J. Landon, F. Hill, N. de Freitas, and S. Cabi. Vision-language models as success detectors. *arXiv preprint arXiv:2303.07280*, 2023.
- [69] Y. J. Ma, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman. Liv: Language-image representations and rewards for robotic control. In *International Conference on Machine Learning*, pages 23301–23320. PMLR, 2023.
- [70] X. Zhang, Y. Ding, S. Amiri, H. Yang, A. Kaminski, C. Esselink, and S. Zhang. Grounding classical task planners via vision-language models. *arXiv preprint arXiv:2304.08587*, 2023.
- [71] S. Sontakke, J. Zhang, S. Arnold, K. Pertsch, E. Bıyık, D. Sadigh, C. Finn, and L. Itti. Roboclip: One demonstration is enough to learn robot policies. *Advances in Neural Information Processing Systems*, 36, 2024.

- [72] J. Huang, S. Yong, X. Ma, X. Linghu, P. Li, Y. Wang, Q. Li, S.-C. Zhu, B. Jia, and S. Huang. An embodied generalist agent in 3d world. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [73] X. Li, M. Liu, H. Zhang, C. Yu, J. Xu, H. Wu, C. Cheang, Y. Jing, W. Zhang, H. Liu, et al. Vision-language foundation models as effective robot imitators. *arXiv preprint arXiv:2311.01378*, 2023.
- [74] H. Zhen, X. Qiu, P. Chen, J. Yang, X. Yan, Y. Du, Y. Hong, and C. Gan. 3d-vla: 3d visionlanguage-action generative world model. *arXiv preprint arXiv:2403.09631*, 2024.
- [75] PyTorch. Automatic mixed precision. URL <https://pytorch.org/docs/stable/amp.html>.
- [76] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- [77] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright, H. Shojanazeri, M. Ott, S. Shleifer, et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023.
- [78] N. Dorka, C. Huang, T. Welschhold, and W. Burgard. What matters in employing vision language models for tokenizing actions in robot control? In *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*.
- [79] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pretraining. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11975–11986, 2023.
- [80] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.
- [81] P. Sharma, N. Ding, S. Goodman, and R. Soricut. Conceptual captions: A cleaned, hypernymed, image alt-text dataset for automatic image captioning. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2556–2565, 2018.
- [82] C. Schuhmann, R. Vencu, R. Beaumont, R. Kaczmarczyk, C. Mullis, A. Katta, T. Coombes, J. Jitsev, and A. Komatsuzaki. Laion-400m: Open dataset of

clip-filtered 400 million image-text pairs. *arXiv preprint arXiv:2111.02114*, 2021.

[83] O. Sidorov, R. Hu, M. Rohrbach, and A. Singh. Textcaps: a dataset for image captioning with reading comprehension. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part II 16*, pages 742–758. Springer, 2020.

[84] H. Face. Introducing idefics: An open reproduction of state-of-the-art visual language model. *Hugging Face Blog*, 2024.

[85] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36, 2024.

[86] B. McKinzie, Z. Gan, J.-P. Fauconnier, S. Dodge, B. Zhang, P. Dufter, D. Shah, X. Du, F. Peng, F. Weers, et al. Mm1: Methods, analysis & insights from multimodal llm pre-training. *arXiv preprint arXiv:2403.09611*, 2024.

[87] J. Lin, H. Yin, W. Ping, Y. Lu, P. Molchanov, A. Tao, H. Mao, J. Kautz, M. Shoenybi, and S. Han. Vila: On pre-training for visual language models. *arXiv preprint arXiv:2312.07533*, 2023.

[88] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35: 30318–30332, 2022.

[89] NVIDIA. Tensorrt-llm. URL <https://github.com/NVIDIA/TensorRT-LLM>.

[90] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[91] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

[92] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.

[93] E. Rosete-Beas, O. Mees, G. Kalweit, J. Boedecker, and W. Burgard. Latent plans for task agnostic offline reinforcement learning. In *Proceedings of the 6th*

Conference on Robot Learning (CoRL), 2022.

[94] O. Mees, J. Borja-Diaz, and W. Burgard. Grounding language with visual affordances over unstructured data. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, London, UK, 2023.

[95] S. Dass, J. Yapeter, J. Zhang, J. Zhang, K. Pertsch, S. Nikolaidis, and J. J. Lim. CLVR jaco play dataset, 2023. URL <https://github.com/clvr-ai/clvr-jaco-play-dataset>.

[96] J. Luo, C. Xu, X. Geng, G. Feng, K. Fang, L. Tan, S. Schaal, and S. Levine. Multi-stage cable routing through hierarchical imitation learning. *arXiv preprint arXiv:2307.08927*, 2023.

[97] A. Mandlekar, Y. Zhu, A. Garg, J. Booher, M. Spero, A. Tung, J. Gao, J. Emmons, A. Gupta, E. Orbay, S. Savarese, and L. Fei-Fei. RoboTurk: A crowdsourcing platform for robotic skill learning through imitation. *CoRR*, abs/1811.02790, 2018. URL <http://arxiv.org/abs/1811.02790>.

[98] Y. Zhu, A. Joshi, P. Stone, and Y. Zhu. Viola: Imitation learning for vision-based manipulation with object proposal priors, 2023.

[99] L. Y. Chen, S. Adebola, and K. Goldberg. Berkeley UR5 demonstration dataset. <https://sites.google.com/view/berkeley-ur5/home>.

[100] G. Zhou, V. Dean, M. K. Srirama, A. Rajeswaran, J. Pari, K. Hatch, A. Jain, T. Yu, P. Abbeel, L. Pinto, C. Finn, and A. Gupta. Train offline, test online: A real robot learning benchmark, 2023.

[101] C. Lynch, A. Wahid, J. Thompson, T. Ding, J. Betker, R. Baruch, T. Armstrong, and P. Florence. Interactive language: Talking to robots in real time. *IEEE Robotics and Automation Letters*, 2023.

[102] S. Belkhale, Y. Cui, and D. Sadigh. Hydra: Hybrid robot actions for imitation learning. *arxiv*, 2023.

[103] Y. Zhu, P. Stone, and Y. Zhu. Bottom-up skill discovery from unsegmented demonstrations for long-horizon robot manipulation. *IEEE Robotics and Automation Letters*, 7(2):4126–4133, 2022.

[104] Z. J. Cui, Y. Wang, N. M. M. Shafiullah, and L. Pinto. From play to policy: Conditional behavior generation from uncured robot data. *arXiv preprint*

arXiv:2210.10047, 2022.

[105] M. Heo, Y. Lee, D. Lee, and J. J. Lim. Furniturebench: Reproducible real-world benchmark for long-horizon complex manipulation. In *Robotics: Science and Systems*, 2023.

[106] G. Yan, K. Wu, and X. Wang. ucsd kitchens Dataset. August 2023.

[107] S. Nasiriany, T. Gao, A. Mandlekar, and Y. Zhu. Learning and retrieval from prior data for skill-based imitation learning. In *Conference on Robot Learning (CoRL)*, 2022.

[108] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu. Robot learning on the job: Human-in-the-loop autonomy and learning during deployment. In *Robotics: Science and Systems (RSS)*, 2023.

[109] G. Quere, A. Hagengruber, M. Iskandar, S. Bustamante, D. Leidner, F. Stulp, and J. Vogel. Shared Control Templates for Assistive Robotics. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, page 7, Paris, France, 2020.

[110] S. Saxena, M. Sharma, and O. Kroemer. Multi-resolution sensing for real-time control with vision-language models. In *7th Annual Conference on Robot Learning*, 2023. URL <https://openreview.net/forum?id=WuBv9-IGDUA>.

[111] R. Shah, R. Martín-Martín, and Y. Zhu. MUTEX: Learning unified policies from multimodal task specifications. In *7th Annual Conference on Robot Learning*, 2023. URL <https://openreview.net/forum?id=PwqigaaEzJ>.

[112] X. Zhu, R. Tian, C. Xu, M. Ding, W. Zhan, and M. Tomizuka. Fanuc manipulation: A dataset for learning-based manipulation with fanuc mate 200id robot. 2023.

[113] R. Mendonca, S. Bahl, and D. Pathak. Structured world models from human videos. *CoRL*, 2023.

[114] J. Luo, C. Xu, F. Liu, L. Tan, Z. Lin, J. Wu, P. Abbeel, and S. Levine. Fmb: a functional manipulation benchmark for generalizable robotic learning. *arXiv preprint arXiv:2401.08553*, 2024.

[115] N. M. M. Shafiullah, A. Rai, H. Etukuru, Y. Liu, I. Misra, S. Chintala, and L. Pinto. On bringing robots home, 2023.

[116] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36, 2024.

[117] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

A Data Mixture Details

A 数据混合细节

We list our used data mixture in Table 3. The mixture mostly follows [5], with a few additional datasets.

我们在表3中列出了我们使用的数据混合。该混合主要遵循[5]，并添加了一些额外的数据集。

OpenVLA Training Dataset Mixture	
Fractal [92]	12.7%
Kuka [45]	12.7%
Bridge[6, 47]	13.3%
Taco Play [93, 94]	3.0%
Jaco Play [95]	0.4%
Berkeley Cable Routing [96]	0.2%
Roboturk [97]	2.3%
Viola [98]	0.9%
Berkeley Autolab UR5 [99]	1.2%
Toto [100]	2.0%
Language Table [101]	4.4%
Stanford Hydra Dataset [102]	4.4%
Austin Buds Dataset [103]	0.2%
NYU Franka Play Dataset [104]	0.8%
Furniture Bench Dataset [105]	2.4%
UCSD Kitchen Dataset [106]	<0.1%
Austin Sailor Dataset [107]	2.2%
Austin Sirius Dataset [108]	1.7%
DLR EDAN Shared Control [109]	<0.1%
IAMLab CMU Pickup Insert [110]	0.9%
UTAustin Mutex [111]	2.2%
Berkeley Fanuc Manipulation [112]	0.7%
CMU Stretch [113]	0.2%
BC-Z [55]	7.5%
FMB Dataset [114]	7.1%
DobbE [115]	1.4%
DROID [11]	10.0% ⁶

Table 3: OpenVLA training data mixture using datasets from the Open X-Embodiment dataset [1], following [5] with a few additions.

表 3: OpenVLA 训练数据混合，使用来自 Open X-Embodiment 数据集 [1] 的数据集，遵循 [5] 并进行了一些添加。

B Evaluation Tasks and Detailed Results

B 评估任务与详细结果

In this section, we provide more details on the BridgeData V2 WidowX and Google robot evaluations discussed in Section 5.1, as well as the Franka-Tabletop and Franka-DROID fine-tuning evaluations discussed in Section 5.2.

在本节中，我们将提供关于第5.1节中讨论的BridgeData V2 WidowX和Google机器人评估的更多细节，以及第5.2节中讨论的Franka-Tabletop和Franka-DROID微调评估的更多细节。

B.1 BridgeData V2 WidowX Evaluation Details

B.1 BridgeData V2 WidowX 评估细节

Here we focus specifically on BridgeData V2 evaluations discussed in Section 5.1.

在此，我们特别关注第5.1节中讨论的BridgeData V2评估。

B.1.1 BridgeData V2 Evaluation Tasks

B.1.1 BridgeData V2 评估任务

As described in Section 5.1, we evaluate each generalist robot manipulation policy on 17 tasks with 10 trials each. In this section, we provide details on the task categories and individual tasks.

正如第 5.1 节所述，我们评估了每个通用机器人操作策略在 17 个任务上的表现，每个任务进行 10 次试验。在本节中，我们将详细介绍任务类别和各个任务。

In total, we evaluate on 5 visual generalization tasks, 2 motion generalization tasks, 3 physical generalization tasks, 4 semantic generalization tasks, and 3 language grounding tasks. Note that all tasks we evaluate on introduce some form of distribution shift since we are unable to procure the exact objects used in the original dataset (other distribution shifts naturally arise as we reproduce a real-world test environment originally constructed at a different location; see Appendix B.1.2 for a detailed discussion on such distribution shifts). All 17 tasks are depicted in Fig. 7. Each rollout is

总共，我们评估了 5 个视觉泛化任务、2 个运动泛化任务、3 个物理泛化任务、4 个语义泛化任务和 3 个语言基础任务。请注意，我们评估的所有任务都引入了某种形式的分布偏移，因为我们无法获得原始数据集中使用的确切对象（其他的分布偏移自然会产生，因为我们再现了一个最初在不同地点构建的真实世界测试环境；有关此类分布偏移的详细讨论，请参见附录 B.1.2）。所有 17 个任务如图 7 所示。每次展开是

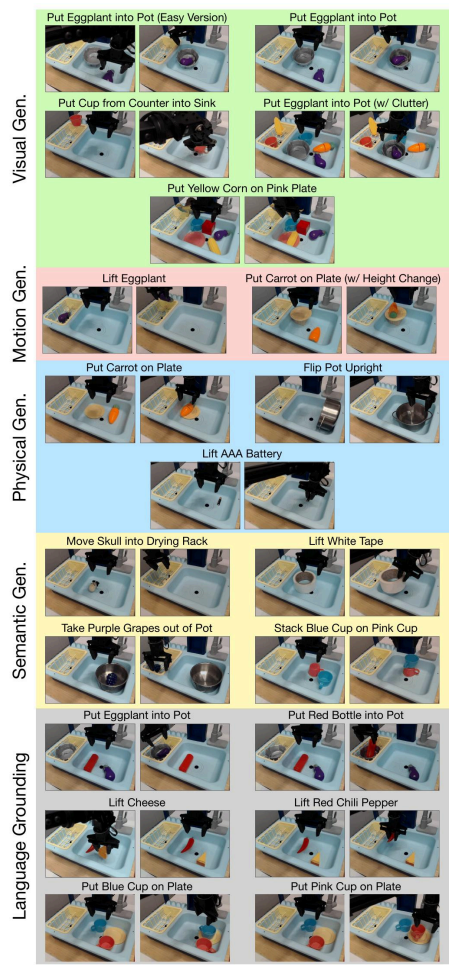


Figure 7: **BridgeData V2 WidowX robot evaluation tasks.** We evaluate every generalist robot policy on 4 types out-of-distribution (OOD) generalization tasks: **visual**, **motion**, **physical**, and **semantic** (as defined in Section 5.1). Every pair of images shows the start state and an example end state after the robot completes the task. We also rigorously assess **language grounding** in the 3 tasks shown in the bottom 3 rows, by changing the prompt while fixing the initial state and testing whether the policy can approach the correct target object.

图 7: BridgeData V2 WidowX 机器人评估任务。我们在 4 种类型的分布外 (OOD) 泛化任务上评估每个通用机器人策略: 视觉、运动、物理和语义 (如第 5.1 节中所定义)。每对图像显示了机器人完成任务后的起始状态和示例结束状态。我们还通过在固定初始状态的同时更改提示, 并测试策略是否可以接近正确的目标对象, 从而严格评估了底部 3 行中所示的 3 个任务中的语言基础。

marked as a failure (0) or success (1). In some more difficult tasks, we record partial successes (0.5); we describe the conditions for partial credit in the task descriptions below.

标记为失败 (0) 或成功 (1)。在一些更困难的任务中, 我们记录部分成功 (0.5); 我们将在下面的任务描述中描述获得部分分数的条件。

Below we describe each of the 17 tasks, in the order shown in Fig. 7:

下面我们按图 7 所示的顺序描述 17 项任务中的每一项:

1. **Put Eggplant into Pot (Easy Version):** The robot’s goal is to pick up the eggplant and drop it into the pot. This is a **visual generalization** task because we use a handcrafted paper pot that has a different appearance than the pot used in the original BridgeData V2 training dataset (since we are unable to procure the original pot). Unlike all 16 other tasks, for this particular task we initialize the robot’s end-effector directly above the eggplant before rolling out the policy; hence, we call this the “Easy Version” of the “Put Eggplant into Pot” task.

1. 将茄子放入锅中（简单版本）：机器人的目标是拿起茄子并将其放入锅中。这是一个视觉泛化任务，因为我们使用了一个手工制作的纸锅，它与原始BridgeData V2训练数据集中使用的锅具有不同的外观（因为我们无法获得原始的锅）。与所有其他16个任务不同，对于这个特定任务，我们在展开策略之前，将机器人的末端执行器直接初始化在茄子的上方；因此，我们称之为“将茄子放入锅中”任务的“简单版本”。

2. **Put Eggplant into Pot:** This is the same task as described above, except that the robot’s end-effector is not initialized directly above the eggplant. Instead, we initialize it in a position that is fixed across all rollouts, which means that the robot must horizontally reach for the eggplant first before manipulating it. (Note: The same applies to all other tasks described below.) This is a **visual generalization** task for the same reason as above.

2. 将茄子放入锅中：此任务与上述任务相同，不同之处在于机器人的末端执行器并非直接在茄子上方初始化。相反，我们将其初始化在一个在所有展开中固定的位置，这意味着机器人必须先水平够到茄子，然后才能操作它。（注意：以下描述的所有其他任务均适用。）由于与上述相同的原因，这是一个视觉泛化任务。

3. **Put Cup from Counter into Sink:** The robot’s goal is to pick up the pink cup from either the kitchen countertop or drying rack and place it into the sink on the right. This is a **visual generalization** task because we use a pink cup rather than a blue cup (a blue cup is used in the original BridgeData V2 dataset, but we find that none of the methods we evaluate is able to manipulate it reliably – most likely because the color of the cup blends in with the color of the sink).

3. 将杯子从台面放入水槽：机器人的目标是从厨房台面或晾碗架上拿起粉色杯子，然后将其放入右侧的水槽中。这是一项视觉泛化任务，因为我们使用粉色杯子而不是蓝色杯子（蓝色杯子在原始的BridgeData V2数据集中使用，但我们发现我们评估的所有方法都无法可靠地操作它——很可能是因为杯子的颜色与水槽的颜色融为一体）。

4. **Put Eggplant into Pot (w/ Clutter):** This is the same task as the “Put Eggplant into Pot” task, except that it is more difficult due to the presence of

several distractor objects. It is a **visual generalization** task for the same reason discussed in the normal “Put Eggplant into Pot” task, and even more so given unseen distractors in the scene. *Partial credit (0.5 out of 1) is rewarded when the robot moves towards the correct target object.*

4. 将茄子放入锅中（有杂物）：此任务与“将茄子放入锅中”任务相同，但由于存在多个干扰物，因此难度更高。由于与普通的“将茄子放入锅中”任务中讨论的原因相同，这是一个视觉泛化任务，并且考虑到场景中未见过的干扰物，更是如此。当机器人朝着正确的目标物体移动时，会给予部分分数（满分 1 分，得 0.5 分）。

5. **Put Yellow Corn on Pink Plate:** The robot’s goal is to pick up the yellow corn and place it on the pink plate. This is a **visual generalization** task due to the presence of unseen distractor objects in the scene, such as a green dinosaur on the countertop in the back section of the sink. *Partial credit (0.5 out of 1) is rewarded when the robot moves towards the correct target object.*

5. 将黄色玉米放在粉色盘子上：机器人的目标是拿起黄色玉米并将其放在粉色盘子上。由于场景中存在未见过的干扰物，例如水槽后部台面上的绿色恐龙，因此这是一项视觉泛化任务。当机器人朝着正确的目标物体移动时，会获得部分分数（满分 1 分，得 0.5 分）。

6. **Lift Eggplant:** The robot’s goal is to grasp and lift the eggplant into the air. This is a **motion generalization** task because the eggplant is initialized in unseen positions and/or orientations, and the robot is forced to move beyond its training distribution of positions and/or orientations and often perform long-range reaching in order to complete the task. (Note: Long-range reaching is not demonstrated in this environment in the original BridgeData V2 demonstrations; see Appendix B.1.2 for details.) We find that this task, though seemingly simple, is deceptively challenging for many policies. *Partial credit (0.5 out of 1) is rewarded when the robot makes contact with the eggplant.*

6. 抬起茄子：机器人的目标是抓住茄子并将其抬到空中。这是一个运动泛化任务，因为茄子被初始化在未见过的位置和/或方向上，并且机器人被迫超出其位置和/或方向的训练分布进行移动，并且经常需要执行长距离的抓取才能完成任务。（注意：在原始的BridgeData V2演示中，此环境中未展示长距离抓取；有关详细信息，请参见附录B. 1. 2。）我们发现，尽管这项任务看似简单，但对于许多策略来说都具有挑战性。当机器人与茄子接触时，会获得部分分数（满分1分，得0.5分）。

7. **Put Carrot on Plate (w/ Height Change):** The robot’s goal is to pick up the carrot and place it on the yellow plate. This is a **motion generalization** task because the plate is elevated from its usual position at the bottom of the sink, and the robot must adjust its trajectory to correctly place the carrot on the

elevated platform (without knocking down the plate in the process). *Partial credit (0.5 out of 1) is rewarded when the robot grasps the carrot and touches the plate with it.*

7. 将胡萝卜放到盘子上（含高度变化）：机器人的目标是拿起胡萝卜并将其放到黄色盘子上。这是一个运动泛化任务，因为盘子从其通常位于水槽底部的位置被抬高，机器人必须调整其轨迹，以便正确地将胡萝卜放到抬高的平台上（在此过程中不会撞倒盘子）。当机器人抓住胡萝卜并用它接触到盘子时，将获得部分分数（满分 1 分，得 0.5 分）。

8. **Put Carrot on Plate:** This is the same task as above, except that the plate is at its normal position (at the bottom of the sink or drying rack). We consider this a **physical generalization** task because the carrot has a different size and shape than the one used in the original BridgeData V2 dataset, which is shorter and narrower. (Note that the previous version of this task listed above would also technically be a physical generalization task since it involves the same carrot, but we list it under the “motion generalization” category since that is the focus there.)

8. 将胡萝卜放到盘子上：此任务与上述任务相同，只是盘子位于其正常位置（在水槽或沥水架的底部）。我们认为这是一个物理泛化任务，因为胡萝卜的尺寸和形状与原始 BridgeData V2 数据集中使用的胡萝卜不同，后者更短更窄。（请注意，上面列出的此任务的先前版本在技术上也是一个物理泛化任务，因为它涉及相同的胡萝卜，但我们将其列在“运动泛化”类别下，因为这是重点所在。）

9. **Flip Pot Upright:** The robot’s goal is to manipulate the pot such that it is oriented upright in the sink at the end of the episode. This is a **physical generalization** task because this pot has a different size and shape than the one used in the original BridgeData V2 training demonstrations (the pot we use is wider and shorter).

9. 将锅翻正：机器人的目标是操纵锅，使其在剧集结束时在水槽中直立。这是一项物理泛化任务，因为这个锅的尺寸和形状与原始BridgeData V2训练演示中使用的锅不同（我们使用的锅更宽更矮）。

10. **Lift AAA Battery:** The robot’s goal is simply to grasp the AAA battery and lift it up into the air. This is considered a **physical generalization** task because the battery is much smaller and thinner than target objects seen in the BridgeData V2 training demonstrations in this environment; see Appendix B.1.2 for details. (Note that this target object does not exist in the original BridgeData V2 demonstrations in this environment, so this is also an instance of “semantic generalization”, but we classify it solely as “physical generalization” since that is the main focus here).

10. 举起AAA电池：机器人的目标很简单，就是抓住AAA电池并将其举到空中。这被认为是一项物理泛化任务，因为电池比该环境中BridgeData V2训练演示中看到的目标物体小得多且薄得多；详见附录B.1.2。（请注意，此目标物体在原始BridgeData V2演示中不存在，因此这也是“语义泛化”的一个实例，但我们仅将其归类为“物理泛化”，因为这是此处的重点。）

11. **Move Skull into Drying Rack:** The robot's goal is to grasp the skull windup toy and drop it into the yellow drying rack in the left part of the sink. This is a **semantic generalization** task since the skull is an unseen target object (does not appear in the BridgeData V2 training demonstrations).

11. 将骷髅头移动到晾碗架中：机器人的目标是抓住骷髅头上弦玩具，并将其放入水槽左侧的黄色晾碗架中。这是一个语义泛化任务，因为骷髅头是一个未见过的目标物体（未出现在 BridgeData V2 训练演示中）。

12. **Lift White Tape:** The robot's goal is to grasp and lift the white roll of tape into the air. This is a **semantic generalization** task since the white tape roll is an unseen target object (does not appear in the BridgeData V2 training demonstrations). (Note that this task may also be considered as “physical generalization” because of its shape being different than the objects seen in the training demonstrations in this environment; most policies struggle to grasp objects with this ring structure, and they often move the robot's end-effector directly into the center region.)

12. 提起白色胶带：机器人的目标是抓住并提起白色胶带卷到空中。这是一个语义泛化任务，因为白色胶带卷是一个未见过的目标物体（未出现在 BridgeData V2训练演示中）。（请注意，由于其形状与此环境中训练演示中见过的物体不同，此任务也可被视为“物理泛化”；大多数策略难以抓住具有这种环状结构的物体，并且它们经常将机器人的末端执行器直接移动到中心区域。）

13. **Take Purple Grapes out of Pot:** The robot's goal is to grasp the purple grapes lying inside the steel pot and remove it from the pot (by lifting it out and/or dropping it anywhere outside the pot). This is a **semantic generalization** task because it is an unseen language instruction; the robot has never seen this task in the original BridgeData V2 training dataset.

13. 从罐中取出紫色葡萄：机器人的目标是抓住位于钢罐内的紫色葡萄，并将其从罐中取出（通过将其提起和/或将其掉落在罐外的任何位置）。这是一项语义泛化任务，因为这是一条未见过的语言指令；机器人在原始 BridgeData V2训练数据集中从未见过此任务。

14. Stack Blue Cup on Pink Cup: The robot's goal is to grasp the blue cup and place it securely on top of the pink cup. This is a **semantic generalization** task because it is an unseen language instruction; the robot has never seen this task in this environment in the original BridgeData V2 training dataset. *Partial credit (0.5 out of 1) is rewarded when the robot grasps the blue cup and touches the pink cup with the blue cup.*

14. 将蓝色杯子堆叠在粉色杯子上：机器人的目标是抓住蓝色杯子并将其稳固地放置在粉色杯子的顶部。这是一个语义泛化任务，因为这是一个未见过的语言指令；机器人在原始BridgeData V2训练数据集中从未见过此任务。当机器人抓住蓝色杯子并用蓝色杯子接触粉色杯子时，将获得部分分数（满分1分，得0.5分）。

15. Put {Eggplant, Red Bottle} into Pot: This is a **language grounding** task. The robot's goal is to put the specified target object into the pot. Both the eggplant and red bottle are present in the scene. We conduct paired evaluations: for the same initial state, we prompt the policy to target the eggplant in one episode, and then the red bottle in the next episode. We test each method 5 times with the eggplant and 5 times with the red bottle, using the same set of 5 initial states for both target objects. *Partial credit (0.5 out of 1) is rewarded when the robot moves towards the correct target object.*

15. 将{茄子，红色瓶子}放入锅中：这是一项语言基础任务。机器人的目标是将指定的目标物体放入锅中。茄子和红色瓶子都出现在场景中。我们进行配对评估：对于相同的初始状态，我们提示策略在一个episode中以茄子为目标，然后在下一个episode中以红色瓶子为目标。我们使用相同的5个初始状态集合，对每种方法用茄子测试5次，用红色瓶子测试5次。当机器人朝着正确的物体移动时，奖励部分分数（满分1分，得0.5分）。

16. Lift {Cheese, Red Chili Pepper}: This is a **language grounding** task. The robot's goal is to grasp and lift the specified target object. We conduct paired evaluations as described in the task above. *Partial credit (0.5 out of 1) is rewarded when the robot moves towards the correct target object.*

16. 抬起{奶酪，红辣椒}：这是一个语言基础任务。机器人的目标是抓住并抬起指定的目标物体。我们按照上述任务描述进行配对评估。当机器人朝着正确的目标物体移动时，会给予部分分数（1分中的0.5分）。

17. Put {Blue Cup, Pink Cup} on Plate: This is a **language grounding** task. The robot's goal is to grasp the specified target object and place it onto the plate. We conduct paired evaluations as described in other language grounding tasks. *Partial credit (0.5 out of 1) is rewarded when the robot moves towards the correct target object.*

17. 将{蓝色杯子, 粉色杯子}放在盘子上: 这是一项语言基础任务。机器人的目标是抓住指定的目标物体并将其放置在盘子上。我们按照其他语言基础任务中所述进行配对评估。当机器人朝着正确的目标物体移动时, 会奖励部分分数(满分1分, 得0.5分)。

B.1.2 Comparing Evaluation Tasks to Original BridgeData V2 Training Data

B.1.2 评估任务与原始BridgeData V2训练数据对比

We conduct our evaluations in a sink environment used in the original BridgeData V2 dataset [6]. We reproduce the environment to match the original environment in the BridgeData V2 dataset with rough approximations for the robot’s location relative to the sink, as well as the camera’s placement relative to the scene. Given the lack of precise measurements of these positions in the original dataset, we are unable to reproduce the *exact* environment setup, and natural distribution shifts arise due to slightly different robot, sink, and camera placements. In addition, since we evaluate robot policies in a different location than where the training demonstrations were collected from, other natural distribution shifts arise. For example, the lighting conditions and background (e.g., visible areas behind the sink) are inevitably different than what was seen in the training dataset. Furthermore, we are unable to procure the exact set of objects used in the original BridgeData V2 dataset, so there are distribution shifts between the objects used at train time and those used at test time.

我们在原始BridgeData V2数据集[6]中使用的水槽环境中进行评估。我们重现了该环境, 使其与BridgeData V2数据集中的原始环境相匹配, 并对机器人相对于水槽的位置以及相机相对于场景的放置进行了粗略的近似。鉴于原始数据集中缺乏对这些位置的精确测量, 我们无法重现完全相同的环境设置, 并且由于机器人、水槽和相机位置的略微不同, 自然会出现分布偏移。此外, 由于我们在与收集训练演示不同的位置评估机器人策略, 因此会出现其他自然分布偏移。例如, 照明条件和背景(例如, 水槽后面可见的区域)不可避免地会与训练数据集中看到的不同。此外, 我们无法获得原始BridgeData V2数据集中使用的确切对象集, 因此在训练时使用的对象和测试时使用的对象之间存在分布偏移。

Despite all these challenges, we find that certain generalist policies, such as OpenVLA and RT-2-X, can still generalize and perform various tasks fairly reliably “out-of-the-box”. Other generalist policies, such as RT-1-X and Octo, can also complete some tasks, though they struggle when tested with more difficult generalization tasks in our BridgeData V2 evaluation suite.

尽管存在所有这些挑战, 我们发现某些通用策略, 例如 OpenVLA 和 RT-2-X, 仍然可以相当可靠地“开箱即用”地推广并执行各种任务。其他通用策略, 例如

RT-1-X 和 Octo, 也可以完成一些任务, 但当在我们的 BridgeData V2 评估套
 件中使用更困难的泛化任务进行测试时, 它们会遇到困难。

The original BridgeData V2 dataset includes demonstrations of the following seven tasks in this specific sink environment: “Flip Pot Upright”, “Put Carrot on Plate”, “Put Cup from Counter (or Drying Rack) into Sink”, “Put Eggplant into Pot”, “Put Knife on Cutting Board”, “Put Spoon in Pot”, and “Turn Lever Vertical to Front”. See Fig. 8 for samples images of all these tasks from the original dataset. Note that all training demonstrations collected in this environment are initialized such that the robot’s end-effector is positioned directly above the target object in the beginning of the episode. (However, this is not the case across all environments in the BridgeData V2 dataset; in some other environments, the robot is initialized farther away from the target object, so it must horizontally reach for the object first before manipulating it.)

最初的 BridgeData V2 数据集包括以下七个任务的演示, 这些任务均在特定的水槽环境中进行: “翻转罐子使其直立”、“将胡萝卜放在盘子上”、“将杯子从柜台(或晾碗架)放入水槽”、“将茄子放入锅中”、“将刀放在砧板上”、“将勺子放入锅中”以及“将控制杆垂直转向前方”。有关原始数据集中所有这些任务的样本图像, 请参见图 8。请注意, 在此环境中收集的所有训练演示都进行了初始化, 使得机器人的末端执行器在剧集开始时直接位于目标对象的正上方。(但是, BridgeData V2 数据集中的所有环境并非如此; 在其他一些环境中, 机器人的初始化位置距离目标对象较远, 因此它必须先水平地够到该对象, 然后才能对其进行操作。)



Figure 8: **Original BridgeData V2 sink environment tasks.** Images from sample demonstrations in the sink environment from the original BridgeData V2 dataset reveal that all demonstrations in this environment were initialized such that the robot’s end-effector was positioned immediately above the target object. Note that these initial states are different from the initial states we use in our BridgeData V2 evaluation tasks shown in Fig. 7. In our evaluations, we always initialize the robot’s end-effector to a fixed location above the sink, rather than positioning it directly above the target object (except for one task: “Put Eggplant into Pot (Easy Version)”).

图 8: 原始 BridgeData V2 水槽环境任务。来自原始 BridgeData V2 数据集中水槽环境的示例演示图像显示, 此环境中的所有演示都被初始化为使机器人的末端执行器直接位于目标对象上方。请注意, 这些初始状态与我们在图 7 所示的 BridgeData V2 评估任务中使用的初始状态不同。在我们的评估中, 我们始终将机器人的末端执行器初始化到水槽上方的固定位置, 而不是直接将其放置在目标对象上方(除了一个任务: “将茄子放入锅中(简单版)”)。

In our BridgeData V2 evaluation suite, only one task – “Put Eggplant into Pot (Easy Version)” – is initialized with the robot’s end-effector hovering directly over the target object; in all 16 other tasks, the end-effector is initialized at a fixed location above the sink such that the robot must horizontally reach towards the object. This initial condition, in combination with the distribution shifts we introduce in the various types of OOD generalization in our evaluation suite, challenges the generalist policies and requires a high degree of robustness in order to complete the tasks successfully. Hence, the success rates for policies like RT-1-X and Octo are lower than what is reported in prior works. However, we find that other policies such as RT-2-X and OpenVLA still achieve relatively strong performance despite all these distribution shifts and challenges.

在我们的BridgeData V2评估套件中，只有一项任务——“将茄子放入锅中（简单版本）”——的初始化状态是机器人的末端执行器直接悬停在目标物体上方；在其他所有16项任务中，末端执行器都被初始化在水槽上方的一个固定位置，因此机器人必须水平地伸向物体。这种初始条件，结合我们在评估套件中各种类型的OOD泛化中引入的分布偏移，对通用策略提出了挑战，并且需要高度的鲁棒性才能成功完成任务。因此，像RT-1-X和Octo这样的策略的成功率低于先前工作中报告的水平。然而，我们发现，尽管存在所有这些分布偏移和挑战，RT-2-X和OpenVLA等其他策略仍然取得了相对较强的性能。

B.1.3 Detailed BridgeData V2 Evaluation Results

B.1.3 BridgeData V2 详细评估结果

See Table 4 for the full BridgeData V2 WidowX evaluation results. The number of successes for each method, out of 10 trials, is listed for each of 17 tasks.

OpenVLA achieves strongest performance in the majority of the tasks and has the highest aggregate success rate among the generalist policies. RT-2-X also shows good performance, outperforming RT-1-X and Octo, though it does not perform as well as OpenVLA. RT-1-X and Octo generally experience difficulty in these generalization tasks.

完整 BridgeData V2 WidowX 评估结果见表 4。对于 17 项任务中的每一项，列出了每种方法的成功次数（共 10 次试验）。OpenVLA 在大多数任务中表现出最强的性能，并且在通用策略中具有最高的总体成功率。RT-2-X 也表现出良好的性能，优于 RT-1-X 和 Octo，但其性能不如 OpenVLA。RT-1-X 和 Octo 在这些泛化任务中通常会遇到困难。

Table 4: **Detailed BridgeData V2 WidowX evaluation results.** We report performance on the full evaluation suite of 17 tasks (discussed in Section 5.1), including visual/motion/physical/semantic generalization tasks and language grounding tasks. Note that *partial success* (score of 0.5) is possible for some tasks; see Appendix B.1.1 for details. We find that OpenVLA performs best in most tasks and achieves highest performance overall, followed by RT-2-X. On the other hand, RT-1-X and Octo

struggle in the evaluations, only getting 0–2 successes in several tasks. See Fig. 7 for illustrations of all tasks.

表 4: BridgeData V2 WidowX 详细评估结果。我们报告了对包含 17 个任务的完整评估套件的性能（在第 5.1 节中讨论），包括视觉/运动/物理/语义泛化任务和语言基础任务。请注意，对于某些任务，部分成功（得分为 0.5）是可能的；有关详细信息，请参见附录 B.1.1。我们发现 OpenVLA 在大多数任务中表现最佳，并实现了最高的整体性能，其次是 RT-2-X。另一方面，RT-1-X 和 Octo 在评估中表现不佳，在几个任务中仅获得 0-2 次成功。有关所有任务的说明，请参见图 7。

Category	Task	# Trials	RT-1-X # Successes	Octo # Successes	RT-2-X # Successes	OpenVLA (ours) # Successes
Visual gen	Put Eggplant into Pot (Easy Version)	10	1	5	7	10
Visual gen	Put Eggplant into Pot	10	0	1	5	10
Visual gen	Put Cup from Counter into Sink	10	1	1	0	7
Visual gen	Put Eggplant into Pot (w/ Clutter)	10	1	3.5	6	7.5
Visual gen	Put Yellow Corn on Pink Plate	10	1	4	8	9
Motion gen	Lift Eggplant	10	3	0.5	6.5	7.5
Motion gen	Put Carrot on Plate (w/ Height Change)	10	2	1	4.5	4.5
Physical gen	Put Carrot on Plate	10	1	0	1	8
Physical gen	Flip Pot Upright	10	2	6	5	8
Physical gen	Lift AAA Battery	10	0	0	2	7
Semantic gen	Move Skull into Drying Rack	10	1	0	5	5
Semantic gen	Lift White Tape	10	3	0	0	1
Semantic gen	Take Purple Grapes out of Pot	10	6	0	5	4
Semantic gen	Stack Blue Cup on Pink Cup	10	0.5	0	5.5	4.5
Language grounding	Put (Eggplant, Red Bottle) into Pot	10	2.5	4	8.5	7.5
Language grounding	Lift (Cheese, Red Chili Pepper)	10	1.5	2.5	8.5	10
Language grounding	Put (Blue Cup, Pink Cup) on Plate	10	5	5.5	8.5	9.5
Mean Success Rate			18.5±2.7%	20.0±2.6%	50.6±3.5%	70.6±3.2%

Additionally, in Table 5, we provide the full evaluation results for the quantized inference experiments that were summarized in Table 2. For these evaluations, we test policies on 8 representative BridgeData V2 tasks spanning all task categories in the full evaluation suite.

此外，在表5中，我们提供了量化推理实验的完整评估结果，这些结果在表2中进行了总结。对于这些评估，我们在8个具有代表性的BridgeData V2任务上测试策略，这些任务涵盖了完整评估套件中的所有任务类别。

Table 5: **Full quantized inference results.** Here we present the detailed version of the results shown in Table 2.

表5: 完整量化推理结果。这里我们展示了表2中所示结果的详细版本。

Category	Task	# Trials	bfloat16 # Successes	int8 # Successes	int4 # Successes
Visual gen	Put Eggplant into Pot (Easy Version)	10	9	7	9
Visual gen	Put Eggplant into Pot	10	7	7	7
Visual gen	Put Cup from Counter into Sink	10	5	3	7
Motion gen	Lift Eggplant	10	6	4	7.5
Physical gen	Put Carrot on Plate	10	6	5	7
Physical gen	Lift AAA Battery	10	7	5	3
Semantic gen	Take Purple Grapes out of Pot	10	8	8	9
Language grounding	Put (Eggplant, Red Bottle) into Pot	10	9	7.5	8
Mean Success Rate			71.3 ± 4.8%	58.1 ± 5.1%	71.9 ± 4.7%

B.2 Google Robot Evaluation Details

B.2 Google机器人评估细节

In this section, we provide more details on the Google robot evaluations introduced in Section 5.1.

在本节中，我们将提供关于第5.1节中介绍的谷歌机器人评估的更多细节。

B.2.1 Google Robot Evaluation Tasks

B.2.1 Google机器人评估任务

On the Google robot, we evaluate each generalist robot policy on 12 tasks with 5 rollouts each, for a total of 60 rollouts. The first five tasks test on in-distribution conditions, and the last seven tasks test on more difficult out-of-distribution (OOD) conditions. All tasks are depicted in Fig. 9. Each rollout is marked as a failure (0) or success (1).

在谷歌机器人上，我们评估每个通用机器人策略在12个任务上的表现，每个任务进行5次rollout，总共60次rollout。前五个任务测试的是同分布条件，后七个任务测试的是更困难的异分布（OOD）条件。所有任务都如图9所示。每次rollout都被标记为失败（0）或成功（1）。

We describe the 12 tasks below:

我们在下面描述这12个任务：

1. **Pick Coke Can** (in-distribution): The robot is positioned in front of a platform with a can of Coke on top of it. The robot's goal is to grasp and lift the Coke can.

1. 拿起可乐罐（同分布）：机器人位于一个平台上，平台上放有一罐可乐。机器人的目标是抓住并举起可乐罐。

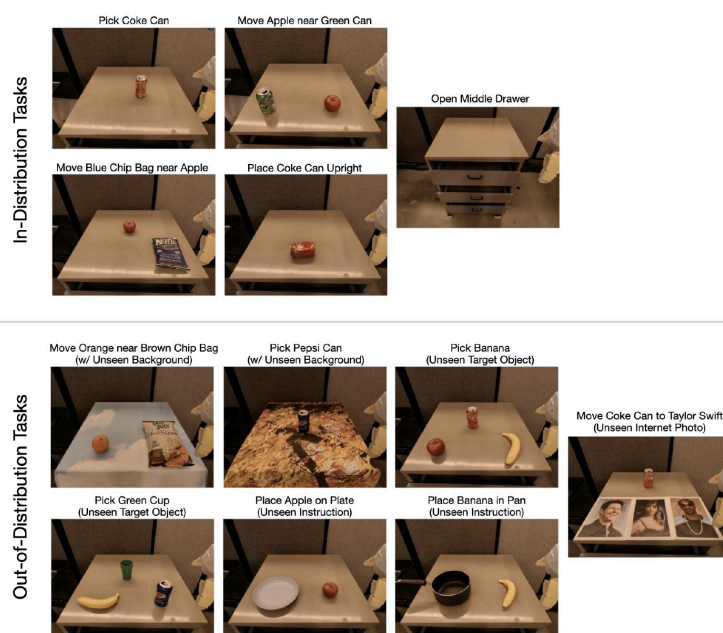


Figure 9: **Google robot evaluation tasks.** We evaluate every generalist robot policy on in-distribution tasks and out-of-distribution (OOD) generalization tasks. OOD tasks involve unseen backgrounds, target objects, instructions/object relations, and semantic concepts (e.g., photos from the Internet that do not appear in robot action data).

图 9：谷歌机器人评估任务。我们评估每一个通用机器人策略在同分布任务和异分布（OOD）泛化任务上的表现。异分布任务涉及未见过的背景、目标物体、指令/物体关系以及语义概念（例如，来自互联网的、未出现在机器人动作数据中的照片）。

2. **Move Apple near Green Can** (in-distribution): The robot is positioned in front of a platform with an apple and a green soda can on top of it. The robot's goal is to grasp the apple and move it next to the green can.

2. 将苹果移动到绿罐附近（分布内）：机器人位于一个平台前，平台上有一个苹果和一个绿色苏打罐。机器人的目标是抓住苹果并将其移动到绿罐旁边。

3. **Move Blue Chip Bag near Apple** (in-distribution): The robot is positioned in front of a platform with a blue bag of chips and an apple on top of it. The robot's goal is to grasp the blue bag of chips and move it close to the apple.

3. 移动蓝薯片袋到苹果附近（同分布）：机器人定位在一个平台上，平台上有一个蓝色薯片袋和一个苹果。机器人的目标是抓住蓝色薯片袋并将其移动到苹果附近。

4. **Place Coke Can Upright** (in-distribution): The robot is positioned in front of a platform with a can of Coke on top of it, and the can is oriented horizontally on its side. The robot's goal is to grasp the Coke can and orient it to be in a vertical position.

4. 将可乐罐直立放置（在分布内）：机器人位于一个平台上，平台上有一个可乐罐，罐子水平侧放。机器人的目标是抓住可乐罐，并将其调整到垂直位置。

5. **Open Middle Drawer** (in-distribution): The robot is positioned in front of a set of three drawers. The robot's goal is to grasp the middle drawer handle and pull the drawer open.

5. 打开中间抽屉（同分布）：机器人位于一组三个抽屉的前面。机器人的目标是抓住中间抽屉的把手并拉开抽屉。

6. **Move Orange near Brown Chip Bag** (OOD): The robot is positioned in front of a platform with a brown bag of chips and an orange on top of it. A tablecloth with blue sky and white cloud patterns covers the platform underneath the objects. The robot's goal is to grasp the orange and bring it next to the bag of chips. This task is OOD because the orange is an unseen object relative to the training dataset, and the tablecloth is an unseen background.⁷

6. 将橙子移动到棕色薯片袋附近（OOD）：机器人位于一个平台上，平台上有一个棕色的薯片袋和一个橙子。一块印有蓝天白云图案的桌布覆盖着平台。机器人的目标是抓住橙子并将其移动到薯片袋旁边。此任务是OOD，因为相对于训练数据集，橙子是一个未见过的物体，而桌布是一个未见过的背景。

7. **Pick Pepsi Can** (OOD): The robot is positioned in front of a platform with a can of Pepsi on top of it. A tablecloth with bright yellow/brown patterns covers the platform underneath the can. The robot's goal is to grasp and lift the can. This task is OOD because the Pepsi can is an unseen object, and the tablecloth is an unseen background.

7. 拿起百事可乐罐 (OOD): 机器人位于一个平台上, 平台上放着一罐百事可乐。一块带有鲜艳黄/棕色图案的桌布覆盖着罐子下方的平台。机器人的目标是抓住并举起罐子。这项任务是OOD, 因为百事可乐罐是一个未见过的物体, 而桌布是一个未见过的背景。

8. **Pick Banana** (OOD): The robot is positioned in front of a platform with an apple, a can of Coke, and a banana. The robot's goal is to grasp and lift the banana. This task is OOD because the banana is an unseen target object.

8. 摘取香蕉 (OOD): 机器人位于一个平台上, 平台上有苹果、一罐可乐和一根香蕉。机器人的目标是抓取并抬起香蕉。此任务是OOD, 因为香蕉是一个未见过的目标物体。

9. **Pick Green Cup** (OOD): The robot is positioned in front of a platform with a banana, a can of Pepsi, and a green cup. The robot's goal is to grasp and lift the green cup. This task is OOD because all objects in the scene are unseen in the training data.

9. 拿起绿色杯子 (OOD): 机器人位于一个平台前, 平台上有一个香蕉、一罐百事可乐和一个绿色杯子。机器人的目标是抓取并举起绿色杯子。此任务是OOD, 因为场景中的所有物体在训练数据中都是未见过的。

10. **Place Apple on Plate** (OOD): The robot is positioned in front of a platform with a plate and an apple. The robot's goal is to grasp the apple and move it onto the plate. This task is OOD because it is a novel instruction describing an unseen object relation: training demonstrations only cover moving the apple *near* the plate, rather than placing it *on top of* the plate.

10. 将苹果放在盘子上 (OOD): 机器人位于一个带有盘子和苹果的平台前。机器人的目标是抓住苹果并将其移动到盘子上。此任务是OOD, 因为它是一个新颖的指令, 描述了一种未见过的物体关系: 训练演示仅涵盖将苹果移动到盘子附近, 而不是将其放在盘子顶部。

11. **Place Banana in Pan** (OOD): The robot is positioned in front of a platform with a pan and a banana. The robot's goal is to grasp the banana and move it into the pan. This task is OOD because the banana is an unseen target object, and it is a novel instruction describing an unseen object relation, as explained in the previous task.

11. 将香蕉放入锅中（OOD）：机器人位于一个平台上，平台上有锅和香蕉。机器人的目标是抓住香蕉并将其移动到锅中。此任务是OOD，因为香蕉是未见过的目标对象，并且它是一个描述未见对象关系的新颖指令，如前一个任务中所述。

12. **Move Coke Can to Taylor Swift** (OOD): The robot is positioned in front of a platform with a can of Coke and photos of three different celebrities, including Taylor Swift. The robot’s goal is to grasp the can and move it to the photo of Taylor Swift. This task is OOD because the photos of the celebrities are unseen in the robot interaction data.

12. 将可乐罐移动到泰勒·斯威夫特处（OOD）：机器人定位在一个平台上，平台上有一个可乐罐和三张不同名人的照片，其中包括泰勒·斯威夫特。机器人的目标是抓住罐子并将其移动到泰勒·斯威夫特的照片处。这项任务是OOD，因为名人的照片在机器人交互数据中是未见过的。

B.2.2 Detailed Google Robot Evaluation Results

B. 2. 2 谷歌机器人评估详细结果

Table 6: **Detailed Google robot evaluation results.** We report full evaluation results for Google robot evaluations discussed in Section 5.1. Each generalist policy is evaluated with 60 rollouts across 12 tasks, covering both in-distribution and out-of-distribution (OOD) testing conditions. In the bottom row, we report mean success rate $\pm$ StdErr for each policy. OpenVLA and RT-2-X both significantly outperform RT-1-X and Octo overall (we bold the mean success rate for both due to overlapping error bars). See Fig. 9 for illustrations of all tasks.

表6：详细的谷歌机器人评估结果。我们报告了第5.1节中讨论的谷歌机器人评估的完整评估结果。每个通用策略都使用60次rollout在12个任务上进行评估，涵盖了分布内和分布外（OOD）的测试条件。在底行，我们报告了每个策略的平均成功率 $\pm$ 标准误差。OpenVLA和RT-2-X都明显优于RT-1-X和Octo（由于误差条重叠，我们用粗体标出了两者的平均成功率）。所有任务的图示见图9。

Category	Task	# Trials	RT-1-X # Successes	Octo # Successes	RT-2-X # Successes	OpenVLA (ours) # Successes
In-distribution	Pick Coke Can	5	5	1	5	5
In-distribution	Move Apple near Green Can	5	3	3	3	5
In-distribution	Move Blue Chip Bag near Apple	5	0	3	4	5
In-distribution	Place Coke Can Upright	5	0	0	4	4
In-distribution	Open Middle Drawer	5	0	4	2	3
OOD	Move Orange near Brown Chip Bag	5	1	2	5	5
OOD	Pick Pepsi Can	5	3	0	5	4
OOD	Pick Banana	5	5	3	5	5
OOD	Pick Green Cup	5	1	0	5	5
OOD	Place Apple on Plate	5	0	0	4	4
OOD	Place Banana in Pan	5	0	0	2	4
OOD	Move Coke Can near Taylor Swift	5	2	0	3	2
Mean Success Rate			33.3 $\pm$ 6.1%	26.7 $\pm$ 5.8%	78.3$\pm$5.4%	85.0$\pm$4.6%

Full results for the Google robot evaluations are shown in Table 6. Overall, we find that RT-1-X and Octo experience difficulty on the evaluation tasks; they are often unable to achieve a single success out of five trials in several tasks. On the other hand, RT-2-X and OpenVLA demonstrate strong performance, completing every task at least two times out of five trials; these two VLA policies perform comparably with each other on this particular evaluation suite.

Google 机器人评估的完整结果如表 6 所示。总的来说，我们发现 RT-1-X 和 Octo 在评估任务中遇到了困难；在几个任务中，它们通常无法在五次试验中成

功一次。另一方面，RT-2-X 和 OpenVLA 表现出强大的性能，在五次试验中至少完成了两次任务；在这项特定的评估套件中，这两个 VLA 策略的表现相当。

B.3 Data-Efficient Adaptation Experiment Details

B.3 数据高效适应实验细节

In this section, we provide more details on the data-efficient adaptation experiments discussed in Section 5.2, where we investigate the effectiveness of fine-tuned OpenVLA policies on new robot setups such as Franka-Tabletop and Franka-DROID.

在本节中，我们将详细介绍第 5.2 节中讨论的数据高效适应实验，其中我们研究了微调后的 OpenVLA 策略在新机器人设置（如 Franka-Tabletop 和 Franka-DROID）上的有效性。

B.3.1 Franka-Tabletop and Franka-DROID Tasks

B.3.1 Franka-Tabletop 和 Franka-DROID 任务

We collect 10–150 demonstrations of each of seven tasks. The first six tasks correspond to a robot setup which we denote as “Franka-Tabletop” (Franka Emika Panda robot mounted on top of a table), and the final task corresponds to a robot setup which we call “Franka-DROID”.

我们收集了七个任务中每个任务的10–150个演示。前六个任务对应于一个机器人设置，我们将其表示为“Franka-Tabletop”（安装在桌子上的Franka Emika Panda机器人），最后一个任务对应于一个机器人设置，我们称之为“Franka-DROID”。

In the Franka-Tabletop setup, the first three of six tasks correspond to single-instruction tasks and are narrow, while the last three tasks correspond to multi-instruction tasks in which multiple objects are present in the scene and the robot must manipulate the correct one depending on the language instruction.

在Franka-Tabletop设置中，六项任务中的前三项对应于单指令任务，范围较窄；而后三项任务对应于多指令任务，场景中存在多个对象，机器人必须根据语言指令操纵正确的对象。

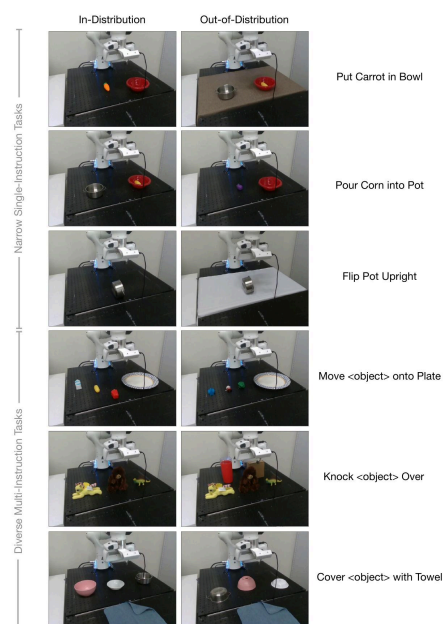


Figure 10: **Franka-Tabletop fine-tuning tasks.** Franka-Tabletop tasks used in the data-efficient adaptation experiments in Section 5.2 and described in detail in Fig. 10 are depicted above. The first three of six tasks, shown in the top three rows, only involve a single instruction, while the last three tasks in the bottom three rows involve multiple objects and instructions (the instructions specify the target object or target location). The first column shows sample initial states matching the training data distribution, while the second column shows out-of-distribution (OOD) initial states (e.g., unseen backgrounds, target objects, distractors, and object positions/orientations). Every policy in Section 5.2 is evaluated with 10–12 rollouts on in-distribution tasks and 5–6 rollouts on OOD tasks.

图10: Franka-Tabletop微调任务。图10详细描述了第5.2节中用于数据高效适应实验的Franka-Tabletop任务，如上图所示。前六个任务中的前三个任务（显示在顶部三行）仅涉及单个指令，而底部三行中的后三个任务涉及多个对象和指令（指令指定目标对象或目标位置）。第一列显示与训练数据分布匹配的样本初始状态，而第二列显示了分布外（OOD）的初始状态（例如，未见过的背景、目标对象、干扰物和对对象位置/方向）。第5.2节中的每个策略都在分布内任务上评估10–12次，在OOD任务上评估5–6次。

Below we describe each of the six Franka-Tabletop tasks shown in Fig. 10:

下面我们描述了图 10 中所示的六个 Franka-Tabletop 任务中的每一个：

1. **Put Carrot in Bowl** (single-instruction): The robot’s goal is to grasp the carrot and place it into the bowl. We collect 50 demonstrations of this task for the training dataset, randomly placing the carrot and the bowl at different locations on the table in every episode. The carrot is always initialized on the left side of the bowl. During evaluation, each trial is recorded as a success (1) or failure (0); there is no partial credit.

1. 将胡萝卜放入碗中（单指令）：机器人的目标是抓住胡萝卜并将其放入碗中。我们收集了50个此任务的演示作为训练数据集，在每个episode中随机将胡萝卜和碗放置在桌子上不同的位置。胡萝卜始终在碗的左侧初始化。在评估期间，每个试验都被记录为成功（1）或失败（0）；没有部分信用。

2. Pour Corn into Pot (single-instruction): The robot's goal is to grasp the red bowl, move towards the steel pot, and pour the contents (a yellow corn) into the pot. We collect 50 demonstrations of this task for the training dataset, randomly placing the bowl and the pot at different locations on the table in every episode. The bowl is always initialized on the right side of the pot. During evaluation, each trial is recorded as a success (1) or failure (0); there is no partial credit.

2. 将玉米倒入锅中（单指令）：机器人的目标是抓住红色碗，移向钢锅，并将里面的东西（黄色玉米）倒入锅中。我们收集了50个此任务的演示作为训练数据集，在每一集中随机地将碗和锅放置在桌子上不同的位置。碗总是初始化在锅的右侧。在评估过程中，每次试验都被记录为成功（1）或失败（0）；没有部分信用。

3. Flip Pot Upright (single-instruction): The robot's goal is to grasp the steel pot (which is initially oriented vertically), rotate it to be in the upright position, and place it back onto the table. We collect only 10 demonstrations of this task for the training dataset, randomly placing the steel pot at various locations within a small section of the table. During evaluation, each trial is recorded as a success (1), failure (0), or partial success (0.5). Partial successes include grasping the pot but not orienting it upright, or knocking it over to the upright position but not carefully guiding it. The robot must release the pot at the end of the episode for full credit.

3. 将锅翻正（单指令）：机器人的目标是抓住钢锅（最初垂直放置），将其旋转至直立位置，然后放回桌子上。我们仅收集了此任务的 10 个演示作为训练数据集，随机将钢锅放置在桌子一小部分的各个位置。在评估期间，每次试验都被记录为成功（1）、失败（0）或部分成功（0.5）。部分成功包括抓住锅但未将其扶正，或将其撞倒至直立位置但未小心引导。机器人必须在结束时松开锅才能获得全部积分。

4. Move <object> onto Plate (multi-instruction): The robot's goal is to grasp one out of three objects (depending on the target specified in the language instruction) and place it on the plate on the right side of the table. We collect 150 demonstrations of this task for the training dataset, randomly placing different combinations of three objects on the table and selecting one as the target. The plate is always initialized on the right side of the table. During evaluation, each trial is recorded as a success (1), failure (0), or partial success (0.5). Partial success is recorded when the first object that the robot makes contact with is the correct target object (i.e., the object specified in the language instruction), but the robot does not complete the task.

4. 将<物体>移动到盘子上（多指令）：机器人的目标是抓住三个物体中的一个（取决于语言指令中指定的目标），并将其放置在桌子右侧的盘子上。我

们收集了150个此任务的演示，用于训练数据集，随机将三种物体的不同组合放置在桌子上，并选择其中一个作为目标。盘子始终在桌子的右侧初始化。在评估期间，每次试验都被记录为成功（1）、失败（0）或部分成功（0.5）。当机器人接触的第一个物体是正确的目标物体（即，语言指令中指定的物体）时，但机器人未完成任务，则记录为部分成功。

5. **Knock <object> Over** (multi-instruction): The robot's goal is to approach one out of three objects (depending on the target specified in the language instruction) and push it until it falls over. We collect 70 demonstrations of this task for the training dataset, randomly placing different combinations of three objects on the table and selecting one as the target. During evaluation, each trial is recorded as a success (1), failure (0), or partial success (0.5). Partial success is recorded when the first object that the robot makes contact with is the correct target object (i.e., the object specified in the language instruction), but the robot does not complete the task.

5. 推倒 <物体>（多指令）：机器人的目标是接近三个物体中的一个（取决于语言指令中指定的目标），并推动它直到它倒下。我们收集了70个此任务的演示作为训练数据集，随机地将三种不同的物体组合放置在桌子上，并选择其中一个作为目标。在评估期间，每次试验都被记录为成功（1）、失败（0）或部分成功（0.5）。当机器人接触的第一个物体是正确的目标物体（即，语言指令中指定的物体）时，但机器人没有完成任务，则记录为部分成功。

6. **Cover <object> with Towel** (multi-instruction): The robot's goal is to grasp the blue towel and place it on one out of three objects (depending on the target specified in the language instruction). We collect 45 demonstrations of this task for the training dataset, randomly placing different combinations of three objects on the table. During evaluation, each trial is recorded as a success (1), failure (0), or partial success (0.5). Partial success is recorded when the first object that the robot touches with the towel is the correct target object (i.e., the object specified in the language instruction), but the robot does not complete the task (e.g., it drops the towel onto the table instead of on top of the target object). Full credit is given when any part of the towel is resting over the top surface of the target object, i.e., the object does not need to be fully covered.

6. 用毛巾覆盖<物体>（多指令）：机器人的目标是抓住蓝色毛巾，并将其放置在三个物体中的一个上（具体取决于语言指令中指定的目标）。我们收集了45个此任务的演示，作为训练数据集，随机地将三种物体的不同组合放置在桌子上。在评估期间，每次试验都被记录为成功（1）、失败（0）或部分成功（0.5）。当机器人用毛巾接触的第一个物体是正确的目标物体（即语言指令中指定的物体）时，但机器人未完成任务（例如，它将毛巾掉在桌子上而不是放在目标物体顶部），则记录为部分成功。当毛巾的任何部分位于目标物体的顶面上时，则给予全部信用，即不需要完全覆盖该物体。

For every Franka-Tabletop task, we evaluate each method with 10–12 in-distribution trials and 5–6 OOD generalization trials. The in-distribution and OOD test conditions are depicted in Fig. 10 (second column).

对于每个Franka-Tabletop任务，我们使用10 – 12个同分布试验和5 – 6个OOD泛化试验来评估每种方法。同分布和OOD测试条件如图10（第二列）所示。

We describe the OOD test conditions for each of the six tasks below: 1. Put Carrot in Bowl (OOD): An eggplant (unseen object) replaces the carrot.

我们描述了以下六个任务中每一个任务的OOD测试条件：1. 将胡萝卜放入碗中（OOD）：用茄子（未见过的物体）代替胡萝卜。

2. Pour Corn into Pot (OOD): An unseen brown tablecloth covers the tabletop.

2. 将玉米倒入锅中（OOD）：一张看不见的棕色桌布覆盖着桌面。

3. Flip Pot Upright (OOD): An unseen white tablecloth covers the tabletop

3. 将罐子翻正（OOD）：一张看不见的白色桌布覆盖了桌面

4. Move <object> onto Plate (OOD): A set of three unseen objects are placed on the table.

4. 将<物体>移动到盘子上（OOD）：将一组三个未见过的物体放置在桌子上。

5. Knock <object> Over (OOD): Two unseen distractor objects (red plastic cup and brown box) are positioned behind the set of three seen objects.

5. 击倒<物体>（OOD）：两个未见过的干扰物（红色塑料杯和棕色盒子）被放置在三个可见物体组的后面。

6. Cover <object> with Towel (OOD): The three objects on the table are placed upside-down and at unseen positions.

6. 用毛巾盖住<物体>（OOD）：桌子上的三个物体被倒置并放置在看不见的位置。

Finally, in the Franka-DROID environment, we experiment with one task and variants of it: **Wipe Table** (see Fig. 11). In this task, the robot’s goal is to grab the brush and sweep all three small brown objects into the dustpan. We collect 70 demonstrations for this task for the training dataset, varying the positions of all the objects.

最后，在Franka-DROID环境中，我们用一项任务及其变体进行实验：擦拭桌子（见图11）。在这项任务中，机器人的目标是抓住刷子，并将所有三个棕色小物

体扫入簸箕中。我们为此任务收集了70个演示，作为训练数据集，并改变了所有物体的位置。

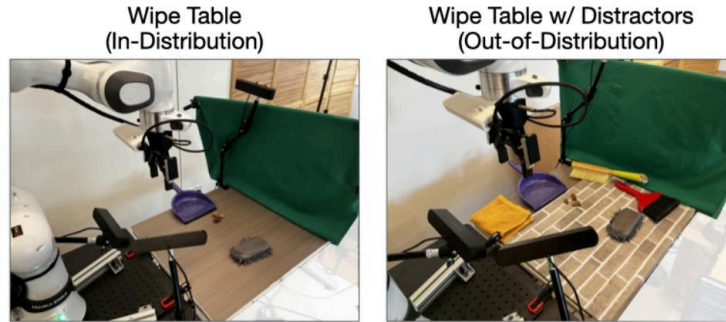


Figure 11: **Franka-DROID fine-tuning task.** The “Wipe Table” task shown here is the final task used in the data-efficient adaptation experiments in Section 5.2. The left image shows the initial conditions for an in-distribution trial. The right image shows an out-of-distribution trial in which unseen distractor objects are present on the table. To fully complete the task, the robot must grab the brush and sweep all three objects into the dustpan.

图11: Franka-DROID微调任务。此处显示的“擦拭桌子”任务是第5.2节中数据高效适应实验中使用的最终任务。左图显示了分布内试验的初始条件。右图显示了分布外试验，其中桌子上存在未见过的干扰物。要完全完成任务，机器人必须抓住刷子并将所有三个物体扫入簸箕中。

At test time, we evaluate on in-distribution conditions matching the training data (Fig. 11, left), as well as out-of-distribution (OOD) conditions in which distractor objects are also present in the scene on the table (Fig. 11, right). Since there are various possible outcomes for each trial, we define a scoring rubric as follows: The maximum score for each trial is 2 points. The policy receives the full 2 points if the robot sweeps all three objects into the dustpan. It receives 1 point for successfully sweeping one or two objects into the dustpan. Otherwise, it receives 0 points. We evaluate each policy with 18 in-distribution trials and 12 OOD trials, so each policy receives an aggregate score out of 60 points.

在测试时，我们评估与训练数据相匹配的同分布条件（图 11，左），以及异分布（OOD）条件，其中干扰对象也存在于桌面的场景中（图 11，右）。由于每次试验都有各种可能的结果，我们定义评分标准如下：每次试验的最高分数为 2 分。如果机器人将所有三个物体扫入簸箕，则该策略获得完整的 2 分。如果成功地将一个或两个物体扫入簸箕，则获得 1 分。否则，获得 0 分。我们使用 18 次同分布试验和 12 次异分布试验评估每个策略，因此每个策略获得的总分数为 60 分。

B.3.2 Detailed Franka-Tabletop and Franka-DROID Evaluation Results

B. 3. 2 Franka-Tabletop和Franka-DROID详细评估结果

Full evaluation results for both Franka-Tabletop and Franka-DROID evaluations are shown in Table 7. We evaluate the methods discussed in Section 5.2. We find that Diffusion Policy demonstrates strong performance on the single-instruction

Franka-Tabletop tasks (e.g., “Put Carrot in Bowl” and “Pour Corn in Pot”), outperforming other methods. However, OpenVLA and Octo achieve higher performance in the more diverse multi-instruction tasks (“Move <object> onto Plate”, “Knock <object> Over”, and “Cover <object> with Towel”). In the Franka-DROID environment, OpenVLA obtains best results. Overall, we find that OpenVLA achieves the highest average performance across both tasks.

Franka-Tabletop和Franka-DROID评估的完整评估结果如表7所示。我们评估了第5.2节中讨论的方法。我们发现，Diffusion Policy在单指令Franka-Tabletop任务（例如，“把胡萝卜放入碗中”和“把玉米倒入锅中”）上表现出强大的性能，优于其他方法。然而，OpenVLA和Octo在更多样化的多指令任务（“将<物体>移动到盘子上”，“击倒<物体>”和“用毛巾盖住<物体>”）中实现了更高的性能。在Franka-DROID环境中，OpenVLA获得了最佳结果。总的来说，我们发现OpenVLA在两项任务中都取得了最高的平均性能。

Additionally, in Table 8, we show the detailed version of the parameter-efficient fine-tuning experiment results summarized in Table 1. In these experiments, we use a representative subset of two FrankaTabletop tasks, with both in-distribution and OOD variants: one narrow single-instruction task (“Put Carrot in Bowl”) and one diverse multi-instruction task (“Move <object> onto Plate”). We use the same number of training demonstrations used in Section 5.2 (50 and 150, respectively), which is delineated in Appendix B.3.1.

此外，在表8中，我们展示了参数高效微调实验结果的详细版本，该结果总结于表1中。在这些实验中，我们使用了两个具有代表性的FrankaTabletop任务子集，包括分布内（in-distribution）和分布外（OOD）变体：一个狭窄的单指令任务（“Put Carrot in Bowl”）和一个多样化的多指令任务（“Move <object> onto Plate”）。我们使用了与第5.2节中相同数量的训练演示（分别为50和150），这在附录B.3.1中进行了详细说明。

Table 7: Detailed data-efficient adaptation experiment results. Here we present the full breakdown of results summarized in Fig. 5. We report the performance of Diffusion Policy trained from scratch on new robot tasks, as well as generalist policies fine-tuned on the same data. Each policy is tested against both in-distribution and out-of-distribution (OOD) generalization conditions (see Fig. 10 for Franka-Tabletop tasks and Fig. 11 for Franka-DROID tasks). We find that no single policy performs best on all tasks: Diffusion Policy achieves high success rates on single-instruction tasks, while OpenVLA and Octo performs well on diverse multi-instruction tasks. In terms of aggregate performance, however, OpenVLA obtains the highest average success rate across both environments.

表 7：详细的数据高效适应实验结果。此处我们展示了图 5 中总结的完整结果细分。我们报告了从头开始在新机器人任务上训练的扩散策略的性能，以及在相同数据上微调的通用策略的性能。每个策略都针对分布内和分布外（OOD）泛化条件进行测试（Franka-Tabletop 任务见图 10，Franka-DROID 任务见图 11）。我们发现没有单一策略在所有任务上表现最佳：扩散策略在单指令任务上实现了高成功率，而 OpenVLA 和 Octo 在多样化的多指令任务上表现良好。然而，就总体性能而言，OpenVLA 在两个环境中都获得了最高的平均成功率。

		# trials	Diffusion Policy	Diffusion Policy (matched)	Octo	OpenVLA (scratch)	OpenVLA (ours)
Franka-Tabletop (5Hz)	"Put Carrot in Bowl" (in-distribution)	10	90.0%	80.0%	40.0%	70.0%	70.0%
	"Put Carrot in Bowl" (OOD)	5	20.0%	0.0%	20.0%	0.0%	40.0%
	"Pour Corn into Pot" (in-distribution)	10	100.0%	90.0%	0.0%	10.0%	50.0%
	"Pour Corn into Pot" (OOD)	5	80.0%	60.0%	0.0%	20.0%	60.0%
	"Flip Pot Upright" (in-distribution)	10	100.0%	85.0%	40.0%	85.0%	100.0%
	"Flip Pot Upright" (OOD)	5	50.0%	20.0%	0.0%	40.0%	80.0%
	"Move <object> onto Plate" (in-distribution)	12	25.0%	25.0%	41.7%	8.3%	75.0%
	"Move <object> onto Plate" (OOD)	6	8.3%	33.3%	8.3%	33.3%	58.3%
	"Knock <object> Over" (in-distribution)	12	33.3%	25.0%	83.3%	75.0%	75.0%
	"Knock <object> Over" (OOD)	6	16.7%	16.7%	33.3%	58.3%	83.3%
	"Cover <object> with Towel" (in-distribution)	12	16.7%	20.8%	91.7%	41.7%	50.0%
	"Cover <object> with Towel" (OOD)	6	16.7%	33.3%	91.7%	50.0%	50.0%
Average			48.5±4.9%	43.4±4.7%	43.4±4.4%	43.4±4.6%	67.2±4.0%
Franka-DROID (15Hz)	"Wipe Table" (in-distribution)	18	50.0%	27.8%	52.8%	25.0%	55.6%
	"Wipe Table" + Distractors (OOD)	12	12.5%	25.0%	16.7%	16.7%	62.5%
	Average		35.0±8.0%	26.7±7.5%	38.3±8.5%	21.7±6.6%	58.3±7.2%

Table 8: **Detailed parameter-efficient fine-tuning experiment results.** Here we present the detailed task performance results summarized in Table 1.

表 8：参数高效微调实验的详细结果。这里我们展示了表 1 中总结的详细任务性能结果。

		# trials	Full FT	Last layer only	Frozen vision	Sandwich	LoRA, r=32	LoRA, r=64
Franka-Tabletop (5Hz)	"Put Carrot in Bowl" (in-distribution)	10	90.0	40.0	40.0	90.0	60.0	90.0
	"Put Carrot in Bowl" (OOD)	5	40.0	0.0	40.0	0.0	60.0	40.0
	"Move <object> onto Plate" (in-distribution)	12	79.2	33.3	50.0	75.0	75.0	62.5
	"Move <object> onto Plate" (OOD)	6	41.7	33.3	58.3	41.7	75.0	66.7
	Average		69.7±7.2%	30.3±6.1%	47.0±6.9%	62.1±7.9%	68.2±7.5%	68.2±7.8%

C RT-2-X vs. OpenVLA in BridgeData V2 Evaluations

C RT-2-X 与 OpenVLA 在 BridgeData V2 评估中的比较

In this section, we provide additional details on RT-2-X vs. OpenVLA comparisons in BridgeData V2 evaluations discussed in Section 5.1. As discussed previously, OpenVLA is pretrained on a larger subset of OpenX data than RT-2-X and uses a fused SigLIP-DinoV2 vision backbone rather than a single visual encoder. However, in addition to these factors, we believe that OpenVLA’s significant improvement upon RT-2-X specifically in BridgeData V2 evaluations (as shown in Fig. 3) also stems from more careful preprocessing of the Bridge dataset.

在本节中，我们将提供关于RT-2-X对比的更多细节。如第5.1节所讨论的 BridgeData V2评估中的OpenVLA比较。如前所述，OpenVLA在比RT-2-X更大的 OpenX数据子集上进行了预训练，并使用融合的SigLIP-DinoV2视觉骨干网络，而不是单个视觉编码器。然而，除了这些因素之外，我们认为OpenVLA在 BridgeData V2评估中对RT-2-X的显著改进（如图3所示）也源于对Bridge数据集更仔细的预处理。

During the development of the OpenVLA model, we discovered that the original version of the BridgeData V2 dataset contained many transitions with all-zero (no-op) actions. For instance, in every demonstration, an all-zero action was recorded as the ground-truth action in the first timestep. Consequently, training a highly expressive VLA model on the original dataset without any data preprocessing led to a policy that frequently predicted all-zero actions and froze during evaluations. Therefore, we simply filtered out the first transition in every demonstration when training the OpenVLA model, and this was sufficient for mitigating the freezing behavior in most cases.

在OpenVLA模型的开发过程中，我们发现原始版本的BridgeData V2数据集中包含许多具有全零（无操作）动作的转换。例如，在每个演示中，全零动作都被记录为第一个时间步中的真实动作。因此，在没有任何数据预处理的情况下，在原始数据集上训练一个高表达性的VLA模型会导致策略频繁预测全零动作，并在评估期间冻结。因此，我们在训练OpenVLA模型时，简单地过滤掉了每个演示中的第一个转换，这足以在大多数情况下缓解冻结行为。

However, the RT-2-X model was trained without such data preprocessing, so it often suffers the aforementioned freezing behavior if deployed out of the box without modifying the model querying procedure – which severely deteriorates rollout performance. Since this is a proprietary model that is infeasible for us to re-train (e.g., with our preprocessed version of the BridgeData V2 dataset), we mitigated this issue by simply querying the *second-most-likely* action from the model, since the first-most-likely action was often all zeros while the second-most-likely action was not. (Note that this is the same workaround that was applied by the developers of the RT-2-X model for BridgeData V2 evaluations reported in the Open X-Embodiment experiments [1].) This workaround led to much stronger RT-2-X performance on BridgeData V2 evaluations – though we believe that it is still suboptimal compared to re-training the model on the preprocessed version of the dataset.

然而，RT-2-X模型在训练时没有进行此类数据预处理，因此，如果在不修改模型查询程序的情况下直接部署，它经常会遇到上述的冻结行为，这会严重降低rollout性能。由于这是一个专有模型，我们无法对其进行重新训练（例如，使用我们预处理过的BridgeData V2数据集），因此，我们通过简单地查询模型中可能性第二大的动作来缓解这个问题，因为可能性最大的动作通常全部为零，而可能性第二大的动作则不是。（请注意，这与RT-2-X模型的开发者在Open X-Embodiment实验中报告的BridgeData V2评估中使用的解决方法相同[1]。）这种解决方法使得RT-2-X在BridgeData V2评估中表现出更强的性能——尽管我们认为与在数据集的预处理版本上重新训练模型相比，这仍然是次优的。

We also tried to dynamically query RT-2-X, i.e., by first sampling the first-most-likely action and then sampling the second-most-likely action if the first one was all zeros. However, we empirically found that dynamic querying led to worse performance than simply querying the second-most-likely action at all times. We hypothesize that this is due to a change in the robot’s dynamics that arises from dynamic querying: pausing in the middle of a trajectory to re-query the model leads to slight interruptions in the robot’s movement due to non-negligible latency in the querying pipeline, and this leads to subtle performance degradation. Therefore, we report the performance of RT-2-X when always querying the second-most-likely action, as done in the Open X-Embodiment project [1].

我们还尝试动态查询RT-2-X，即首先采样最有可能的动作，如果第一个动作全为零，则采样第二有可能的动作。然而，我们通过实验发现，动态查询导致比始终查询第二有可能的动作更差的性能。我们假设这是由于动态查询引起的机器人动力学变化：在轨迹中间暂停以重新查询模型会导致机器人运动的轻微中断，这是由于查询管道中存在不可忽略的延迟，从而导致细微的性能下降。因此，我们报告了RT-2-X在始终查询第二有可能的动作时的性能，如Open X-Embodiment项目[1]中所做的那样。

D Additional Experiments and Ablations

D 更多实验与消融研究

In this section, we conduct several additional experiments to analyze the effects of individual components of the OpenVLA model architecture and training scheme, as well as provide quantitative evidence for claims made in earlier sections of this work. We aim to answer the following questions:

在本节中，我们将进行几个额外的实验，以分析OpenVLA模型架构和训练方案中各个组成部分的影响，并为本文前面部分提出的主张提供定量证据。我们的目标是回答以下问题：

1. How important is OpenX training and how does it impact OpenVLA's performance (Appendix D.1)?
1. OpenX训练有多重要，它如何影响OpenVLA的性能（附录D.1）？
2. What effect does using a fused SigLIP-DinoV2 vision encoder have on OpenVLA's performance, compared to using a SigLIP-only vision encoder (Appendix D.2)?
2. 相比于仅使用SigLIP视觉编码器，使用融合的SigLIP-DinoV2视觉编码器对OpenVLA的性能有何影响（附录D.2）？
3. Is it better to fine-tune or freeze the vision encoder in OpenVLA (Appendix D.3)?
3. 在OpenVLA中，微调视觉编码器还是冻结视觉编码器更好（附录D.3）？
4. How do the quantized inference results discussed in Section 5.3 change when policy performance is disentangled from model inference speed (Appendix D.4)?
4. 当策略性能与模型推理速度分离时（附录D.4），第5.3节中讨论的量化推理结果会发生怎样的变化？

We discuss the experimental setup and results addressing each of the above questions sequentially in the following sections.

在接下来的章节中，我们将按顺序讨论实验装置和结果，以解决上述每个问题。

D.1 OpenX Training Data Ablation Experiments

D.1 OpenX训练数据消融实验

As discussed in Section 3.3, OpenVLA is trained on a large dataset of robot embodiments, scenes, and tasks from the Open X-Embodiment dataset [1] (OpenX). In this section, we ablate the OpenX mixture and train a VLA policy solely on one robot dataset, to assess the impact of OpenX training on policy performance. Note that we have already observed the negative effect of ablating OpenX training in the fine-tuning regime, as discussed in Section 5.2 (see OpenVLA (Scratch)), but we discuss additional experiments on another robot embodiment in this section to provide more supporting evidence.

正如第3.3节所讨论的，OpenVLA是在来自Open X-Embodiment数据集[1] (OpenX)的大量机器人实体、场景和任务的数据集上训练的。在本节中，我们消融OpenX混合，并仅在一个机器人数据集上训练VLA策略，以评估OpenX训练对策略性能的影响。请注意，正如第5.2节所讨论的（参见OpenVLA (Scratch)），我们已经观察到在微调机制中消融OpenX训练的负面影响，但我们将在本节中讨论关于另一个机器人实体的额外实验，以提供更多的支持证据。

Experimental setup and tasks. We compare the original OpenVLA model with **OpenVLA-Bridge**, which is produced by taking the same pretrained VLM as OpenVLA (Prismatic VLM [44]) and fine-tuning it solely on BridgeData V2 [6] rather than the entire OpenX training mixture discussed in Appendix A. We evaluate OpenVLA and OpenVLA-Bridge on a subset of 8 representative tasks from the BridgeData V2 WidowX robot evaluation suite discussed in Appendix B.1.1. The tasks are listed in Table 9.

实验装置与任务。我们将原始OpenVLA模型与OpenVLA-Bridge进行比较，后者是通过采用与OpenVLA相同的预训练VLM (Prismatic VLM [44])并在BridgeData V2 [6]上进行微调而生成的，而不是在附录A中讨论的整个OpenX训练混合物上进行微调。我们在BridgeData V2 WidowX机器人评估套件（在附录B.1.1中讨论）的8个代表性任务的子集上评估OpenVLA和OpenVLA-Bridge。这些任务列于表9中。

Results. Results for the OpenX training mixture ablation are shown in Table 9. By comparing OpenVLA with OpenVLA-Bridge, we see that performance drops drastically (reduction of 30 percent in absolute success rate), which demonstrates the importance of OpenX pretraining on final policy performance. Although the

language grounding performance is not impacted, we observe performance reduction across all generalization categories. This result suggests that the large diversity of scenes, objects, and tasks in the OpenX training mixture is essential for unlocking improved generalization capabilities in the OpenVLA model.

结果。OpenX训练混合消融实验的结果如表9所示。通过比较OpenVLA和OpenVLA-Bridge，我们发现性能急剧下降（绝对成功率降低30个百分点），这表明OpenX预训练对最终策略性能的重要性。尽管语言基础性能没有受到影响，但我们观察到所有泛化类别上的性能都有所下降。这一结果表明，OpenX训练混合中场景、物体和任务的大量多样性对于解锁OpenVLA模型中改进的泛化能力至关重要。

D.2 Dual vs. Single Vision Encoder Experiments

D.2 双视觉编码器与单视觉编码器实验

The OpenVLA model architecture consists of a fused vision backbone that combines the SigLIP [9] and DinoV2 [25] encoders. In this section, we ablate the DinoV2 component to assess the importance of using a dual vision encoder.

OpenVLA模型架构包含一个融合的视觉骨干网络，该网络结合了SigLIP [9]和DinoV2 [25]编码器。在本节中，我们消融了DinoV2组件，以评估使用双视觉编码器的重要性。

Experimental setup and tasks. We instantiate a model, **OpenVLA-Bridge-SigLIP**, which is a version of OpenVLA that is trained only on BridgeData V2 and consists of only the SigLIP encoder as the vision backbone. We compare this model with the OpenVLA-Bridge model discussed in the previous section (Appendix D.1), which shares the same model architecture as the original OpenVLA model and is only trained on Bridge robot data. Therefore, the only difference between OpenVLA-Bridge-SigLIP and OpenVLA-Bridge is that the former omits the DinoV2 encoder in the vision backbone. We evaluate these models on the same subset of 8 Bridge tasks described in the previous section.

实验装置与任务。我们实例化一个模型，OpenVLA-Bridge-SigLIP，它是OpenVLA的一个版本，仅在BridgeData V2上训练，并且仅包含SigLIP编码器作为视觉骨干。我们将此模型与上一节（附录D.1）中讨论的OpenVLA-Bridge模型进行比较，该模型与原始OpenVLA模型共享相同的模型架构，并且仅在Bridge机器人数据上进行训练。因此，OpenVLA-Bridge-SigLIP和OpenVLA-Bridge之间唯一的区别是前者省略了视觉骨干中的DinoV2编码器。我们在上一节中描述的8个Bridge任务的相同子集上评估这些模型。

Table 9: **BridgeData V2 WidowX ablation experiment results.** We evaluate various methods on a subset of 8 representative tasks to assess the importance of different components of the OpenVLA model architecture and training scheme. OpenVLA-Bridge is a version of OpenVLA without OpenX training (it is trained only on BridgeData V2), and OpenVLA-Bridge-SigLIP additionally ablates the

fused vision backbone by removing the DinoV2 encoder (its vision backbone only consists of the SigLIP encoder). We observe that both OpenX training and the fused vision encoder improve policy performance, though the former has a much greater effect than the latter.

表 9: BridgeData V2 WidowX 消融实验结果。我们评估了 8 个具有代表性的任务子集上的各种方法，以评估 OpenVLA 模型架构和训练方案中不同组件的重要性。OpenVLA-Bridge 是一个没有 OpenX 训练的 OpenVLA 版本（仅在 BridgeData V2 上训练），OpenVLA-Bridge-SigLIP 通过移除 DinoV2 编码器来进一步消融融合视觉主干网络（其视觉主干网络仅由 SigLIP 编码器组成）。我们观察到 OpenX 训练和融合视觉编码器都提高了策略性能，尽管前者比后者具有更大的影响。

Category	Task	# Trials	OpenVLA # Successes	OpenVLA-Bridge # Successes	OpenVLA-Bridge-SigLIP # Successes
Visual gen	Put Eggplant into Pot (Easy Version)	10	10	8	8
Visual gen	Put Eggplant into Pot	10	10	2	3
Visual gen	Put Cup from Counter into Sink	10	7	4	2
Motion gen	Lift Eggplant	10	7.5	5.5	6.5
Physical gen	Put Carrot on Plate	10	8	4	1
Physical gen	Lift AAA Battery	10	7	2	2
Semantic gen	Take Purple Grapes out of Pot	10	4	3	3
Language grounding	Put (Eggplant, Red Bottle) into Pot	10	7.5	8	7
Mean Success Rate			76.3 ± 4.8%	45.6 ± 5.6%	40.6 ± 5.5%

Results. Results for the dual vision encoder ablation are shown in Table 9. The drop in performance from OpenVLA-Bridge to OpenVLA-Bridge-SigLIP implies that additionally including the DinoV2 encoder in the vision backbone improves policy performance. However, the 5 percent reduction in performance here is not as significant as the 30 percent drop in performance observed from ablating OpenX training. The low-level spatial features represented in DinoV2 appear to aid generalization in only some cases.

结果。双重视觉编码器消融实验的结果如表9所示。从OpenVLA-Bridge到OpenVLA-Bridge-SigLIP的性能下降意味着在视觉主干中额外包含DinoV2编码器可以提高策略性能。然而，此处5%的性能下降不如消融OpenX训练所观察到的30%的性能下降那么显著。DinoV2中表示的低级空间特征似乎仅在某些情况下有助于泛化。

D.3 Fine-Tuned vs. Frozen Vision Encoder Experiments

D. 3 微调 vs. 冻结视觉编码器实验

As discussed in Section 3.4, prior work on VLMs observed higher performance from freezing the vision encoder than fine-tuning its parameters [44]. However, when training OpenVLA, we fine-tuned all 7B parameters in the model, including the SigLIP-DinoV2 vision backbone, as we discovered early on during development that fine-tuning the vision encoder led to higher-performing VLAs — a finding which held across various pretrained VLMs and model architectures. We discuss details of such findings below.

正如第3.4节所讨论的，先前关于VLMs的研究观察到冻结视觉编码器的性能高于微调其参数[44]。然而，在训练OpenVLA时，我们微调了模型中的所有7B参数，包括SigLIP-DinoV2视觉骨干网络，因为我们在开发初期就发现微调视觉编码器会带来更高性能的VLA——这一发现在各种预训练的VLM和模型架构中都成立。我们将在下文讨论这些发现的细节。

Experimental setup and tasks. In this section, we report the performance of two VLA policies produced by fine-tuning two different pretrained models from the Prismatic VLMs [44] repository on BridgeData V2. The two pretrained models are named **SigLIP ViT-SO 224px** and **LLaVa v1.5 7B(Reproduction)**; see Karamcheti et al. [44] for details on their architectures and training mixtures. We evaluate both policies on various Bridge tasks shown in Table 10. Note that the evaluation configurations here differ from previously discussed Bridge evaluations, so the results are not directly comparable to results in other similar experiments.

实验装置与任务。在本节中，我们报告了通过在BridgeData V2上微调Prismatic VLMs [44] 存储库中的两个不同预训练模型而产生的两个VLA策略的性能。这两个预训练模型分别名为SigLIP ViT-SO 224px和LLaVa v1.5 7B (Reproduction)；有关其架构和训练混合的详细信息，请参见Karamcheti等人 [44]。我们在表10中显示的各种Bridge任务上评估了这两种策略。请注意，此处的评估配置与先前讨论的Bridge评估不同，因此结果无法与其它类似实验中的结果直接比较。

Results. Results for the fine-tuned vs. frozen vision encoder experiments are shown in Table 10. We find that for both VLAs tested, fine-tuning the vision encoder leads to significantly higher success rates across various tasks. Qualitatively, in some cases, deploying the frozen vision encoder policies leads to unstable robot behaviors that are clearly suboptimal. Consequently, we decided early on during development to not conduct further experimentation with frozen vision encoders.

结果。微调与冻结视觉编码器实验的结果如表10所示。我们发现，对于测试的两种VLA，微调视觉编码器可以显著提高各种任务的成功率。从定性角度来看，在某些情况下，部署冻结的视觉编码器策略会导致不稳定的机器人行为，这显然不是最优的。因此，我们在开发初期就决定不再进行冻结视觉编码器的进一步实验。因此，我们在开发初期就决定不再进行冻结视觉编码器的进一步实验。

D.4 Additional Quantized Inference Experiments: Disentangling Policy Performance and Model Inference Speed

D. 4 额外的量化推理实验：解耦策略性能和模型推理速度

In Section 5.3, we evaluated OpenVLA with different levels of precision at inference time: half precision (bfloat16), 8-bit quantization, and 4-bit quantization. 8-bit quantization led to lower BridgeData V2 performance relative to the other two approaches, and we hypothesized that the reduction in performance was caused by lower model inference speed from the operations used in 8-bit quantization. In this section, we conduct experiments to assess the veracity of this claim.

在第5.3节中，我们评估了OpenVLA在推理时不同精度水平下的表现：半精度（bfloat16）、8位量化和4位量化。相对于其他两种方法，8位量化导致BridgeData V2的性能降低，我们假设性能的降低是由于8位量化中使用的操作导致模型推理速度降低。在本节中，我们将进行实验以评估这一说法的真实性。

Specifically, we evaluate OpenVLA again with the three different levels of precision listed above, but now with blocking control. In other words, each action is fully executed on the robot before the next one is predicted by the policy and executed by the controller. This scheme controls system dynamics across methods with varying amounts of latency and thus allows us to test the quality of a policy’s action predictions, independent of its prediction speed. Effectively, the precision levels that have higher throughput – bfloat16 and 4-bit quantization – are forced to run slower to match the dynamics observed when deploying OpenVLA with 8-bit precision. Therefore, we expect OpenVLA’s performance with 8-bit precision to match the performance of bfloat16 and 4-bit precision under blocking control.

具体而言，我们再次使用上述三种不同的精度级别评估 OpenVLA，但这次采用阻塞控制。换句话说，每个动作都在机器人上完全执行，然后策略预测下一个动作并由控制器执行。该方案控制了具有不同延迟量的方法的系统动力学，因此使我们能够测试策略动作预测的质量，而与其预测速度无关。实际上，具有更高吞吐量的精度级别（bfloat16 和 4 位量化）被迫运行得更慢，以匹配使用 8 位精度部署 OpenVLA 时观察到的动力学。因此，我们期望 OpenVLA 在 8 位精度下的性能与在阻塞控制下 bfloat16 和 4 位精度的性能相匹配。

Table 10: Fine-tuned vs. frozen vision encoder experiment results. We evaluate the performance of fine-tuning (“Fine-Tuned”) vs. freezing the vision encoder (“Frozen Vision”) in two VLA policies built on top of two different pretrained VLMs from the Prismatic VLMs [44] repository. BridgeData V2 WidowX tasks shown here are performed in the same sink environment used for other Bridge experiments in this work (however, the initial environment configurations here differ, as these evaluations were conducted at an earlier stage in the project). We find that fine-tuning the vision encoder is crucial to obtain good policy performance. Certain frozen vision encoder evaluations were discontinued due to very poor (near-zero) performance and unstable robot behaviors. Among the evaluations where both frozen vision and fine-tuned approaches are tested, fine-tuning the vision encoder leads to 80.0% average success versus 46.7% average success from leaving it frozen.

表10：微调与冻结视觉编码器实验结果。我们评估了在两个VLA策略中微调（“微调”）与冻结视觉编码器（“冻结视觉”）的性能，这两个策略构建于Prismatic VLMs [44]仓库中两个不同的预训练VLM之上。此处显示的BridgeData V2 WidowX任务与本工作中其他Bridge实验在相同的sink环境中执行（然而，此处的初始环境配置不同，因为这些评估是在项目的早期阶段进行的）。我们发现，微调视觉编码器对于获得良好的策略性能至关重要。由于非常差（接近于零）的性能和不稳定的机器人行为，某些冻结视觉编码器评估已停止。在测试了冻结视觉和微调方法的评估中，微调视觉编码器带来了80.0%的平均成功率，而冻结视觉编码器带来的平均成功率为46.7%。

Task	# Trials	SigLIP ViT-SO 224px		LLaVa v1.5 7B (Reproduction)	
		Frozen Vision # Successes	Fine-Tuned # Successes	Frozen Vision # Successes	Fine-Tuned # Successes
Put Eggplant into Pot	10	7	10	5	9
Put Corn on Plate	10	10	9	0	9
Mean Success Rate		85	95	25	90
Put { Eggplant, Red Bottle } into Pot	4	2	4	–	3
Put { Blue Cup, Pink Cup } on Plate	4	0	0	–	0
Lift { Cheese, Red Chili Pepper }	4	0	3	–	2
Put { Strawberry, Lime } into Pot	4	1	0	–	3
Move { Sushi, Grapes }	4	3	4	–	3
Mean Success Rate		30	55	–	55

Experimental setup and tasks. We report the performance of OpenVLA with blocking control and quantized inference on the same subset of 8 BridgeData V2 tasks used in Appendix D.1 and Appendix D.2.

实验装置与任务。我们报告了OpenVLA在附录D. 1和附录D. 2中使用的8个BridgeData V2任务的相同子集上，采用分块控制和量化推理的性能。

Results. Quantized inference experiment results with blocking control are shown in Table 11. Unlike in Table 2, where 8-bit quantization led to the worst rollout performance due to low inference speed, here we observe that 8-bit quantization performs comparably to bfloat16 precision and 4-bit quantization given that we evaluate with blocking control to remove the influence of varying inference speeds on task performance. This confirms our hypothesis about the effect of inference speed on 8-bit quantization performance in previous experiments (when using non-blocking control). We also see no substantial performance degradation when using the lowest precision, 4-bit, as also observed in Section 5.3.

结果。表11展示了使用阻塞控制的量化推理实验结果。与表2不同，在表2中，由于推理速度慢，8位量化导致了最差的rollout性能，而在这里我们观察到，在采用阻塞控制以消除不同推理速度对任务性能的影响的情况下，8位量化的性能与bfloat16精度和4位量化相当。这证实了我们关于先前实验中推理速度对8位量化性能影响的假设（当使用非阻塞控制时）。我们也没有看到在使用最低精度（4位）时出现明显的性能下降，正如第5.3节中观察到的那样。

Table 11: **Quantized inference experiment results with blocking control.** We report the success rate and standard error of OpenVLA on various BridgeData V2 WidowX tasks with bfloat16 precision (the default approach), 8-bit quantization (int8), and 4-bit quantization (int4) at inference time. All average success rates have overlapping error bars, which suggests that all methods perform comparably.

表11：采用分块控制的量化推理实验结果。我们报告了OpenVLA在各种BridgeData V2 WidowX任务上，使用bfloat16精度（默认方法）、8位量化（int8）和4位量化（int4）在推理时的成功率和标准误差。所有平均成功率都具有重叠的误差条，这表明所有方法的性能相当。

Category	Task	# Trials	bfloat16 # Successes	int8 # Successes	int4 # Successes
Visual gen	Put Eggplant into Pot (Easy Version)	10	10	10	10
Visual gen	Put Eggplant into Pot	10	9	10	10
Visual gen	Put Cup from Counter into Sink	10	5	5	3
Motion gen	Lift Eggplant	10	8	7	7.5
Physical gen	Put Carrot on Plate	10	10	10	10
Physical gen	Lift AAA Battery	10	3	6	4
Semantic gen	Take Purple Grapes out of Pot	10	2	2	2
Language grounding	Put {Eggplant, Red Bottle} into Pot	10	9	9.5	8.5
Mean Success Rate			70.0 ± 5.1%	74.4 ± 4.9%	68.8 ± 5.2%

E LIBERO Simulation Experiments

E LIBERO 仿真实验

Our previous discussions in Section 5.2 and Section 5.3 focused on adapting OpenVLA to novel *real-world* robot setups and tasks. This section explores adapting OpenVLA to *simulated* robot setups and tasks, specifically utilizing the **LIBERO** benchmark [116]. Our experimentation in simulation offers two key advantages:

我们在第5.2节和第5.3节中之前的讨论主要集中于将OpenVLA适配到新的真实世界机器人设置和任务。本节探讨将OpenVLA适配到模拟机器人设置和任务，特别是利用LIBERO基准[116]。我们在模拟中的实验提供了两个主要优势：

1. Demonstration of versatility: We show that OpenVLA, despite having been pretrained exclusively on real-world robot data, can effectively adapt to simulated domains, overcoming potential disparities between real-world and simulated environments and dynamics.
1. 多功能性演示：我们展示了OpenVLA尽管仅在真实世界机器人数据上进行了预训练，但仍能有效适应模拟领域，克服了真实世界和模拟环境及动力学之间可能存在的差异。
2. Enhanced accessibility and reproducibility: Integration of OpenVLA into a publicly available simulation platform makes our model more accessible to other researchers, especially those who may not have access to robotic hardware. Additionally, simulated experiments are more easily reproduced than their real-world counterparts.
2. 增强可访问性和可重复性：将OpenVLA集成到公开可用的仿真平台中，使我们的模型更容易被其他研究人员访问，特别是那些可能无法获得机器人硬件的研究人员。此外，相比于真实世界的实验，仿真实验更容易重现。

We discuss the experimental setup in Appendix E.1 and the results in Appendix E.2. We release the materials required to reproduce the experiments along with the OpenVLA codebase.

我们在附录E.1中讨论了实验装置，并在附录E.2中讨论了结果。我们发布了重现实验所需的材料以及OpenVLA代码库。

E.1 LIBERO Simulation Experimental Setup

E.1 LIBERO 仿真实验设置

Simulation setup and tasks. The LIBERO benchmark [116] consists of four task suites designed for studying lifelong learning in robotic manipulation, and the original paper therefore investigates both forward and backward transfer to a

variety of tasks. In our experiments, we focus solely on supervised fine-tuning on the target task suite, measuring the performance of various policies trained via behavioral cloning on successful demonstrations of the tasks.

仿真设置与任务。LIBERO基准[116]由四个任务套件组成，旨在研究机器人操作中的终身学习，因此原始论文研究了对各种任务的正向和反向迁移。在我们的实验中，我们仅关注目标任务套件上的监督微调，测量通过行为克隆在任务的成功演示上训练的各种策略的性能。

We perform experiments with the following four task suites, which each contain 10 tasks with 50 human-teleoperated demonstrations each:

我们使用以下四个任务集进行实验，每个任务集包含 10 个任务，每个任务有 50 个人工遥操作演示：

- **LIBERO-Spatial** consists of the same set of objects but different layouts, and tests the model’s understanding of spatial relationships.
- LIBERO-Spatial包含相同的对象集合但不同的布局，并测试模型对空间关系的理解。
- **LIBERO-Object** consists of the same scene layouts but different objects, and tests the model’s understanding of object types.
- LIBERO-Object 包含相同的场景布局但不同的对象，并测试模型对对象类型的理解。
- **LIBERO-Goal** consists of the same objects and layouts but different task goals, and tests the model’s knowledge of different task-oriented behaviors.
- LIBERO-Goal包含相同的对象和布局，但具有不同的任务目标，并测试模型对不同面向任务行为的知识。
- **LIBERO-Long** (also called **LIBERO-10**) consists of *long-horizon* tasks with diverse objects, layouts, and tasks.
- LIBERO-Long（也称为LIBERO-10）由具有不同对象、布局和任务的长程任务组成。

We make the following modifications to each of the training datasets above:

我们对上述每个训练数据集进行了以下修改：

1. To accommodate methods requiring higher-resolution images (such as 256×256 px or 224×224 px), we regenerate all demonstrations at an increased resolution of 256×256 px. Originally, the dataset provided by the benchmark

consists of 128×128 px images. We find that simply upscaling these images to 256×256 px results in poor image quality. Therefore, we choose to begin with higher-resolution images, which can be downscaled as necessary, ensuring higher image quality across various resolution requirements. These higher-resolution images were obtained by stepping through the simulation environments with the actions stored in the provided human-collected demonstrations and saving the images rendered by the simulator.

1. 为了适应需要更高分辨率图像的方法（例如 256×256 px 或 224×224 px），我们以 256×256 px 的更高分辨率重新生成所有演示。最初，基准测试提供的数据集由 128×128 px 的图像组成。我们发现简单地将这些图像放大到 256×256 px 会导致图像质量下降。因此，我们选择从更高分辨率的图像开始，这些图像可以根据需要进行缩小，从而确保各种分辨率要求下更高的图像质量。这些更高分辨率的图像是通过步进模拟环境，使用存储在提供的人工收集演示中的动作，并保存模拟器渲染的图像来获得的。
2. We filter out all “no-op” actions from the dataset, i.e., actions that have near-zero magnitude in the translation and rotation components and do not change the state of the robot’s gripper. We find that this simple data cleaning step is crucial for highly expressive single-step policies such as OpenVLA, which otherwise learn to imitate these no-op actions and consequently freeze indefinitely at certain states during evaluation.
2. 我们从数据集中过滤掉所有“无操作”动作，即在平移和旋转分量中幅度接近于零且不改变机器人夹持器状态的动作。我们发现，这个简单的数据清理步骤对于像OpenVLA这样具有高度表达能力的单步策略至关重要，否则这些策略会学习模仿这些无操作动作，从而在评估期间无限期地冻结在某些状态。
3. We rotate all third-person images at both train and test time by 180 degrees because we observe that the LIBERO environments return images that are upside down on our hardware.
3. 我们在训练和测试时将所有第三人称图像旋转180度，因为我们观察到LIBERO环境返回的图像在我们的硬件上是倒置的。
4. Since we train policies via imitation learning, which expects demonstrations to be successful, we replay all demonstrations in the corresponding simulation environments and filter out the demonstrations that fail to complete the task (as determined by the environments’ success criteria). As a result, we remove 68 of 500 LIBERO-Spatial demonstrations, 46 of 500 LIBERO-Object demonstrations, 72 of 500 LIBERO-Goal demonstrations, and 121 of 500 LIBERO-Long demonstratinos.

4. 由于我们通过模仿学习训练策略，这要求演示是成功的，因此我们在相应的模拟环境中回放所有演示，并过滤掉未能完成任务的演示（由环境的成功标准确定）。因此，我们移除了500个LIBERO-Spatial演示中的68个，500个LIBERO-Object演示中的46个，500个LIBERO-Goal演示中的72个，以及500个LIBERO-Long演示中的121个。

5. For all methods in our comparisons, we only utilize the static third-person camera images; we do not use the wrist camera images that are additionally provided in the original datasets. This is for sake of having fair comparisons, as OpenVLA’s visual inputs only consist of third-person camera images.

5. 在我们的所有对比方法中，我们仅使用静态的第三人称视角相机图像；我们不使用原始数据集中额外提供的腕部相机图像。这是为了进行公平的比较，因为OpenVLA的视觉输入仅包含第三人称视角相机图像。

Comparisons. The methods that we compare include Diffusion Policy⁸ [3] trained from scratch, **Octo** [5] fine-tuned on the target dataset, and **OpenVLA** fine-tuned on the target dataset via LoRA ($r = 32$) as described in Section 5.3. Each policy is trained independently on each of the task suites above (rather than training a single policy on all four suites combined). All policies are trained with the same set of demonstrations, so all methods benefit from the data cleaning steps described above.

比较。我们比较的方法包括从头开始训练的Diffusion Policy⁸ [3]、在目标数据集上微调的Octo [5]，以及通过LoRA ($r = 32$) 在目标数据集上微调的OpenVLA（如第5.3节所述）。每个策略都在上述每个任务套件上独立训练（而不是在所有四个套件上组合训练单个策略）。所有策略都使用相同的演示集进行训练，因此所有方法都受益于上述数据清理步骤。

Evaluation details. To ensure lower variance in the experimental results, all methods are evaluated across 500 trials for each task suite, and the reported performance is the average success rate over three random seeds (resulting in 1500 total trials per statistic). Although we modify the training datasets, as described earlier, we do not change the test environments but rather use the same initial environment configurations provided by the original LIBERO benchmark.

评估细节。为了确保实验结果中较低的方差，所有方法在每个任务套件中都进行了 500 次试验的评估，并且报告的性能是三个随机种子上的平均成功率（导致每个统计量总共 1500 次试验）。尽管我们修改了训练数据集（如前所述），但我们并没有改变测试环境，而是使用了原始 LIBERO 基准提供的相同初始环境配置。

E.2 LIBERO Simulation Experimental Results

E. 2 LIBERO仿真实验结果

We present the LIBERO experimental results in Table 12. Importantly, we observe that OpenVLA can be effectively adapted to tasks in the LIBERO simulation environments, as it obtains highest average success rate and rank among the tested methods. However, we find that the overall margin between OpenVLA and the other methods are tighter here than in the real-world fine-tuning experiments discussed in Section 5.2. We attribute this to the fact that OpenVLA was pretrained with purely real-world robot data and no simulation data, which suggests that fine-tuning the model on simulated robot tasks may not be as effective as fine-tuning it on real-world tasks due to the domain gap between simulated and real-world environments and dynamics. We see evidence for this notion in the results obtained by Octo – another policy pretrained on large amounts of real-world robot data – which also only achieves a small boost in overall performance relative to a simple, strong baseline such as Diffusion Policy trained from scratch. We expect increased gains in performance for the pretrained and fine-tuned methods if simulation data is added to the pretraining data mixture.

我们在表12中展示了LIBERO的实验结果。重要的是，我们观察到OpenVLA可以有效地适应LIBERO模拟环境中的任务，因为它在测试方法中获得了最高的平均成功率和排名。然而，我们发现OpenVLA与其他方法之间的总体差距比第5.2节中讨论的真实世界微调实验中的差距要小。我们认为这是因为OpenVLA完全使用真实世界的机器人数据进行预训练，而没有使用任何模拟数据，这表明在模拟机器人任务上微调模型可能不如在真实世界任务上微调有效，这是由于模拟环境和真实世界环境以及动力学之间的领域差距所致。我们从Octo获得的结果中看到了这一概念的证据——Octo是另一种在大量真实世界机器人数据上进行预训练的策略——相对于从头开始训练的简单而强大的基线（如Diffusion Policy），它也仅实现了整体性能的少量提升。我们预计，如果将模拟数据添加到预训练数据混合中，预训练和微调方法的性能将获得更大的提升。

Table 12: **LIBERO simulation benchmark results.** We report the success rate (SR) and standard error of each method for the four task suites in the LIBERO benchmark, averaged over three random seeds with 500 trials each. In addition, we show the ranking of each method within each task suite, where a rank of 1 indicates the strongest method in the suite and a rank of 3 indicates the weakest method. (The average ranking is important to note since it informs which method may be most suitable to use as a default for a variety of tasks; it is more informative than the average success rate, which is not normalized by individual task suite difficulty.) Overall, we find that fine-tuned OpenVLA achieves highest average success rate and rank, followed by fine-tuned Octo and then Diffusion Policy trained from scratch.

表12: LIBERO模拟基准测试结果。我们报告了LIBERO基准测试中四个任务套件的每种方法的成功率（SR）和标准误差，结果是对三个随机种子进行平均，每个种子进行500次试验。此外，我们展示了每种方法在每个任务套件中的排名，其中排名1表示该套件中最强的方法，排名3表示最弱的方法。（平均排名非常重要，因为它表明哪种方法最适合用作各种任务的默认方法；它比平均成功率更有参考价值，因为平均成功率没有按各个任务套件的难度进行标准化。）总的来说，我们发现微调后的

OpenVLA实现了最高的平均成功率和排名，其次是微调后的Octo，然后是从头开始训练的Diffusion Policy。

	LIBERO-Spatial		LIBERO-Object		LIBERO-Goal		LIBERO-Long		Average	
	SR (%)	Rank (↓)	SR (%)	Rank (↓)	SR (%)	Rank (↓)	SR (%)	Rank (↓)	SR (%)	Rank (↓)
Diffusion Policy from scratch	78.3 ± 1.1%	3	92.5 ± 0.7%	1	68.3 ± 1.2%	3	50.5 ± 1.3%	3	72.4 ± 0.7%	2.5
Octo fine-tuned	78.9 ± 1.0%	2	85.7 ± 0.9%	3	84.6 ± 0.9%	1	51.1 ± 1.3%	2	75.1 ± 0.6%	2
OpenVLA fine-tuned (ours)	84.7 ± 0.9%	1	88.4 ± 0.8%	2	79.2 ± 1.0%	2	53.7 ± 1.3%	1	76.5 ± 0.6%	1.5